

Files modified per step — Part 0 → Part B (SMP)

Source: the local `xpack-development` branch of each repo (the integrated dry run), compared against the single-core baseline (`origin/xpack-development`). **Scope:** from **Part 0** (dissolve `devices`) through the end of **Part B** (the SMP infrastructure, Steps 14–23 collapsed into one `step/14`). Parts C (new architectures/platforms/tests, Steps 24–31) are **not** covered here.

Legend: **A** added · **M** modified · **D** deleted. Each entry names the file, where in it the change lands, and **why** it is needed. The authoritative recipe (with the fail-loud assertions) is in `scripts/smp/chunks/` and the per-step sections of `Implementation-SMP-Integration.md`; this document is the file-level cross-reference extracted from the branch.

Repos: **K** = `micro-os-plus-iii` (kernel), **P** = `micro-os-plus-iii-posix-arch`, **C** = `micro-os-plus-iii-cortexm`.

Part 0 — Dissolve the `devices` repository

Why (whole part): `origin/xpack-development` has six components; the `smp` branch adds a seventh, `micro-os-plus-iii-devices`. Upstream has no such repo and no place for one. Rather than make everyone depend on a new repo, its code is folded (history-preserving `git subtree`) into the architecture repos that already own it, so the integration proceeds with the six repos upstream has.

Repo	Change	Location	Why
P	A <code>soc/native/</code>	whole subtree	The POSIX "native SoC" (host SD-image back-end) moves from <code>devices</code> into the port that uses it.
P	A <code>drivers/include/</code> , <code>drivers/src/</code> , <code>drivers/fatfs/</code>	whole subtrees	The shared block/FatFs/USB drivers move in beside the port; history preserved.
P	M <code>src/...mailbox</code> , <code>fatfs</code>	spinlock / extent fixes	Carried-over driver fixes (atomic-spinlock mailbox sync; flatfs append-to-empty allocates an extent).
C	A <code>soc/stm32f4xx/</code> , <code>soc/rp2350/</code>	whole subtrees	The Cortex-M silicon support (STM32F4xx, RP2350) moves from <code>devices</code> into the Cortex-M port.

The BCM2837 SoC likewise dissolves into the aarch32/aarch64 repos (Part C); only the P/C migrations belong to this phase's branches.

Part A — Uniprocessor correctness (Steps 1–13, kernel unless noted)

These are the fixes that are correct and valuable **without** SMP; landing them first keeps each step bisectable and keeps Part B a pure SMP diff.

Step 1 — ISO C conformance, list iterators, exported suspend

- **M** `include/cmsis-plus/posix/dirent.h` — guard the `DIR`/`dirent` typedefs so a modern glibc (`__USE_MISC`/ISO C23) does not double-declare them. *Why: the native build's host libc otherwise errors.*
- **M** `include/cmsis-plus/utils/lists.h` — iterator conformance for the intrusive lists. *Why: used by newer standard-library expectations.*
- **M** `src/rtos/os-thread.cpp` — make the exported `thread::suspend()` symbol resolve (the out-of-line definition). *Why: a surgical lift — only this hunk of a file that Steps 9/14 also touch.*
- **Baseline kept:** `src/libc/stdlib/timex.c` — the smp hunk hides the prototype behind `#if !defined(__GLIBC__)`, which regresses glibc ≥ 2.44 (`-Werror=missing-prototypes`); the baseline already compiles everywhere.

Step 2 — Memory: overflow, usable size, calloc/realloc, block-pool

- **M** `include/cmsis-plus/rtos/os-memory.h`, `include/cmsis-plus/memory/first-fit-top.h` — declare `do_usable_size()` / overflow-checked `align_size()`.
- **M** `src/rtos/os-memory.cpp`, `src/memory/first-fit-top.cpp`, `src/memory/lifo.cpp`, `src/memory/block-pool.cpp`, `src/libc/stdlib/malloc.cpp` — the base-class definition, the override, and calloc/realloc overflow handling. *Why: the header declaration + base definition + override must land together or `memory_resource::do_usable_size` is an undefined reference. `first-fit-top.cpp` also gets a diagnostic-pragma wrap.*

Step 3 — C++17 aligned new/delete, static error category, chrono

- **M** `src/libcpp/new.cpp` — C++17 aligned `operator new`/`delete`.
- **M** `src/libcpp/system-error.cpp` — static (Meyers-singleton) error categories; **correction:** add a clang `-Wexit-time-destructors` suppression for those singletons.
- **M** `src/libcpp/chrono.cpp` — guard the cycle-count multiplication against overflow.

Step 4 — C wrapper: one-shot timer, polymorphic delete, 64-bit timeouts

- M `src/rtos/os-c-wrapper.cpp` — one-shot-timer default, polymorphic mutex/semaphore delete, 64-bit CMSIS-v1 timeouts. *Correction: the whole-file lift is stripped of the Step-9 `destroying` `static_assert` and the Step-20 SMP affinity blocks, so Step 4 carries only its own concern.*

Step 5 — POSIX I/O thread-safety

- M `src/posix-io/file-descriptors-manager.cpp` — mutex the descriptor table; **correction:** file-level clang `-Wunsafe-buffer-usage` suppression for the new bounds-checked array accesses.
- M `include/cmsis-plus/posix-io/file-system.h`, `include/cmsis-plus/posix-io/net-stack.h` — free-list locking.
- M `src/posix-io/block-device.cpp` — block-device size fix.

Step 6 — ARMv8-M guards, SecureFault, semihosting fstat

- M `include/cmsis-plus/arm/semihosting.h` — ARMv8-M-mainline guards.
- M `src/startup/exception-handlers.c` — add `SecureFault_Handler`.
- M `src/semihosting/c-syscalls-semihosting.cpp` — semihosting `fstat` fix. (*All portable kernel code.*)

Step 7 — timer callback outside critical section

- M `src/rtos/internal/os-lists.cpp`, `src/rtos/os-timer.cpp` — run the timer callback outside the critical section; periodic catch-up re-arm.
- M `include/cmsis-plus/rtos/os-thread.h` — **surgical** insert of only the handler-mode errno-scratch field (Step-9 `destroying=7` and Step-20 affinity are kept out).

Step 8 — mutex priority ceiling

- M `src/rtos/os-mutex.cpp` — set the priority ceiling before taking ownership; track the max-waiter boost; recompute on unlock; cache the owner across the uncritical window.

Step 9 — thread lifecycle (`state::destroying` , atomic join, detach, reaper)

- M `include/cmsis-plus/rtos/os-thread.h`, `src/rtos/os-thread.cpp`, `src/rtos/os-idle.cpp` — lifted as a **single-core projection** (`sc-project.py` strips the `OS_USE_SMP_SCHEDULER` / `NCPU>1` blocks); `os-thread.cpp` alone has ~37 SMP hunks, so hand-stripping is infeasible.
- M `include/cmsis-plus/rtos/os-c-decls.h` — **surgical:** lift the `destroying` enum value but keep Step-10's `void* clock` field out.

Step 10 — condition-variable atomicity

- **M** `src/rtos/os-condvar.cpp`, `include/cmsis-plus/rtos/os-condvar.h`, `include/cmsis-plus/diag/instrumentation.h` — rewritten `wait()` / `timed_wait()`, a `clock*` member, the CONDVAR suspend-cause constant.
- **M** `include/cmsis-plus/rtos/os-c-decls.h` — **surgical**: uncomment only `os_condvar_t`'s `void* clock` (the size counterpart; keep Step-16 SMP klock out).

Step 11 — std::thread functor lifetime & join

- **M** `src/libcpp/thread-cpp.h`, `include/cmsis-plus/estd/thread_internal.h` — join waits for the native handle; retained functor lifetime. (*thread-cpp.h* lives under `src/libcpp/`.)

Step 12 — message-queue reschedule

- **M** `src/rtos/os-mqueue.cpp` — reschedule after send/receive wakes a higher-priority thread (preemption on posix-arch).

Step 13 — high-resolution clock port synchronization (first cross-repo step)

- **K M** `include/cmsis-plus/rtos/os-decls.h`, `src/rtos/os-clocks.cpp` — declare and call `port::clock_highres::has_hardware_counter()`.
 - **C M** `include/cmsis-plus/rtos/port/os-inlines.h` — single-core projection: `has_hardware_counter()→false`, `hardware_counter()→0`, `cycles_since_tick()` ICSR `PENDSTSET` fix.
 - **P M** `include/cmsis-plus/rtos/port/os-inlines.h` — **surgical** insert of only the `clock_highres` block (`CLOCK_MONOTONIC`); the baseline file mixes later-step per-CPU content, so neither whole-copy nor projection is safe.
-

Part B — SMP infrastructure (Steps 14–23, one interwoven

step/14)

Why (whole part): the scheduler, thread state, idle, main and the C wrapper are too entangled to lift file-by-file once SMP is on, so Steps 14–23 land as one cohesive blob across the three repos, then the hardening fixes A–J are applied. This is the first point where `OS_USE_SMP_SCHEDULER` actually changes behaviour.

Step 14 — kernel (K)

- **M** `include/cmsis-plus/rtos/os-c-decls.h` — per-CPU C-ABI fields (the SMP klock etc.).
- **M** `include/cmsis-plus/rtos/os-sched.h` — the SMP scheduler interface (per-CPU current thread, port lock).
- **M** `include/cmsis-plus/rtos/os-thread.h` — thread CPU affinity + per-CPU bookkeeping.

- **M** `src/rtos/os-core.cpp` — the portable half of the SMP scheduler (per-CPU ready lists, the kernel spinlock, IPI reschedule); the context-switch decision that must not switch while the lock is held.
- **M** `src/rtos/os-c-wrapper.cpp` — the CMSIS-v1 affinity blocks stripped in Step 4.
- **M** `src/rtos/os-idle.cpp` — one idle thread per core.
- **M** `src/rtos/os-main.cpp` — the main thread pinned to core 0.
- **M** `src/rtos/os-thread.cpp` — the full SMP thread lifecycle.

Step 14 — Cortex-M port (C)

- **M** `include/cmsis-plus/rtos/port/os-c-decls.h`, `os-decls.h`, `os-inlines.h` — the SMP port contract (word size, atomics, the port lock) for the M-class SMP (M33/SSE-200).
- **M** `src/rtos/os-core.cpp` — the port half of the SMP scheduler on Cortex-M.

Step 14 — POSIX-arch port (P)

- **M** `include/cmsis-plus/rtos/port/os-c-decls.h`, `os-decls.h`, `os-inlines.h` — the host SMP port contract.
- **A** `include/host_cpu.hpp`, `src/host_cpu.cpp` — the host CPU model (threads-as-cores) that drives `os-core.cpp::initialize()`.
- **A** `include/exception_handler.hpp`, `src/exception_handler.cpp` — the port's exception/startup hooks the SMP core calls.
- **A** `src/free-store.cpp` — the host free-store bring-up (`os_startup_initialize_free_store`).
- **A** `include/hw_result.hpp`, `src/board-contract.cpp` — the shared PASS/FAIL contract + compile-time board-fact contract (also used by the ARM ports).
- **M** `src/rtos/os-core.cpp` — the posix-arch port half of the SMP scheduler.
- **M** `CMakeLists.txt` — register the new SMP sources **additively** in the port's INTERFACE target (not smp's standalone `UOS_SMP_DIR` model).

Corrections applied in Part B (fixes A–J): - **A/B** posix-arch `os-inlines.h`: `struct timespec tp;` → `timespec tp;` (baseline gate is `-Werror=redundant-tags`). - **C** posix-arch CMake: additive registration only; `board-contract.cpp` is board-specific (`#error`s without `PORT_GREETING`) so it is not built by the port itself. - **D** register `host_cpu.cpp` / `free-store.cpp` / `exception_handler.cpp` in the port target. - **F/G** `exception_handler.cpp`: drop a redundant `port_cpu_id` decl; add a port-side weak definition for the test-board `g_core_stage[]` symbol. - **H** kernel `os-core.cpp` / `os-thread.cpp`: wrap the `port_cpu_id` redundant decls with `#pragma GCC diagnostic ignored "-Wredundant-decls"`. - **I/J** posix-arch clang `-Weverything` hardening: port headers add `-Wc++98-compat-pedantic` / `-Wreserved-identifier` / `-Wunsafe-buffer-usage`; the SMP `.cpp` files get a comprehensive clang suppression block before their includes.

The port **release** commits (posix-arch `v1.1.0`, cortexm `v1.2.0`) bump only `package.json`; they are Step 24 (Part C) but carry the Part-B SMP port model, so the SMP source they publish is exactly the Step-14 content above.

Modifications organized by subject / theme / domain

The by-step view above is chronological (bisectable order). This chapter is the **cross-cut**: the same changes grouped by the *domain* they belong to, **one modification per line**. It is built by this rule — read every step's file list, and for each `(file, concern)` pair place it under the one domain that owns that concern; a file touched by several steps appears once per distinct modification, tagged with its step. Use this view to answer "what changed in domain X?"; use the by-step view to answer "what did step N do?".

Tag key: `[Sn]` = Step n · `K/P/C` = kernel / posix-arch / cortexm · `A/M/D` = added/modified/deleted.

1. Memory management

- `M` `include/cmsis-plus/rtos/os-memory.h` — declare overflow-checked `align_size()` + `do_usable_size()`. `[S2 K]`
- `M` `include/cmsis-plus/memory/first-fit-top.h` — `do_usable_size()` override declaration. `[S2 K]`
- `M` `src/rtos/os-memory.cpp` — base `memory_resource::do_usable_size()` + overflow guards. `[S2 K]`
- `M` `src/memory/first-fit-top.cpp` — the override; + diagnostic-pragma wrap. `[S2 K]`
- `M` `src/memory/lifo.cpp`, `src/memory/block-pool.cpp` — usable-size + pool fixes. `[S2 K]`
- `M` `src/libc/stdlib/malloc.cpp` — calloc/realloc overflow handling. `[S2 K]`

2. SMP scheduler core

- `M` `include/cmsis-plus/rtos/os-sched.h` — per-CPU scheduler interface (current thread, port lock). `[S14 K]`
- `M` `src/rtos/os-core.cpp` — portable half of the SMP scheduler: per-CPU ready lists, kernel spinlock, IPI reschedule, no-switch-while-locked rule. `[S14 K]`
- `M` `src/rtos/os-core.cpp` — port half of the SMP scheduler. `[S14 C]` `[S14 P]`
- `M` `src/rtos/os-idle.cpp` — one idle thread per core. `[S14 K]` (single-core projection `[S9 K]`)
- `M` `src/rtos/os-main.cpp` — main thread pinned to core 0. `[S14 K]`
- `M` `include/cmsis-plus/rtos/os-c-decls.h` — per-CPU C-ABI fields / SMP klock. `[S14 K]`

3. Thread lifecycle

- `M` `src/rtos/os-thread.cpp` — exported `thread::suspend()` symbol (surgical). `[S1 K]`
- `M` `include/cmsis-plus/rtos/os-thread.h` — handler-mode errno scratch (surgical). `[S7 K]`
- `M` `include/cmsis-plus/rtos/os-thread.h` + `src/rtos/os-thread.cpp` — `state::destroying`, atomic join, detach, reaper (single-core projection). `[S9 K]`
- `M` `include/cmsis-plus/rtos/os-c-decls.h` — the `destroying` enum value (surgical). `[S9 K]`

- M `include/cmsis-plus/rtos/os-thread.h` + `src/rtos/os-thread.cpp` — full SMP thread lifecycle + CPU affinity. [S14 K]
- M `src/libcpp/thread-cpp.h` + `include/cmsis-plus/estd/thread_internal.h` — `std::thread` join waits for the native handle; functor lifetime. [S11 K]

4. Synchronization primitives (mutex, condvar)

- M `src/rtos/os-mutex.cpp` — priority ceiling before ownership, max-waiter boost, unlock recompute, owner cache. [S8 K]
- M `src/rtos/os-condvar.cpp` + `include/cmsis-plus/rtos/os-condvar.h` — rewritten atomic `wait()` / `timed_wait()` + `clock*` member. [S10 K]
- M `include/cmsis-plus/rtos/os-c-decls.h` — `os_condvar_t`'s `void* clock` field (surgical). [S10 K]

5. Timers & message queue

- M `src/rtos/os-timer.cpp` — callback outside the critical section; periodic catch-up re-arm. [S7 K]
- M `src/rtos/internal/os-lists.cpp` — list support for the above. [S7 K]
- M `src/rtos/os-mqueue.cpp` — reschedule after send/receive wakes a higher-priority thread. [S12 K]

6. Clocks (high-resolution)

- M `include/cmsis-plus/rtos/os-decls.h` — declare `port::clock_highres::has_hardware_counter()`. [S13 K]
- M `src/rtos/os-clocks.cpp` — call it. [S13 K]
- M `include/cmsis-plus/rtos/port/os-inlines.h` — `has_hardware_counter()→false`, `hardware_counter()→0`, ICSR `PENDSTSET` fix (projection). [S13 C]
- M `include/cmsis-plus/rtos/port/os-inlines.h` — `clock_highres` block, `CLOCK_MONOTONIC` (surgical). [S13 P]

7. POSIX I/O

- M `include/cmsis-plus/posix/dirent.h` — guard `DIR` / `dirent` typedefs vs modern glibc. [S1 K]
- M `src/posix-io/file-descriptors-manager.cpp` — mutex the descriptor table (+ clang `-Wunsafe-buffer-usage`). [S5 K]
- M `include/cmsis-plus/posix-io/file-system.h`, `include/cmsis-plus/posix-io/net-stack.h` — free-list locking. [S5 K]
- M `src/posix-io/block-device.cpp` — block-device size fix. [S5 K]

8. C / C++ runtime (libc / libcpp)

- **M** `src/libcpp/new.cpp` — C++17 aligned `operator new / delete`. **[S3 K]**
- **M** `src/libcpp/system-error.cpp` — static error categories (+ clang `-Wexit-time-destructors`). **[S3 K]**
- **M** `src/libcpp/chrono.cpp` — cycle-multiplication overflow guard. **[S3 K]**
- **M** `include/cmsis-plus/utils/lists.h` — iterator conformance. **[S1 K]**
- **Baseline kept:** `src/libc/stdlib/timemgm.c` — smp hunk regresses glibc ≥ 2.44 , so not applied. **[S1 K]**

9. C wrapper (CMSIS-v1 C API)

- **M** `src/rtos/os-c-wrapper.cpp` — one-shot timer default, polymorphic delete, 64-bit timeouts (Step-9/20 content stripped). **[S4 K]**
- **M** `src/rtos/os-c-wrapper.cpp` — the SMP affinity blocks. **[S14 K]**

10. ARM startup / exceptions / semihosting

- **M** `src/startup/exception-handlers.c` — `SecureFault_Handler`, ARMv8-M guards. **[S6 K]**
- **M** `include/cmsis-plus/arm/semihosting.h` — ARMv8-M-mainline guards. **[S6 K]**
- **M** `src/semihosting/c-syscalls-semihosting.cpp` — semihosting `fstat` fix. **[S6 K]**

11. Port SMP contract (ABI headers)

- **M** `include/cmsis-plus/rtos/port/os-c-decls.h`, `os-decls.h`, `os-inlines.h` **[S14 C]** **[S14 P]** — word size, atomics, port lock, per-CPU current.
- **M** (posix-arch `os-inlines.h`) `struct timespec tp; → timespec tp;` — `-Werror=redundant-tags` (fix A/B). **[S14 P]**
- **M** (posix-arch port headers) add `-Wc++98-compat-pedantic` / `-Wreserved-identifier` / `-Wunsafe-buffer-usage` (fix I). **[S14 P]**

12. POSIX-arch host runtime (new SMP port files)

- **A** `include/host_cpu.hpp`, `src/host_cpu.cpp` — host CPU model (threads-as-cores). **[S14 P]**
- **A** `include/exception_handler.hpp`, `src/exception_handler.cpp` — port exception/startup hooks (+ fixes F/G: redundant `port_cpu_id`, weak `g_core_stage[]`). **[S14 P]**
- **A** `src/free-store.cpp` — host free-store bring-up. **[S14 P]**
- **A** `include/hw_result.hpp`, `src/board-contract.cpp` — PASS/FAIL + compile-time board-fact contract. **[S14 P]**

13. Diagnostics / instrumentation

- **M** `include/cmsis-plus/diag/instrumentation.h` — CONDVAR suspend-cause constant. **[S10 K]**

14. Toolchain hardening (cross-cutting clang `-Weverything` corrections)

- `src/libcpp/system-error.cpp` `-Wexit-time- destructors` [S3] · `src/posix-io/file-descriptors-manager.cpp` `-Wunsafe-buffer-usage` [S5] · `src/memory/first-fit-top.cpp` diagnostic wrap [S2] · posix-arch port headers + SMP `.cpp` comprehensive clang block (fixes I/J) [S14 P] · kernel `os-core.cpp` / `os-thread.cpp` `-Wredundant-decls` wrap (fix H) [S14 K] .

15. Build system

- M (posix-arch) `CMakeLists.txt` — register the new SMP sources additively in the INTERFACE target (fix C/D). [S14 P]
- M (posix-arch/cortexm) `package.json` — version bump for the SMP port release. [release]

16. Devices dissolution (Part 0)

- A (posix-arch) `soc/native/`, `drivers/{include,src,fatfs}/` — moved from the `devices` repo (history-preserving subtree). [P0 P]
- M (posix-arch) driver fixes — BCM2837 mailbox atomic spinlock; fatfs append-to-empty extent. [P0 P]
- A (cortexm) `soc/stm32f4xx/`, `soc/rp2350/` — moved from `devices` . [P0 C]

Appendix — full diffs per step

Generated from the integrated branch (`git show` of each step commit = baseline→step patch). Part 0 is a set of history-preserving subtree moves, so only its diffstat is shown; the file contents are unchanged by the move.

Part 0 — devices migration (diffstat only)

```
# posix-arch
4514a37 migrate(devices): soc/native -> soc/native (history preserved)
  2 files changed, 282 insertions(+)
f0bae5d migrate(devices): fatfs -> drivers/fatfs (history preserved)
  7 files changed, 8394 insertions(+)
ac8d5d6 migrate(devices): src -> drivers/src (history preserved)
  3 files changed, 2988 insertions(+)
52a32a5 migrate(devices): include -> drivers/include (history preserved)
  4 files changed, 704 insertions(+)
# cortexm
b484b99 migrate(devices): soc/stm32f4xx -> soc/stm32f4xx (history preserved)
  36 files changed, 63693 insertions(+)
3195eed migrate(devices): soc/rp2350 -> soc/rp2350 (history preserved)
  12 files changed, 9556 insertions(+)
```

smp-step(01): ISO C dirent, list iterators, exported suspend (timegm kept baseline)

```

diff --git a/include/cmsis-plus/posix/dirent.h b/include/cmsis-plus/posix/dirent.h
index 1f0f0dbe..19a5165c 100644
--- a/include/cmsis-plus/posix/dirent.h
+++ b/include/cmsis-plus/posix/dirent.h
@@ -54,7 +54,8 @@ extern "C"
    // and casted to DIR.
    typedef struct
    {
-       ;
+       int reserved; /* C forbids an empty struct; a named member also avoids
+           -Wextra-semi on the old `` null declaration. */
    } DIR;

    // -----
diff --git a/include/cmsis-plus/utils/lists.h b/include/cmsis-plus/utils/lists.h
index b8c642d9..f80da4dd 100644
--- a/include/cmsis-plus/utils/lists.h
+++ b/include/cmsis-plus/utils/lists.h
@@ -917,7 +917,7 @@ namespace os
    double_list_iterator<T, N, MP, U>::operator++ (int)
    {
        const auto tmp = *this;
-       node_ = static_cast<iterator_pointer> (node_->next);
+       node_ = static_cast<iterator_pointer> (node_->next ());
        return tmp;
    }

@@ -925,7 +925,7 @@ namespace os
    inline double_list_iterator<T, N, MP, U>&
    double_list_iterator<T, N, MP, U>::operator-- ()
    {
-       node_ = static_cast<iterator_pointer> (node_->prev);
+       node_ = static_cast<iterator_pointer> (node_->prev ());
        return *this;
    }

@@ -934,7 +934,7 @@ namespace os
    double_list_iterator<T, N, MP, U>::operator-- (int)
    {
        const auto tmp = *this;
-       node_ = static_cast<iterator_pointer> (node_->prev);
+       node_ = static_cast<iterator_pointer> (node_->prev ());
        return tmp;
    }

```

```
diff --git a/src/rtos/os-thread.cpp b/src/rtos/os-thread.cpp
index 6476782f..3b8f293a 100644
--- a/src/rtos/os-thread.cpp
+++ b/src/rtos/os-thread.cpp
@@ -1837,7 +1837,7 @@ namespace os
     *
     * @warning Cannot be invoked from Interrupt Service Routines.
     */
-    inline void
+    void
     suspend (void)
     {
         os::instrumentation::thread::suspend (_thread ());
     }
 }
```

smp-step(02): memory overflow, usable size, calloc/realloc

```

diff --git a/include/cmsis-plus/memory/first-fit-top.h b/include/cmsis-plus/memory/first-fit-top.h
index 39ac2791..c3942395 100644
--- a/include/cmsis-plus/memory/first-fit-top.h
+++ b/include/cmsis-plus/memory/first-fit-top.h
@@ -213,6 +213,14 @@ namespace os
    virtual std::size_t
    do_max_size (void) const noexcept override;

+    /**
+     * @brief Implementation of the function to get the usable block size.
+     * @param [in] addr Address of a previously allocated block.
+     * @return Number of usable bytes from `addr` to the end of its chunk.
+     */
+    virtual std::size_t
+    do_usable_size (void* addr) const noexcept override;
+
    /**
     * @brief Implementation of the function to reset the memory manager.
     * @par Parameters
diff --git a/include/cmsis-plus/rtos/os-memory.h b/include/cmsis-plus/rtos/os-memory.h
index 3d808899..3f567594 100644
--- a/include/cmsis-plus/rtos/os-memory.h
+++ b/include/cmsis-plus/rtos/os-memory.h
@@ -84,6 +84,10 @@ namespace os
    constexpr std::size_t
    align_size (std::size_t size, std::size_t align) noexcept
    {
+    if (size > static_cast<std::size_t> (-1) - (align - 1L))
+    {
+        return static_cast<std::size_t> (-1);
+    }
    return ((size) + (align)-1L) & ~((align)-1L);
    }

@@ -274,6 +278,21 @@ namespace os
    std::size_t
    max_size (void) const noexcept;

+    /**
+     * @brief Get the usable size of a previously allocated block.
+     * @param addr Address of a block returned by `allocate()`.
+     * @return Number of usable bytes, or 0 if the size is unknown.
+     *
+     * @details

```

```

+      * This is an extension used by `realloc()` to copy at most the
+      * old contents; the allocator is the only one who knows the real
+      * size of a block when the caller passed 0 to `deallocate()`.
+      *
+      * @see do_usable_size();
+      */
+      std::size_t
+      usable_size (void* addr) const noexcept;
+
+      /**
+       * @brief Set the out of memory handler.
+       * @param handler Pointer to new handler.
+@@ -422,6 +441,18 @@ namespace os
+       virtual std::size_t
+       do_max_size (void) const noexcept;
+
+      /**
+       * @brief Implementation of the function to get the usable block size.
+       * @param addr Address of a previously allocated block.
+       * @return Number of usable bytes, or 0 if the size is unknown.
+       *
+       * @details
+       * The default implementation returns 0 (unknown); allocators that
+       * keep the block size, like `first_fit_top`, override it.
+       */
+      virtual std::size_t
+      do_usable_size (void* addr) const noexcept;
+
+      /**
+       * @brief Implementation of the function to reset the memory manager.
+       * @par Parameters
+@@ -1342,6 +1373,15 @@ namespace os
+       return do_max_size ();
+   }
+
+   /**
+    * @see do_usable_size();
+    */
+   inline std::size_t
+   memory_resource::usable_size (void* addr) const noexcept
+   {
+       return do_usable_size (addr);
+   }
+
+   /**

```



```

        * @see do_reset();
        */

diff --git a/src/libc/stdlib/malloc.cpp b/src/libc/stdlib/malloc.cpp
index 9ff93d07..91ffcc4a 100644
--- a/src/libc/stdlib/malloc.cpp
+++ b/src/libc/stdlib/malloc.cpp
@@ -160,6 +160,17 @@ @@ calloc (size_t nelem, size_t elbytes)
    return nullptr;
}

+ // Reject requests whose total size would overflow `size_t`. Without this
+ // guard the product wraps to a small value (e.g. on a 32-bit target
+ // `calloc(0x10001, 0x10000)` becomes 0) and a tiny block is returned while
+ // the caller believes the full, huge array was allocated. `elbytes` is
+ // known non-zero here, so the division is safe.
+ if (nelem > static_cast<std::size_t> (-1) / elbytes)
+ {
+     errno = ENOMEM;
+     return nullptr;
+ }
+
void* mem;
{
    // ----- Begin of critical section -----
@@ -294,7 +305,16 @@ @@ realloc (void* ptr, size_t bytes)
    mem = estd::pmr::get_default_resource ()->allocate (bytes);
    if (mem != nullptr)
    {
-        memcpy (mem, ptr, bytes);
+        // POSIX requires the contents to be preserved "up to the lesser of
+        // the new and old sizes". The old size is queried from the allocator
+        // (the bare-metal default resource is a `first_fit_top`, which keeps
+        // it in the chunk header). Copying `bytes` unconditionally would read
+        // past the end of the old block whenever the new size is larger,
+        // i.e. on every growing `realloc()`.
+        std::size_t old_bytes
+            = estd::pmr::get_default_resource ()->usable_size (ptr);
+        std::size_t copy_bytes = (bytes < old_bytes) ? bytes : old_bytes;
+        memcpy (mem, ptr, copy_bytes);
+        estd::pmr::get_default_resource ()->deallocate (ptr, 0);
    }
    else
diff --git a/src/memory/block-pool.cpp b/src/memory/block-pool.cpp
index b3f583d1..a2432310 100644
--- a/src/memory/block-pool.cpp

```

```

+++ b/src/memory/block-pool.cpp
@@ -168,7 +168,7 @@ namespace os
        pool_addr_, align_sz);

        // std::align() will fail if it cannot fit the adjusted block size.
-       if (res != nullptr)
+       if (res == nullptr)
        {
            assert (res != nullptr);
        }
diff --git a/src/memory/first-fit-top.cpp b/src/memory/first-fit-top.cpp
index 09478a00..cce94591 100644
--- a/src/memory/first-fit-top.cpp
+++ b/src/memory/first-fit-top.cpp
@@ -122,13 +122,32 @@ namespace os
    {
        using namespace os;

+
+       if (bytes > total_bytes_)
+       {
+           return nullptr;
+       }
+
        std::size_t block_padding = calc_block_padding (alignment);
        std::size_t alloc_size = rtos::memory::align_size (bytes, chunk_align);
+       if (alloc_size == static_cast<std::size_t> (-1)
+           || alloc_size > total_bytes_)
+       {
+           return nullptr;
+       }
        alloc_size += block_padding;
        alloc_size += chunk_offset;
+       if (alloc_size > total_bytes_)
+       {
+           return nullptr;
+       }

        std::size_t block_minchunk = calc_block_minchunk (block_padding);
        alloc_size = os::rtos::memory::max (alloc_size, block_minchunk);
+       if (alloc_size > total_bytes_)
+       {
+           return nullptr;
+       }

```

```

        chunk_t* chunk;

@@ -428,6 +447,37 @@ namespace os
        return total_bytes_;
    }

#pragma GCC diagnostic push
#pragma GCC diagnostic ignored "-Wcast-align"
#ifdef __clang__
#pragma clang diagnostic ignored "-Wunsafe-buffer-usage"
#endif
+    std::size_t
+    first_fit_top::do_usable_size (void* addr) const noexcept
+    {
+        // Recover the chunk header placed immediately before the payload.
+        chunk_t* chunk = reinterpret_cast<chunk_t*> (static_cast<char*> (addr)
+                                                    - chunk_offset);
+
+        // For an aligned block, the adjusted header stores the (negative)
+        // alignment offset as its size; step back to the real chunk header
+        // exactly as do_deallocate() does.
+        if (static_cast<std::ptrdiff_t> (chunk->size) < 0)
+        {
+            chunk = reinterpret_cast<chunk_t*> (
+                reinterpret_cast<char*> (chunk)
+                + static_cast<std::ptrdiff_t> (chunk->size));
+        }
+
+        // Usable bytes are those from the given address to the end of the
+        // chunk; this naturally accounts for any alignment slack between the
+        // chunk payload and the returned pointer.
+        return static_cast<std::size_t> (
+            reinterpret_cast<char*> (chunk) + chunk->size
+            - static_cast<char*> (addr));
+    }
#pragma GCC diagnostic pop
+
+    void*
+    first_fit_top::internal_align_ (chunk_t* chunk, std::size_t bytes,
+                                    std::size_t alignment)

diff --git a/src/memory/lifo.cpp b/src/memory/lifo.cpp
index 3519fb3e..9924fbf2 100644
--- a/src/memory/lifo.cpp
+++ b/src/memory/lifo.cpp
@@ -70,13 +70,31 @@ namespace os

```

```

void*
lifo::do_allocate (std::size_t bytes, std::size_t alignment)
{
+   if (bytes > total_bytes_)
+   {
+       return nullptr;
+   }
+
    std::size_t block_padding = calc_block_padding (alignment);
    std::size_t alloc_size = rtos::memory::align_size (bytes, chunk_align);
+   if (alloc_size == static_cast<std::size_t> (-1)
+       || alloc_size > total_bytes_)
+   {
+       return nullptr;
+   }
    alloc_size += block_padding;
    alloc_size += chunk_offset;
+   if (alloc_size > total_bytes_)
+   {
+       return nullptr;
+   }

    std::size_t block_minchunk = calc_block_minchunk (block_padding);
    alloc_size = os::rtos::memory::max (alloc_size, block_minchunk);
+   if (alloc_size > total_bytes_)
+   {
+       return nullptr;
+   }

    chunk_t* chunk = nullptr;

@@ -126,6 +144,14 @@ namespace os
        // If this was the last chunk, the free list is empty.
    }
}
+   else
+   {
+       // The head chunk is smaller than the request, so this arena
+       // cannot satisfy it. Clear `chunk` so the loop below falls
+       // through to the out-of-memory handler (or returns nullptr)
+       // instead of handing the too-small chunk to internal_align().
+       chunk = nullptr;
+   }
}

```

```

        if (chunk != nullptr)
diff --git a/src/rtos/os-memory.cpp b/src/rtos/os-memory.cpp
index 7b3c9baa..d5f89855 100644
--- a/src/rtos/os-memory.cpp
+++ b/src/rtos/os-memory.cpp
@@ -438,6 +438,23 @@ namespace os
    return 0;
}

+ /**
+  * @details
+  * The default implementation of this virtual function returns
+  * zero, meaning the usable size is not known.
+  *
+  * Override this function to return the actual size.
+  *
+  * @par Standard compliance
+  * Extension to standard.
+  */
+ std::size_t
+ memory_resource::do_usable_size (void* addr) const noexcept
+ {
+     static_cast<void> (addr);
+     return 0;
+ }
+
+ /**
+  * @details
+  * The default implementation of this virtual function

```

smp-step(03): C++17 aligned operator new/delete, static system_error category, chrono overflow

```

diff --git a/src/libcpp/chrono.cpp b/src/libcpp/chrono.cpp
index dfbe1829..6246956a 100644
--- a/src/libcpp/chrono.cpp
+++ b/src/libcpp/chrono.cpp
@@ -113,13 +113,15 @@ namespace os
 #endif

    // The duration is the number of sum of SysTick ticks plus the current
    // count of CPU cycles (computed from the SysTick counter).
-    // Notice: a more exact solution would be to compute
-    // ticks * divisor + cycles, but this severely reduces the
-    // range of ticks.
+    // Decompose into seconds and remainder to avoid 64-bit overflow
+    // on cycles * 1e9 after ~13 minutes.
+    uint64_t freq = rtos::hrclock.input_clock_frequency_hz ();
+    uint64_t sec = cycles / freq;
+    uint64_t rem = cycles % freq;
+    uint64_t ns = sec * 1000000000ULL + (rem * 1000000000ULL) / freq;
    return time_point{
        duration{
-            duration{ cycles * 1000000000ULL
-                / rtos::hrclock.input_clock_frequency_hz () }
+            duration{ ns }
+                + realtime_clock::startup_time_point.time_since_epoch () } //
        };

    #pragma GCC diagnostic pop
diff --git a/src/libcpp/new.cpp b/src/libcpp/new.cpp
index 098df1d9..5203ee12 100644
--- a/src/libcpp/new.cpp
+++ b/src/libcpp/new.cpp
@@ -288,6 +288,135 @@ void* __attribute__ ((weak)) operator new[] (std::size_t bytes,

    // -----

+/**
+ * @ingroup cmsis-plus-rtos-memres
+ * @brief Allocate over-aligned space for a new object instance.
+ * @param bytes Number of bytes to allocate.
+ * @param alignment Required alignment (C++17 `std::align_val_t`).
+ * @return Pointer to aligned allocated object.
+ *
+ * @details
+ * C++17 calls this overload for types whose alignment exceeds
+ * `alignof(std::max_align_t)`. The requested alignment is passed through to
+ * the RTOS default memory resource; the bare-metal `first_fit_top` honours

```

```

+ * it, whereas the plain `size_t` overload only promises `max_align`.
+ *
+ * @warning Cannot be invoked from Interrupt Service Routines.
+ */
+void* __attribute__ ((weak))
+operator new (std::size_t bytes, std::align_val_t alignment)
+{
+    assert (!rtos::interrupts::in_handler_mode ());
+    if (bytes == 0)
+    {
+        bytes = 1;
+    }
+
+    // ----- Begin of critical section -----
+    rtos::scheduler::critical_section scs;
+
+    while (true)
+    {
+        void* mem = estd::pmr::get_default_resource ()->allocate (
+            bytes, static_cast<std::size_t> (alignment));
+
+        if (mem != nullptr)
+        {
+            return mem;
+        }
+
+        if (new_handler_)
+        {
+            new_handler_ ();
+        }
+        else
+        {
+            estd::__throw_bad_alloc ();
+        }
+    }
+
+    // ----- End of critical section -----
+}
+
+/**
+ * @ingroup cmsis-plus-rtos-memres
+ * @brief Allocate over-aligned space for a new object instance (nothrow).
+ * @param bytes Number of bytes to allocate.
+ * @param alignment Required alignment (C++17 `std::align_val_t`).
+ * @param nothrow (unused)

```



```

+ * @return Pointer to aligned allocated object or nullptr.
+ *
+ * @warning Cannot be invoked from Interrupt Service Routines.
+ */
+void* __attribute__ ((weak))
+operator new (std::size_t bytes, std::align_val_t alignment,
+             const std::nothrow_t& nothrow __attribute__ ((unused))) noexcept
+{
+    assert (!rtos::interrupts::in_handler_mode ());
+    if (bytes == 0)
+    {
+        bytes = 1;
+    }
+
+    // ----- Begin of critical section -----
+    rtos::scheduler::critical_section scs;
+
+    while (true)
+    {
+        void* mem = estd::pmr::get_default_resource ()->allocate (
+            bytes, static_cast<std::size_t> (alignment));
+
+        if (mem != nullptr)
+        {
+            return mem;
+        }
+
+        if (new_handler_)
+        {
+            new_handler_ ();
+        }
+        else
+        {
+            break; // return nullptr
+        }
+    }
+
+    // ----- End of critical section -----
+
+    return nullptr;
+}
+
+/**
+ * @ingroup cmsis-plus-rtos-memres
+ * @brief Allocate over-aligned space for an array of new object instances.

```

```

+ * @param bytes Number of bytes to allocate.
+ * @param alignment Required alignment (C++17 `std::align_val_t`).
+ * @return Pointer to aligned allocated object.
+ *
+ * @warning Cannot be invoked from Interrupt Service Routines.
+ */
+void* __attribute__ ((weak))
+operator new[] (std::size_t bytes, std::align_val_t alignment)
+{
+ return ::operator new (bytes, alignment);
+}
+
+/**
+ * @ingroup cmsis-plus-rtos-memres
+ * @brief Allocate over-aligned space for an array of new object
+ * instances (nothrow).
+ *
+ * @warning Cannot be invoked from Interrupt Service Routines.
+ */
+void* __attribute__ ((weak))
+operator new[] (std::size_t bytes, std::align_val_t alignment,
+               const std::nothrow_t& nothrow __attribute__ ((unused))) noexcept
+{
+ return ::operator new (bytes, alignment, std::nothrow);
+}
+
+// -----
+
+/**
+ * @ingroup cmsis-plus-rtos-memres
+ * @brief Deallocate the dynamically allocated object instance.
+@@ -516,6 +645,116 @@ void __attribute__ ((weak)) operator delete[] (void* ptr, const std::nothrow_t
+ ::operator delete (ptr, nothrow);
+ }
+
+// -----
+
+/**
+ * @ingroup cmsis-plus-rtos-memres
+ * @brief Deallocate an over-aligned dynamically allocated object.
+ * @param ptr Pointer to object.
+ * @param alignment Required alignment (C++17 `std::align_val_t`).
+ * @par Returns
+ * Nothing.
+ */

```

```

+ * @details
+ * Counterpart of `operator new(std::size_t, std::align_val_t)`.
+ *
+ * @warning Cannot be invoked from Interrupt Service Routines.
+ */
+void __attribute__((weak))
+operator delete (void* ptr, std::align_val_t alignment) noexcept
+{
+    assert (!rtos::interrupts::in_handler_mode ());
+
+    if (ptr)
+    {
+        // ----- Begin of critical section -----
+        rtos::scheduler::critical_section scs;
+
+        estd::pmr::get_default_resource ()->deallocate (
+            ptr, 0, static_cast<std::size_t> (alignment));
+        // ----- End of critical section -----
+    }
+}
+
+/**
+ * @ingroup cmsis-plus-rtos-memres
+ * @brief Deallocate an over-aligned dynamically allocated object (sized).
+ * @param ptr Pointer to object.
+ * @param bytes Number of bytes to deallocate.
+ * @param alignment Required alignment (C++17 `std::align_val_t`).
+ * @par Returns
+ * Nothing.
+ *
+ * @warning Cannot be invoked from Interrupt Service Routines.
+ */
+void __attribute__((weak))
+operator delete (void* ptr, std::size_t bytes,
+                std::align_val_t alignment) noexcept
+{
+    assert (!rtos::interrupts::in_handler_mode ());
+
+    if (ptr)
+    {
+        // ----- Begin of critical section -----
+        rtos::scheduler::critical_section scs;
+
+        estd::pmr::get_default_resource ()->deallocate (
+            ptr, bytes, static_cast<std::size_t> (alignment));

```

```

+    // ----- End of critical section -----
+    }
+}
+
+/**
+ * @ingroup cmsis-plus-rtos-memres
+ * @brief Deallocate an over-aligned dynamically allocated array.
+ *
+ * @warning Cannot be invoked from Interrupt Service Routines.
+ */
+void __attribute__((weak))
+operator delete[] (void* ptr, std::align_val_t alignment) noexcept
+{
+    ::operator delete (ptr, alignment);
+}
+
+/**
+ * @ingroup cmsis-plus-rtos-memres
+ * @brief Deallocate an over-aligned dynamically allocated array (sized).
+ *
+ * @warning Cannot be invoked from Interrupt Service Routines.
+ */
+void __attribute__((weak))
+operator delete[] (void* ptr, std::size_t bytes,
+                  std::align_val_t alignment) noexcept
+{
+    ::operator delete (ptr, bytes, alignment);
+}
+
+/**
+ * @ingroup cmsis-plus-rtos-memres
+ * @brief Deallocate an over-aligned dynamically allocated object (nothrow).
+ *
+ * @warning Cannot be invoked from Interrupt Service Routines.
+ */
+void __attribute__((weak))
+operator delete (void* ptr, std::align_val_t alignment,
+                const std::nothrow_t& nothrow __attribute__((unused))) noexcept
+{
+    ::operator delete (ptr, alignment);
+}
+
+/**
+ * @ingroup cmsis-plus-rtos-memres
+ * @brief Deallocate an over-aligned dynamically allocated array (nothrow).

```

```

+ *
+ * @warning Cannot be invoked from Interrupt Service Routines.
+ */
+void __attribute__ ((weak))
+operator delete[] (void* ptr, std::align_val_t alignment,
+                  const std::nothrow_t& nothrow __attribute__ ((unused))) noexcept
+{
+    ::operator delete[] (ptr, alignment);
+}
+
+    // error: end of file with unbalanced grouping commands
+/*
+ * @}
diff --git a/src/libcpp/system-error.cpp b/src/libcpp/system-error.cpp
index c11ad9f0..dd7bf829 100644
--- a/src/libcpp/system-error.cpp
+++ b/src/libcpp/system-error.cpp
@@ -103,17 +103,33 @@ namespace os

#pragma GCC diagnostic pop

+    // `std::error_code` stores a pointer to its `error_category`, and the
+    // `error_code` (via the thrown `std::system_error`) outlives the throw
+    // expression. The category must therefore have static storage duration;
+    // a temporary would dangle. Function-local statics are constructed on
+    // first use and live until exit.
+    static const std::error_category&
+    get_system_error_category (void) noexcept
+    {
+        static const system_error_category category;
+        return category;
+    }
+
+    static const std::error_category&
+    get_cmsis_error_category (void) noexcept
+    {
+        static const cmsis_error_category category;
+        return category;
+    }
+
+endif

    void
    __throw_system_error (int ev, const char* what_arg)
    {

```

```

    #if defined(__EXCEPTIONS)
-        // error: copying parameter of type 'os::estd::system_error_category'
-        // when binding a reference to a temporary would invoke a deleted
-        // constructor in C++98 [-Werror,-Wc++98-compat-bind-to-temporary-copy]
-        throw std::system_error (std::error_code (ev, system_error_category ()),
-                                what_arg);
+        throw std::system_error (
+            std::error_code (ev, get_system_error_category ()), what_arg);
    #else
        trace_printf ("system_error(%d, %s)\n", ev, what_arg);
        std::abort ();
@@ -124,10 +140,7 @@ namespace os
    __throw_cmsis_error (int ev, const char* what_arg)
    {
        #if defined(__EXCEPTIONS)
-        // error: copying parameter of type 'os::estd::cmsis_error_category' when
-        // binding a reference to a temporary would invoke a deleted constructor
-        // in C++98 [-Werror,-Wc++98-compat-bind-to-temporary-copy]
-        throw std::system_error (std::error_code (ev, cmsis_error_category ()),
+        throw std::system_error (std::error_code (ev, get_cmsis_error_category ()),
                                what_arg);
        #else
            trace_printf ("system_error(%d, %s)\n", ev, what_arg);

```

smp-step(04): one-shot timer default, polymorphic mutex/semaphore delete, 64-bit CMSIS-v1 timeouts

```

diff --git a/src/rtos/os-c-wrapper.cpp b/src/rtos/os-c-wrapper.cpp
index fc898c72..4bd9e1bb 100644
--- a/src/rtos/os-c-wrapper.cpp
+++ b/src/rtos/os-c-wrapper.cpp
@@ -1504,7 +1504,9 @@ os_timer_construct (os_timer_t* timer, const char* name,
    assert (timer != nullptr);
    if (attr == nullptr)
    {
-       attr = (const os_timer_attr_t*)&timer::periodic_initializer;
+       // The C++ and CMSIS defaults are one-shot timers; only an explicit
+       // periodic attribute (os_timer_attr_get_periodic()) should repeat.
+       attr = (const os_timer_attr_t*)&timer::once_initializer;
    }
    new (timer)
        rtos::timer (name, (timer::func_t)function, (timer::func_args_t)args,
@@ -1545,7 +1547,9 @@ os_timer_new (const char* name, os_timer_func_t function,
    {
        if (attr == nullptr)
        {
-       attr = (const os_timer_attr_t*)&timer::periodic_initializer;
+       // The C++ and CMSIS defaults are one-shot timers; only an explicit
+       // periodic attribute (os_timer_attr_get_periodic()) should repeat.
+       attr = (const os_timer_attr_t*)&timer::once_initializer;
        }
        return reinterpret_cast<os_timer_t*> (
            new rtos::timer (name, (timer::func_t)function, (timer::func_args_t)args,
@@ -1773,7 +1777,21 @@ void
os_mutex_delete (os_mutex_t* mutex)
    {
        assert (mutex != nullptr);
-       delete reinterpret_cast<rtos::mutex*> (mutex);
+
+       // `mutex` and `mutex_recursive` share the same C storage type and the base
+       // destructor is intentionally non-virtual (so that `os_mutex_t` keeps the
+       // same size as `rtos::mutex`). The concrete type is recorded in the object
+       // itself, so delete through the derived type and run the correct
+       // destructor instead of relying on undefined behaviour.
+       if (reinterpret_cast<rtos::mutex*> (mutex)->type ()
+           == rtos::mutex::type::recursive)
+       {
+           delete reinterpret_cast<rtos::mutex_recursive*> (mutex);
+       }
+       else
+       {

```



```

+     delete reinterpret_cast<rtos::mutex*> (mutex);
+ }
+ }

/**
@@ -2325,7 +2343,22 @@ void
os_semaphore_delete (os_semaphore_t* semaphore)
{
    assert (semaphore != nullptr);
- delete reinterpret_cast<rtos::semaphore*> (semaphore);
+
+ // `semaphore_binary` and `semaphore_counting` share the same C storage
+ // type and the base destructor is intentionally non-virtual (so that
+ // `os_semaphore_t` keeps the same size as `rtos::semaphore`). The binary
+ // form is the one whose maximum count is 1, so delete through the derived
+ // type and run the correct destructor instead of relying on undefined
+ // behaviour. Both derived destructors are empty, so the routing is safe
+ // even if a counting semaphore is configured with a maximum of 1.
+ if (reinterpret_cast<rtos::semaphore*> (semaphore)->max_value () == 1)
+ {
+     delete reinterpret_cast<rtos::semaphore_binary*> (semaphore);
+ }
+ else
+ {
+     delete reinterpret_cast<rtos::semaphore_counting*> (semaphore);
+ }
+ }

/**
@@ -3505,7 +3538,6 @@ osThreadCreate (const osThreadDef_t* thread_def, void* args)
    thread::attributes attr;
    attr.th_priority = thread_def->tpriority;
    attr.th_stack_size_bytes = thread_def->stacksize;
-
- // Creating thread with invalid priority should fail (validator requirement).
- if (thread_def->tpriority >= osPriorityError)
- {
@@ -3739,7 +3771,7 @@ osDelay (uint32_t millisec)
    }

    result_t res = sysclock.sleep_for (
-     clock_systick::ticks_cast ((uint64_t)(millisec * 1000u)));
+     clock_systick::ticks_cast (((uint64_t) millisec * 1000u)));

    if (res == ETIMEDOUT)

```

```

    {
@@ -3788,7 +3820,7 @@ osWait (uint32_t millisec)
    }

    result_t res = sysclock.wait_for (
-        clock_systick::ticks_cast ((uint64_t)(millisec * 1000u)));
+        clock_systick::ticks_cast (((uint64_t) millisec * 1000u)));

    // TODO: return events
    if (res == ETIMEDOUT)
@@ -3867,7 +3899,7 @@ osTimerStart (osTimerId timer_id, uint32_t millisec)

    result_t res
        = (reinterpret_cast<rtos::timer&> (*timer_id))
-        .start (clock_systick::ticks_cast ((uint64_t)(millisec * 1000u)));
+        .start (clock_systick::ticks_cast (((uint64_t) millisec * 1000u)));

    if (res == result::ok)
    {
@@ -4051,7 +4083,7 @@ osSignalWait (int32_t signals, uint32_t millisec)
    {
        res = this_thread::flags_timed_wait (
            (flags::mask_t)signals,
-            clock_systick::ticks_cast ((uint64_t)(millisec * 1000u)),
+            clock_systick::ticks_cast (((uint64_t) millisec * 1000u)),
            (flags::mask_t*)&event.value.signals);
    }

@@ -4162,7 +4194,7 @@ osMutexWait (osMutexId mutex_id, uint32_t millisec)
    {
        ret = (reinterpret_cast<rtos::mutex&> (*mutex_id))
            .timed_lock (
-            clock_systick::ticks_cast ((uint64_t)(millisec * 1000u)));
+            clock_systick::ticks_cast (((uint64_t) millisec * 1000u)));
        // osErrorTimeoutResource:
    }

@@ -4355,7 +4387,7 @@ osSemaphoreWait (osSemaphoreId semaphore_id, uint32_t millisec)
    {
        res = (reinterpret_cast<rtos::semaphore&> (*semaphore_id))
            .timed_wait (
-            clock_systick::ticks_cast ((uint64_t)(millisec * 1000u)));
+            clock_systick::ticks_cast (((uint64_t) millisec * 1000u)));
        if (res == ETIMEDOUT)
        {

```

```

        return 0;
@@ -4645,7 +4677,7 @@ osMessagePut (osMessageQId queue_id, uint32_t info, uint32_t millisec)
    res = (reinterpret_cast<message_queue*> (*queue_id))
        .timed_send (
            (const char*)&info, sizeof (uint32_t),
-            clock_systick::ticks_cast ((uint64_t)(millisec * 1000u)),
+            clock_systick::ticks_cast (((uint64_t) millisec * 1000u)),
            0);
    // osOK, osErrorTimeoutResource, osErrorParameter
}
@@ -4743,7 +4775,7 @@ osMessageGet (osMessageQId queue_id, uint32_t millisec)
    res = (reinterpret_cast<message_queue*> (*queue_id))
        .timed_receive (
            (char*)&event.value.v, sizeof (uint32_t),
-            clock_systick::ticks_cast ((uint64_t)(millisec * 1000u)),
+            clock_systick::ticks_cast (((uint64_t) millisec * 1000u)),
            nullptr);
    // result::event_message when message;
    // result::event_timeout when timeout;
@@ -4881,7 +4913,7 @@ osMailAlloc (osMailQId mail_id, uint32_t millisec)
    }
    ret = (reinterpret_cast<memory_pool*> (mail_id->pool))
        .timed_alloc (
-            clock_systick::ticks_cast ((uint64_t)(millisec * 1000u)));
+            clock_systick::ticks_cast (((uint64_t) millisec * 1000u)));
    }
    #pragma GCC diagnostic pop
    return ret;
@@ -5044,7 +5076,7 @@ osMailGet (osMailQId mail_id, uint32_t millisec)
    res = (reinterpret_cast<message_queue*> (mail_id->queue))
        .timed_receive (
            (char*)&event.value.p, sizeof (void*),
-            clock_systick::ticks_cast ((uint64_t)(millisec * 1000u)),
+            clock_systick::ticks_cast (((uint64_t) millisec * 1000u)),
            nullptr);
    // osEventMail for ok, osEventTimeout
}

```

smp-step(05): POSIX I/O fd-manager mutexing, free-list locking, block-device size fix

```

diff --git a/include/cmsis-plus/posix-io/file-system.h b/include/cmsis-plus/posix-io/file-system.h
index 28fb822c..4986ca49 100644
--- a/include/cmsis-plus/posix-io/file-system.h
+++ b/include/cmsis-plus/posix-io/file-system.h
@@ -28,6 +28,7 @@
#include <cmsis-plus/utils/lists.h>

#include <cmsis-plus/diag/trace.h>
+#include <cmsis-plus/rtos/os.h>

#include <mutex>
#include <cstdint>
@@ -821,12 +822,14 @@ namespace os
    inline void
    file_system::add_deferred_file (file* fil)
    {
+    rtos::interrupts::critical_section ics;
        deferred_files_list_.link (*fil);
    }

    inline void
    file_system::add_deferred_directory (directory* dir)
    {
+    rtos::interrupts::critical_section ics;
        deferred_directories_list_.link (*dir);
    }

@@ -848,16 +851,21 @@ namespace os
    {
        using file_type = T;

-    file_type* fil;
+    file_type* fil = nullptr;
+    {
+        rtos::interrupts::critical_section ics;
+        if (!deferred_files_list_.empty ())
+        {
+            fil = static_cast<file_type*> (deferred_files_list_.unlink_head ());
+        }
+    }

-    if (deferred_files_list_.empty ())
+    if (fil == nullptr)
    {

```

```

        fil = new file_type (*this);
    }
else
    {
-        fil = static_cast<file_type*> (deferred_files_list_.unlink_head ());
-
        // Call the constructor before reusing the object,
        fil->~file_type ();

@@ -875,16 +883,21 @@ namespace os
    {
        using file_type = T;

-        file_type* fil;
+        file_type* fil = nullptr;
+        {
+            rtos::interrupts::critical_section ics;
+            if (!deferred_files_list_.empty ())
+            {
+                fil = static_cast<file_type*> (deferred_files_list_.unlink_head ());
+            }
+        }

-        if (deferred_files_list_.empty ())
+        if (fil == nullptr)
        {
            fil = new file_type (*this, locker);
        }
else
    {
-        fil = static_cast<file_type*> (deferred_files_list_.unlink_head ());
-
        // Call the constructor before reusing the object,
        fil->~file_type ();

@@ -902,13 +915,20 @@ namespace os
    {
        using file_type = T;

-        // Deallocate all remaining elements in the list.
-        while (!deferred_files_list_.empty ())
+        for (;;)
        {
-            file_type* f
-            = static_cast<file_type*> (deferred_files_list_.unlink_head ());

```

```

-
-         // Call the destructor and the deallocator.
+         file_type* f = nullptr;
+         {
+             rtos::interrupts::critical_section ics;
+             if (!deferred_files_list_.empty ())
+             {
+                 f = static_cast<file_type*> (deferred_files_list_.unlink_head ());
+             }
+         }
+         if (f == nullptr)
+         {
+             break;
+         }
+         delete f;
+     }
}
@@ -919,17 +939,22 @@ namespace os
{
    using directory_type = T;

-    directory_type* dir;
-
-    if (deferred_directories_list_.empty ())
+    directory_type* dir = nullptr;
+    {
+        rtos::interrupts::critical_section ics;
+        if (!deferred_directories_list_.empty ())
+        {
+            dir = static_cast<directory_type*> (
+                deferred_directories_list_.unlink_head ());
+        }
+    }
+
+    if (dir == nullptr)
+    {
+        dir = new directory_type (*this);
+    }
+    else
+    {
-        dir = static_cast<directory_type*> (
-            deferred_directories_list_.unlink_head ());
-
-        // Call the constructor before reusing the object,
        dir->~directory_type ();

```

```

@@ -947,17 +972,22 @@ namespace os
{
    using directory_type = T;

-    directory_type* dir;
-
-    if (deferred_directories_list_.empty ())
+    directory_type* dir = nullptr;
+    {
+        rtos::interrupts::critical_section ics;
+        if (!deferred_directories_list_.empty ())
+        {
+            dir = static_cast<directory_type*> (
+                deferred_directories_list_.unlink_head ());
+        }
+    }
+
+    if (dir == nullptr)
    {
        dir = new directory_type (*this, locker);
    }
    else
    {
-        dir = static_cast<directory_type*> (
-            deferred_directories_list_.unlink_head ());
-
-        // Call the constructor before reusing the object,
        dir->~directory_type ();
    }

```

```

@@@ -975,13 +1005,21 @@@ namespace os
{
    using directory_type = T;

-    // Deallocate all remaining elements in the list.
-    while (!deferred_directories_list_.empty ())
+    for (;;)
    {
-        directory_type* d = static_cast<directory_type*> (
-            deferred_directories_list_.unlink_head ());
-
-        // Call the destructor and the deallocator.
+        directory_type* d = nullptr;
+        {
+            rtos::interrupts::critical_section ics;

```



```

+         if (!deferred_directories_list_.empty ())
+         {
+             d = static_cast<directory_type*> (
+                 deferred_directories_list_.unlink_head ());
+         }
+     }
+     if (d == nullptr)
+     {
+         break;
+     }
+     delete d;
+ }
+ }

diff --git a/include/cmsis-plus/posix-io/net-stack.h b/include/cmsis-plus/posix-io/net-stack.h
index ebced3da..c2d4f3af 100644
--- a/include/cmsis-plus/posix-io/net-stack.h
+++ b/include/cmsis-plus/posix-io/net-stack.h
@@ -26,6 +26,7 @@
#include <cmsis-plus/utils/lists.h>

#include <cmsis-plus/diag/trace.h>
+#include <cmsis-plus/rtos/os.h>

#include <cstdint>
#include <cassert>
@@ -468,6 +469,7 @@ namespace os
    inline void
    net_stack::add_deferred_socket (class socket* sock)
    {
+    rtos::interrupts::critical_section ics;
    deferred_sockets_list_.link (*sock);
    }

@@ -483,17 +485,22 @@ namespace os
    {
        using socket_type = T;

-        socket_type* sock;
-
-        if (deferred_sockets_list_.empty ())
+        socket_type* sock = nullptr;
+        {
+            rtos::interrupts::critical_section ics;
+            if (!deferred_sockets_list_.empty ())
+            {

```

```

+         sock = static_cast<socket_type*> (
+             deferred_sockets_list_.unlink_head ());
+     }
+ }
+
+ if (sock == nullptr)
+     {
+         sock = new socket_type (*this);
+     }
+ else
+     {
-         sock = static_cast<socket_type*> (
-             deferred_sockets_list_.unlink_head ());
-
-         // Call the constructor before reusing the object,
-         sock->~socket_type ();

@@ -501,12 +508,21 @@ namespace os
        new (sock) socket_type (*this);

        // Deallocate all remaining elements in the list.
-        while (!deferred_sockets_list_.empty ())
+        for (;;)
        {
-            socket_type* s = static_cast<socket_type*> (
-                deferred_sockets_list_.unlink_head ());
-
-            // Call the destructor and the deallocator.
+            socket_type* s = nullptr;
+            {
+                rtos::interrupts::critical_section ics;
+                if (!deferred_sockets_list_.empty ())
+                {
+                    s = static_cast<socket_type*> (
+                        deferred_sockets_list_.unlink_head ());
+                }
+            }
+            if (s == nullptr)
+            {
+                break;
+            }
            delete s;
        }
    }

@@ -519,17 +535,22 @@ namespace os

```

```

{
    using socket_type = T;

-   socket_type* sock;
-
-   if (deferred_sockets_list_.empty ())
+   socket_type* sock = nullptr;
+   {
+       rtos::interrupts::critical_section ics;
+       if (!deferred_sockets_list_.empty ())
+       {
+           sock = static_cast<socket_type*> (
+               deferred_sockets_list_.unlink_head ());
+       }
+   }
+
+   if (sock == nullptr)
+   {
+       sock = new socket_type (*this, locker);
+   }
+   else
+   {
-       sock = static_cast<socket_type*> (
-           deferred_sockets_list_.unlink_head ());
-
-       // Call the constructor before reusing the object,
-       sock->~socket_type ();

@@ -537,12 +558,21 @@ namespace os
    new (sock) socket_type (*this, locker);

    // Deallocate all remaining elements in the list.
-   while (!deferred_sockets_list_.empty ())
+   for (;;)
+   {
-       socket_type* s = static_cast<socket_type*> (
-           deferred_sockets_list_.unlink_head ());
-
-       // Call the destructor and the deallocator.
+       socket_type* s = nullptr;
+       {
+           rtos::interrupts::critical_section ics;
+           if (!deferred_sockets_list_.empty ())
+           {
+               s = static_cast<socket_type*> (

```

```

+         deferred_sockets_list_.unlink_head ());
+     }
+ }
+     if (s == nullptr)
+     {
+         break;
+     }
+     delete s;
+ }
+ }
diff --git a/src/posix-io/block-device.cpp b/src/posix-io/block-device.cpp
index 020245ee..2565cf4d 100644
--- a/src/posix-io/block-device.cpp
+++ b/src/posix-io/block-device.cpp
@@ -153,7 +153,7 @@ namespace os
    // Get logical device sector size (to be used for read/writes).
    {
        std::size_t* sz = va_arg (args, std::size_t*);
-        if (sz == nullptr || impl ().block_logical_size_bytes_ != 0)
+        if (sz == nullptr || impl ().block_logical_size_bytes_ == 0)
        {
            errno = EINVAL;

@@ -171,7 +171,7 @@ namespace os
    // Get physical device sector size (internally used for erase).
    {
        std::size_t* sz = va_arg (args, std::size_t*);
-        if (sz == nullptr || impl ().block_physical_size_bytes_ != 0)
+        if (sz == nullptr || impl ().block_physical_size_bytes_ == 0)
        {
            errno = EINVAL;

@@ -189,7 +189,7 @@ namespace os
    // Get device size in bytes.
    {
        uint64_t* sz = va_arg (args, uint64_t*);
-        if (sz == nullptr || impl ().num_blocks_ != 0)
+        if (sz == nullptr || impl ().num_blocks_ == 0)
        {
            errno = EINVAL;

diff --git a/src/posix-io/file-descriptors-manager.cpp b/src/posix-io/file-descriptors-manager.cpp
index 836539f2..699b3017 100644
--- a/src/posix-io/file-descriptors-manager.cpp
+++ b/src/posix-io/file-descriptors-manager.cpp

```

```

@@ -16,6 +16,7 @@
#include <cmsis-plus/posix-io/file-descriptors-manager.h>
#include <cmsis-plus/posix-io/io.h>
#include <cmsis-plus/posix-io/socket.h>
+#include <cmsis-plus/rtos/os.h>

#include <cmsis-plus/diag/trace.h>

@@ -89,6 +90,8 @@ namespace os
    io*
    file_descriptors_manager::io (int fildes)
    {
+        rtos::interrupts::critical_section ics;
+
        // Check if valid descriptor or buffer not yet initialised
        if ((fildes < 0) || (static_cast<std::size_t> (fildes) >= size__)
            || (descriptors_array__ == nullptr))
@@ -106,7 +109,11 @@ namespace os
    bool
    file_descriptors_manager::valid (int fildes)
    {
-        if ((fildes < 0) || (static_cast<std::size_t> (fildes) >= size__))
+        rtos::interrupts::critical_section ics;
+
+        if ((fildes < 0) || (static_cast<std::size_t> (fildes) >= size__)
+            || (descriptors_array__ == nullptr)
+            || (descriptors_array__[fildes] == nullptr))
        {
            return false;
        }
@@ -127,6 +134,8 @@ namespace os
        return -1;
    }

+    rtos::interrupts::critical_section ics;
+
    for (std::size_t i = reserved__; i < size__; ++i)
    {
        #pragma GCC diagnostic push
@@ -167,6 +176,8 @@ namespace os
        return -1;
    }

+    rtos::interrupts::critical_section ics;
+

```

```

#pragma GCC diagnostic push
#if defined(__clang__)
#pragma clang diagnostic ignored "-Wunsafe-buffer-usage"
@@ -184,7 +195,11 @@ namespace os
    trace::printf ("file_descriptors_manager::%s(%d)\n", __func__, fildes);
#endif

-    if ((fildes < 0) || (static_cast<std::size_t> (fildes) >= size__))
+    rtos::interrupts::critical_section ics;
+
+    if ((fildes < 0) || (static_cast<std::size_t> (fildes) >= size__)
+        || (descriptors_array__ == nullptr)
+        || (descriptors_array__[fildes] == nullptr))
    {
        errno = EBADF;
        return -1;
@@ -204,13 +219,19 @@ namespace os
    file_descriptors_manager::socket (int fildes)
    {
        assert ((fildes >= 0) && (static_cast<std::size_t> (fildes) < size__));
+
+    rtos::interrupts::critical_section ics;
+
    #pragma GCC diagnostic push
    #if defined(__clang__)
    #pragma clang diagnostic ignored "-Wunsafe-buffer-usage"
    #endif
-    auto* const io = descriptors_array__[fildes];
+    auto* const io = (descriptors_array__ != nullptr)
+        ? descriptors_array__[fildes]
+        : nullptr;
    #pragma GCC diagnostic pop
-    if (io->get_type () != static_cast<posix::io::type_t> (io::type::socket))
+    if (io == nullptr
+        || io->get_type () != static_cast<posix::io::type_t> (io::type::socket))
    {
        return nullptr;
    }
@@ -220,6 +241,8 @@ namespace os
    size_t
    file_descriptors_manager::used (void)
    {
+    rtos::interrupts::critical_section ics;
+
        std::size_t count = reserved__;
    }

```

```

        for (std::size_t i = reserved__; i < file_descriptors_manager::size ();
            ++i)
@@ -228,7 +251,7 @@ namespace os
    #if defined(__clang__)
    #pragma clang diagnostic ignored "-Wunsafe-buffer-usage"
    #endif
-        if (descriptors_array__[i] != nullptr)
+        if (descriptors_array__ != nullptr && descriptors_array__[i] != nullptr)
        {
            ++count;
        }

```

**smp-step(06): ARMv8-M mainline guards + SecureFault, semihosting fstat fix,
weak console-mirror hook**


```

diff --git a/include/cmsis-plus/arm/semihosting.h b/include/cmsis-plus/arm/semihosting.h
index de63417e..37c85d61 100644
--- a/include/cmsis-plus/arm/semihosting.h
+++ b/include/cmsis-plus/arm/semihosting.h
@@ -73,7 +73,8 @@ extern "C"
 #endif

// For thumb only architectures use the BKPT instruction instead of SWI.
#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
- || defined(__ARM_ARCH_6M__)
+ || defined(__ARM_ARCH_6M__) || defined(__ARM_ARCH_8M_MAIN__) \
+ || defined(__ARM_ARCH_8M_BASE__)
#define AngelSWIInsn "bkpt"
#define AngelSWIAsm bkpt
#else
@@ -103,6 +104,9 @@ extern "C"
    " mov r1, %[arg] \n"
    #if defined(OS_DEBUG_SEMIHOSTING_FAULTS)
        " " AngelSWITestFault " \n"
+elif defined(SEMIHOST_TRAP_HLT)
+    " .arm \n"
+    " .inst 0xE10F0070 \n"
    #else
        " " AngelSWIInsn " %[swi] \n"
    #endif
diff --git a/src/semihosting/c-syscalls-semihosting.cpp b/src/semihosting/c-syscalls-semihosting.cpp
index e3ff04c4..8b6d034c 100644
--- a/src/semihosting/c-syscalls-semihosting.cpp
+++ b/src/semihosting/c-syscalls-semihosting.cpp
@@ -263,9 +263,12 @@ __semihosting_stat (int fd, struct stat* st)
    return -1;
}

- /* Always assume a character device,
-  with 1024 byte blocks. */
- st->st_mode |= S_IFCHR;
+ /* If the caller did not already specify a file type,
+  default to character device, with 1024 byte blocks. */
+ if ((st->st_mode & S_IFMT) == 0)
+ {
+     st->st_mode |= S_IFCHR;
+ }
+ st->st_blksize = 1024;

int res;

```

```

@@ -437,9 +440,32 @@ __posix_read (int fildes, void* buf, size_t nbyte)
    return nbyte - res;
}

+/**
+ * Optional board console mirror.
+ *
+ * A board whose console is a physical UART (not the debugger) defines this to
+ * copy the console stream there, so the application's stdout/stderr is visible
+ * on the terminal as well as in the debugger's semihosting console. The
+ * semihosting write stays the primary path; the board only adds the UART copy.
+ * Weak: boards that do not need it link this no-op and it is never called.
+ */
+extern "C" void
+os_board_console_mirror (int fildes, const void* buf, size_t nbyte)
+  __attribute__ ((weak));
+
+ssize_t
+__posix_write (int fildes, const void* buf, size_t nbyte)
+{
+ // The board console, if any, gets stdout/stderr whatever happens to the
+ // semihosting side below: on some ports the monitor handles are not set up,
+ // and a dropped printf() would otherwise never be seen. Done before the fd
+ // lookup so it does not depend on it.
+ if ((fildes == 1 || fildes == 2)
+     && (os_board_console_mirror != nullptr))
+ {
+     os_board_console_mirror (fildes, buf, nbyte);
+ }
+
+ struct fdent* pfd;
+ pfd = __semihosting_findslot (fildes);
+ if (pfd == NULL)
diff --git a/src/startup/exception-handlers.c b/src/startup/exception-handlers.c
index 98be7d14..533ffcd7 100644
--- a/src/startup/exception-handlers.c
+++ b/src/startup/exception-handlers.c
@@ -91,7 +91,8 @@ Reset_Handler (void)
    // SCB->VTOR
    // https://developer.arm.com/documentation/dui0552/a/cortex-m3-peripherals/system-control-block/
vector-table-offset-register
    // Mandatory when running from RAM. Not available on Cortex-M0.
-#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)
+#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+ || defined(__ARM_ARCH_8M_MAIN__)

```

```

        *((uint32_t*)0xE000ED08)
        = ((uint32_t)_interrupt_vectors & (uint32_t)(~0x3F));
#endif
@@ -119,14 +120,16 @@ void __attribute__ ((section (".after_vectors"), weak))
NMI_Handler (void)
{
    #if defined(DEBUG)
    -#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)
    +#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
    +    || defined(__ARM_ARCH_8M_MAIN__)
        if ((CoreDebug->DHCSR & CoreDebug_DHCSR_C_DEBUGEN_Msk) != 0)
        {
            __BKPT (0);
        }
    #else
        __BKPT (0);
    -#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) */
    +#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
    +    || defined(__ARM_ARCH_8M_MAIN__) */
    #endif /* defined(DEBUG) */

    while (true)
@@ -139,7 +142,8 @@ NMI_Handler (void)

    #if defined	TRACE)

    -#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)
    +#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
    +    || defined(__ARM_ARCH_8M_MAIN__)

    // The values of BFAR and MMFAR remain unchanged if the BFARVALID or
    // MMARVALID is set. However, if a new fault occurs during the
    @@ -183,7 +187,8 @@ dump_exception_stack (exception_stack_frame_t* frame, uint32_t cfsr,
        trace_printf (" LR/EXC_RETURN = %08X\n", lr);
    }

    -#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) */
    +#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
    +    || defined(__ARM_ARCH_8M_MAIN__) */

    #if defined(__ARM_ARCH_6M__)

    @@ -209,7 +214,8 @@ dump_exception_stack (exception_stack_frame_t* frame, uint32_t lr)

    // -----

```

```

-#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)
+#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+    || defined(__ARM_ARCH_8M_MAIN__)

    #if defined(OS_USE_SEMIHOSTING_SYSCALLS) \
        || defined(OS_USE_TRACE_SEMIHOSTING_STDOUT) \
@@ -470,14 +476,16 @@ HardFault_Handler_C (exception_stack_frame_t* frame __attribute__ ((unused)),
    #endif /* defined	TRACE) */

    #if defined(DEBUG)
-#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)
+#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+    || defined(__ARM_ARCH_8M_MAIN__)
        if ((CoreDebug->DHCSR & CoreDebug_DHCSR_C_DEBUGEN_Msk) != 0)
        {
            __BKPT (0);
        }
    #else
        __BKPT (0);
-#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) */
+#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+    || defined(__ARM_ARCH_8M_MAIN__) */
    #endif /* defined(DEBUG) */

    while (true)
@@ -486,7 +494,8 @@ HardFault_Handler_C (exception_stack_frame_t* frame __attribute__ ((unused)),
    }
}

-#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) */
+#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+    || defined(__ARM_ARCH_8M_MAIN__) */

    #if defined(__ARM_ARCH_6M__)

@@ -532,14 +541,16 @@ HardFault_Handler_C (exception_stack_frame_t* frame __attribute__ ((unused)),
    #endif /* defined	TRACE) */

    #if defined(DEBUG)
-#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)
+#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+    || defined(__ARM_ARCH_8M_MAIN__)
        if ((CoreDebug->DHCSR & CoreDebug_DHCSR_C_DEBUGEN_Msk) != 0)
        {

```

```

        __BKPT (0);
    }
#else
    __BKPT (0);
#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) */
#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+   || defined(__ARM_ARCH_8M_MAIN__) */
#endif /* defined(DEBUG) */

    while (true)
@@ -550,20 +561,23 @@ HardFault_Handler_C (exception_stack_frame_t* frame __attribute__ ((unused)),

#endif /* defined(__ARM_ARCH_6M__) */

#ifndef defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)
+#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+   || defined(__ARM_ARCH_8M_MAIN__)

void __attribute__ ((section (".after_vectors"), weak))
MemManage_Handler (void)
{
    #if defined(DEBUG)
    #if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)
    #if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+   || defined(__ARM_ARCH_8M_MAIN__)
        if ((CoreDebug->DHCSR & CoreDebug_DHCSR_C_DEBUGEN_Msk) != 0)
        {
            __BKPT (0);
        }
    #else
        __BKPT (0);
    #endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) */
    #endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+   || defined(__ARM_ARCH_8M_MAIN__) */
    #endif /* defined(DEBUG) */

    while (true)
@@ -603,14 +617,16 @@ BusFault_Handler_C (exception_stack_frame_t* frame __attribute__ ((unused)),

#endif /* defined	TRACE) */

#if defined(DEBUG)
#ifndef defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)
+#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+   || defined(__ARM_ARCH_8M_MAIN__)

    if ((CoreDebug->DHCSR & CoreDebug_DHCSR_C_DEBUGEN_Msk) != 0)

```

```

    {
        __BKPT (0);
    }
#else
    __BKPT (0);
#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) */
#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+ || defined(__ARM_ARCH_8M_MAIN__) */
#endif /* defined(DEBUG) */

    while (true)
@@ -665,14 +681,16 @@ UsageFault_Handler_C (exception_stack_frame_t* frame __attribute__ ((unused)),
    #endif /* defined	TRACE) */

    #if defined(DEBUG)
    #if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)
    #if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+ || defined(__ARM_ARCH_8M_MAIN__)
        if ((CoreDebug->DHCSR & CoreDebug_DHCSR_C_DEBUGEN_Msk) != 0)
        {
            __BKPT (0);
        }
    #else
        __BKPT (0);
    #endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) */
    #endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+ || defined(__ARM_ARCH_8M_MAIN__) */
    #endif /* defined(DEBUG) */

    while (true)
@@ -687,14 +705,16 @@ void __attribute__ ((section (".after_vectors"), weak))
    SVC_Handler (void)
    {
        #if defined(DEBUG)
        #if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)
        #if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+ || defined(__ARM_ARCH_8M_MAIN__)
            if ((CoreDebug->DHCSR & CoreDebug_DHCSR_C_DEBUGEN_Msk) != 0)
            {
                __BKPT (0);
            }
        #else
            __BKPT (0);
        #endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) */
        #endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+ || defined(__ARM_ARCH_8M_MAIN__) */
    #endif /* defined(DEBUG) */

```

```

+    || defined(__ARM_ARCH_8M_MAIN__) */
#endif /* defined(DEBUG) */

    while (true)
@@ -703,7 +723,8 @@ SVC_Handler (void)
    }
}

-#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)
+#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+    || defined(__ARM_ARCH_8M_MAIN__)

void __attribute__((section(".after_vectors"), weak))
DebugMon_Handler (void)
@@ -721,20 +742,23 @@ DebugMon_Handler (void)
    }
}

-#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) */
+#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+    || defined(__ARM_ARCH_8M_MAIN__) */

void __attribute__((section(".after_vectors"), weak))
PendSV_Handler (void)
{
    #if defined(DEBUG)
-#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)
+#if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+    || defined(__ARM_ARCH_8M_MAIN__)
        if ((CoreDebug->DHCSR & CoreDebug_DHCSR_C_DEBUGEN_Msk) != 0)
        {
            __BKPT (0);
        }
    #else
        __BKPT (0);
-#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) */
+#endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) \
+    || defined(__ARM_ARCH_8M_MAIN__) */
    #endif /* defined(DEBUG) */

    while (true)

```

smp-step(07): timer callback outside critical section, catch-up re-arm, handler-mode errno scratch


```

diff --git a/include/cmsis-plus/rtos/os-thread.h b/include/cmsis-plus/rtos/os-thread.h
index 9cd7a05a..3aecb6a9 100644
--- a/include/cmsis-plus/rtos/os-thread.h
+++ b/include/cmsis-plus/rtos/os-thread.h
@@ -2081,6 +2081,14 @@ namespace os
     inline int* __attribute__((always_inline))
     __errno (void)
     {
+
+        // A timer callback runs inside the tick ISR and a libc call there
+        // (e.g. printf) still touches errno; thread() asserts in handler mode,
+        // so hand back a scratch int instead of the current thread's.
+        if (interrupts::in_handler_mode ())
+        {
+            static int isr_errno;
+            return &isr_errno;
+        }
+        return &this_thread::thread ().errno_;
     }

diff --git a/src/rtos/internal/os-lists.cpp b/src/rtos/internal/os-lists.cpp
index 02ea8ab4..32c0c9a0 100644
--- a/src/rtos/internal/os-lists.cpp
+++ b/src/rtos/internal/os-lists.cpp
@@ -499,33 +499,42 @@ namespace os
     // iterate until a node with future time stamp is identified.
     for (;;)
     {
-
-        // ----- Enter critical section -----
-        interrupts::critical_section ics;
+
+        timestamp_node* node = nullptr;
+
+        {
+            // ----- Enter critical section -----
+            interrupts::critical_section ics;

-            if (empty ())
-            {
-                break;
-            }
+            if (empty ())
+            {
+                break;
+            }

+            if (empty ())
+            {
+                break;
+            }

+            #pragma GCC diagnostic push
+            #if defined(__clang__)

```

```

#elif defined(__GNUC__)
#pragma GCC diagnostic ignored "-Wnull-dereference"
#endif

-         clock::timestamp_t head_ts = head ()->timestamp;
+         clock::timestamp_t head_ts = head ()->timestamp;
#pragma GCC diagnostic pop
-         if (now >= head_ts)
+         if (now >= head_ts)
+         {
+             node = const_cast<timestamp_node*> (head ());
+             node->unlink ();
+         }
+         else
+         {
+             break;
+         }
+         // ----- Exit critical section -----
+     }
+
+     if (node != nullptr)
+     {
+         #if defined(OS_TRACE_RTOS_LISTS_CLOCKS)
+             trace::printf ("%s() %u \n", __func__,
+                             static_cast<uint32_t> (sysclock.now ()));
+         #endif
+         const_cast<timestamp_node*> (head ())->action ();
+     }
+     else
+     {
+         break;
+         node->action ();
+     }
+     // ----- Exit critical section -----
+ }
+ }

diff --git a/src/rtos/os-timer.cpp b/src/rtos/os-timer.cpp
index 7b9c89c5..f5318340 100644
--- a/src/rtos/os-timer.cpp
+++ b/src/rtos/os-timer.cpp
@@ -374,9 +374,17 @@ namespace os
{
    // Re-arm the timer for the next period.
    timer_node_.timestamp += period_;
+
+    clock::timestamp_t now = clock_->steady_now ();

```

```

+         if (timer_node_.timestamp <= now)
+         {
+             // Prevent burst loop if callback or ISR was delayed
+             timer_node_.timestamp = now + period_;
+         }

-         // No need for critical section in ISR.
-         clock_->steady_list ().link (timer_node_);
+         {
+             interrupts::critical_section ics;
+             clock_->steady_list ().link (timer_node_);
+         }
    }
    else
    {

```

**smp-step(08): mutex priority ceiling before ownership, max-waiter boost, unlock
recompute, owner cache**

```

diff --git a/src/rtos/os-mutex.cpp b/src/rtos/os-mutex.cpp
index e8cab9d1..6ce22248 100644
--- a/src/rtos/os-mutex.cpp
+++ b/src/rtos/os-mutex.cpp
@@ -638,6 +638,15 @@ namespace os
    // First lock.
    if (owner_ == nullptr)
    {
+       // Prio ceiling must be at least the priority of the highest
+       // priority thread. Checked before anything is taken: a refused
+       // lock must not leave the mutex on the thread's list of owned
+       // mutexes, nor counted in acquired_mutexes_.
+       if (protocol_ == protocol::protect && th->priority () > prio_ceiling_)
+       {
+           return EINVAL;
+       }
+
        // If the mutex has no owner, own it.
        owner_ = th;

@@ -661,16 +670,6 @@ namespace os

        if (protocol_ == protocol::protect)
        {
-           if (th->priority () > prio_ceiling_)
-           {
-               // No need to keep the lock.
-               owner_ = nullptr;
-
-               // Prio ceiling must be at least the priority of the
-               // highest priority thread.
-               return EINVAL;
-           }
-
            // POSIX: When a thread owns one or more mutexes
            // initialised with the mutex::protocol::protect protocol,
            // it shall execute at the higher of its priority or the
@@ -778,7 +777,11 @@ namespace os
        if (protocol_ == protocol::inherit)
        {
            thread::priority_t prio = th->priority ();
-           boosted_prio_ = prio;
+           if ((boosted_prio_ == thread::priority::none)
+               || (prio > boosted_prio_))

```

```

+         {
+             boosted_prio_ = prio;
+         }

        if (owner_links_.unlinked ())
        {
@@ -788,13 +791,54 @@ namespace os
        }

        // Boost owner priority.
-        if ((boosted_prio_ > owner_>priority_inherited ()))
+
+        // The owner is captured HERE, while the kernel lock is still
+        // held, and the captured pointer -- not the member -- is the
+        // one dereferenced below.
+
+        // The reason is the uncritical section itself: it releases the
+        // kernel lock (`priority_inherited()` ends in a yield, which
+        // must not run scheduler-locked), and while it is open the
+        // owner can finish its own `unlock()` on another CPU.
+        // `internal_unlock_()` ends by setting `owner_` to nullptr, so
+        // re-reading the member after the section -- which is exactly
+        // what `owner_>priority_inherited(...)` compiles to -- then
+        // dereferences a null pointer.
+
+        // Every SMP port has this window. It was found on the POSIX
+        // host port, where `scheduler::unlock()``lock()` is two
+        // `pthread_sigmask()` system calls rather than a handful of
+        // instructions: that widens the window enough to turn a rare
+        // race into the common case (smp_test2 faulted in 21 runs out
+        // of 25 on four CPUs).
+        /* class */ thread* owner = owner_;
+
+        if ((owner != nullptr)
+            && (boosted_prio_ > owner->priority_inherited ()))
        {
-            // ----- Enter uncritical section -----
-            scheduler::uncritical_section succs;
+            {
+                // ----- Enter uncritical section -----
+                scheduler::uncritical_section succs;
+
+                // Still the owner? If it released the mutex while this
+                // section was open there is nothing to inherit, and
+                // boosting it anyway would leave behind an inherited

```

```

+         // priority that no later unlock() would ever clear.
+         if (owner_ == owner)
+         {
+             owner->priority_inherited (boosted_prio_);
+         }
+         // ----- Exit uncritical section -----
+     }

-     owner->priority_inherited (boosted_prio_);
-     // ----- Exit uncritical section -----
+     // Released while the section was open? Then that unlock()
+     // found no waiter to hand the mutex to, and blocking now
+     // would wait for a wake-up that already happened. Decide
+     // again, back under the lock.
+     if (owner_ != owner)
+     {
+         return internal_try_lock_ (th);
+     }
+ }

#if defined(OS_TRACE_RTOS_MUTEX)
@@ -862,14 +906,8 @@ namespace os
    mutexes_list* thread_mutexes
        = reinterpret_cast<mutexes_list*> (&owner_->mutexes_);

-     if (thread_mutexes->empty ())
-     {
-         // If the owner thread has no more mutexes,
-         // clear the inherited priority,
-         // and the assigned priority will take precedence.
-         boosted_prio_ = thread::priority::none;
-     }
-     else
+     thread::priority_t inherited_prio = thread::priority::none;
+     if (!thread_mutexes->empty ())
+     {
+         // If the owner thread acquired other mutexes too,
+         // compute the maximum boosted priority.
@@ -887,10 +925,14 @@ namespace os
    }

    }

#pragma GCC diagnostic pop
-     boosted_prio_ = max_prio;
+     if (max_prio > 0)
+     {

```

```
+         inherited_prio = max_prio;
+     }
+     }
+     // Delayed until end of critical section.
-     owner_->priority_inherited (boosted_prio_);
+     owner_->priority_inherited (inherited_prio);
+     boosted_prio_ = thread::priority::none;
+ }

// Delayed until end of critical section.
```


smp-step(09): thread state::destroying, atomic join, detach, reaper

```

diff --git a/include/cmsis-plus/rtos/os-c-decls.h b/include/cmsis-plus/rtos/os-c-decls.h
index e8565a72..260d1e52 100644
--- a/include/cmsis-plus/rtos/os-c-decls.h
+++ b/include/cmsis-plus/rtos/os-c-decls.h
@@ -315,7 +315,12 @@ extern "C"
    /**
     * @brief Used to check reused threads.
     */
-   os_thread_state_initialising = 6
+   os_thread_state_initialising = 6,
+
+   /**
+    * @brief In process of being destroyed.
+    */
+   os_thread_state_destroying = 7
};

/**
diff --git a/include/cmsis-plus/rtos/os-thread.h b/include/cmsis-plus/rtos/os-thread.h
index 3aecb6a9..cac6777f 100644
--- a/include/cmsis-plus/rtos/os-thread.h
+++ b/include/cmsis-plus/rtos/os-thread.h
@@ -400,7 +400,11 @@ namespace os
    /**
     * @brief Used to check reused threads.
     */
-   initializing = 6 //
+   initializing = 6, //
+
+   /**
+    * @brief In process of being destroyed.
+    */
+   destroying = 7 //
};

/* enum */
}; /* struct state */
@@ -741,6 +745,7 @@ namespace os
    friend port::stack::element_t*
    port::scheduler::switch_stacks (port::stack::element_t* sp);

+
#endif

/**
    * @endcond
@@ -875,6 +880,7 @@ namespace os

```

```

        bool th_enable_assert_reuse = false;

+
        // Add more attributes here.

        /**
@@ -1192,6 +1198,7 @@ namespace os
        priority_t
        priority_inherited (void);

+
        #if 0
            // ???
            result_t
@@ -1689,6 +1696,7 @@ namespace os
            priority_t volatile prio_assigned_ = priority::none;
            priority_t volatile prio_inherited_ = priority::none;

+
            bool volatile interrupted_ = false;

            internal::event_flags event_flags_;
diff --git a/src/rtos/os-idle.cpp b/src/rtos/os-idle.cpp
index 23b05974..0db1ab35 100644
--- a/src/rtos/os-idle.cpp
+++ b/src/rtos/os-idle.cpp
@@ -79,6 +79,14 @@ os_rtos_idle_actions (void)
        interrupts::critical_section ics;
        node = const_cast<internal::waiting_thread_node*> (
            scheduler::terminated_threads_list_.head ());
+
        thread* th = node->thread_;
+
        if (th->state_ == thread::state::destroying
+
            || th->state_ == thread::state::destroyed)
+
        {
+
            node->unlink ();
+
            continue;
+
        }
+
        th->state_ = thread::state::destroying;
        node->unlink ();
        // ----- Exit critical section -----
    }
diff --git a/src/rtos/os-thread.cpp b/src/rtos/os-thread.cpp
index 3b8f293a..79b0a71c 100644
--- a/src/rtos/os-thread.cpp

```

```

+++ b/src/rtos/os-thread.cpp
@@ -19,6 +19,7 @@
#include <memory>
#include <stdexcept>

+
// -----

#ifdef __clang__
@@ -679,10 +680,15 @@ namespace os
    interrupts::critical_section ics;

    // If the thread is not already in the ready list, enqueue it.
-   if (ready_node_.next () == nullptr)
+   // In SMP, a running thread has ready_node_.next() == nullptr;
+   // it must not be re-enqueued while actively executing.
+   if (state_ == state::suspended || state_ == state::initializing)
    {
-       scheduler::ready_threads_list_.link (ready_node_);
-       // state::ready set in above link().
+       if (ready_node_.next () == nullptr)
+       {
+           scheduler::ready_threads_list_.link (ready_node_);
+           // state::ready set in above link().
+       }
    }
    // ----- Exit critical section -----
}

@@ -691,6 +697,7 @@ namespace os

    port::scheduler::reschedule ();

+
#endif

    instrumentation::thread::resume_return (this);
@@ -702,6 +709,7 @@ namespace os
    *
    * @note Can be invoked from Interrupt Service Routines.
    */
+
    thread::priority_t
    thread::priority (void)
    {
@@ -791,17 +799,24 @@ namespace os

```

```

#else

-   if (state_ == state::ready)
-   {
-       // ----- Enter critical section -----
-       interrupts::critical_section ics;
+   {
+       // ----- Enter critical section -----
+       interrupts::critical_section ics;

-       // Remove from initial location and reinsert according
-       // to new priority.
-       ready_node_.unlink ();
-       scheduler::ready_threads_list_.link (ready_node_);
-       // ----- Exit critical section -----
-   }
+   // Test and relink under one lock, and only a thread that is still
+   // linked in the ready list. On SMP another CPU may pick this thread
+   // between a test made outside the lock and the relink: link() would
+   // then put a thread being dispatched back in the ready list, marked
+   // ready, and a third CPU could run the same context at once.
+   if (state_ == state::ready && ready_node_.next () != nullptr)
+   {
+       // Remove from initial location and reinsert according
+       // to new priority.
+       ready_node_.unlink ();
+       scheduler::ready_threads_list_.link (ready_node_);
+   }
+   // ----- Exit critical section -----
+   }

    // Mandatory, the priority might have been raised, the
    // task must be scheduled to run.
@@ -875,17 +890,24 @@ namespace os

#else

-   if (state_ == state::ready)
-   {
-       // ----- Enter critical section -----
-       interrupts::critical_section ics;
+   {
+       // ----- Enter critical section -----
+       interrupts::critical_section ics;

```

```

-         // Remove from initial location and reinsert according
-         // to new priority.
-         ready_node_.unlink ();
-         scheduler::ready_threads_list_.link (ready_node_);
-         // ----- Exit critical section -----
-     }
+     // Test and relink under one lock, and only a thread that is still
+     // linked in the ready list. On SMP another CPU may pick this thread
+     // between a test made outside the lock and the relink: link() would
+     // then put a thread being dispatched back in the ready list, marked
+     // ready, and a third CPU could run the same context at once.
+     if (state_ == state::ready && ready_node_.next () != nullptr)
+     {
+         // Remove from initial location and reinsert according
+         // to new priority.
+         ready_node_.unlink ();
+         scheduler::ready_threads_list_.link (ready_node_);
+     }
+     // ----- Exit critical section -----
+ }

    // Mandatory, the priority might have been raised, the
    // task must be scheduled to run.
@@ -941,7 +963,22 @@ namespace os

    #else

-     // TODO: implement
+     {
+         interrupts::critical_section ics;
+
+         if (state_ == state::destroyed)
+         {
+             instrumentation::thread::detach_retval (this, EINVAL);
+             return EINVAL;
+         }
+
+         if (parent_ != nullptr)
+         {
+             child_links_.unlink ();
+             scheduler::top_threads_list_.link (*this);
+             parent_ = nullptr;
+         }
+     }

```

```

#endif

@@ -997,13 +1034,37 @@ namespace os
    // Fail if current thread
    assert (this != this_thread::_thread ());

-   while (state_ != state::destroyed)
+   thread* crt_thread = this_thread::_thread ();
+   for (;;)
    {
-       joiner_ = this_thread::_thread ();
-       this_thread::_thread ()->internal_suspend_ (
-           OS_INTEGER_INSTRUMENTATION_SUSPEND_CAUSE_JOIN);
+       {
+           // ----- Enter critical section -----
+           interrupts::critical_section ics;
+
+           // Test, register and suspend under one lock, the same lock
+           // internal_destroy() sets `destroyed` and reads `joiner_`
+           // under. Done in steps, a destroy on another CPU could fall
+           // between them and its wake-up be lost.
+           if (state_ == state::destroyed)
+           {
+               break;
+           }
+           joiner_ = crt_thread;
+
+           // Remove this thread from the ready list, if there.
+           port::this_thread::prepare_suspend ();
+
+           crt_thread->state_ = state::suspended;
+           // ----- Exit critical section -----
+       }

+       instrumentation::thread::suspended (
+           crt_thread, OS_INTEGER_INSTRUMENTATION_SUSPEND_CAUSE_JOIN);
+
+       port::scheduler::reschedule ();
    }

+
+   #if defined(OS_TRACE_RTOS_THREAD)
+       trace::printf ("%s() @%p %s joined\n", __func__, this, name ());
+   #endif

```

```

@@ -1268,12 +1329,27 @@ namespace os
    // ----- Exit critical section -----
}

-    state_ = state::destroyed;
+    {
+        // ----- Enter critical section -----
+        interrupts::critical_section ics;
+
+        // From `destroyed` on, a joiner may return from join() and free
+        // this object, so `joiner_` is read under the same lock that
+        // sets it, and the joiner is made ready here, where it cannot
+        // yet have gone -- resume() would do it after the lock.
+        state_ = state::destroyed;

-    if (joiner_ != nullptr)
-    {
-        joiner_>resume ();
-    }
+    thread* joiner = joiner_;
+    if (joiner != nullptr && joiner->state_ == state::suspended
+        && joiner->ready_node_.next () == nullptr)
+    {
+        scheduler::ready_threads_list_.link (joiner->ready_node_);
+        // state::ready set in above link().
+    }
+    // ----- Exit critical section -----
+    }

+    port::scheduler::reschedule ();
+}

#pragma GCC diagnostic pop

@@ -1308,6 +1384,7 @@ namespace os
    // ----- Enter critical section -----
    scheduler::critical_section scs;

+
+    if (state_ == state::destroyed)
+    {
+        #if defined(OS_TRACE_RTOS_THREAD)
@@ -1318,29 +1395,49 @@ namespace os
        return result::ok; // Already exited itself
    }

```



```

+     bool we_claimed = false;
+     {
+         // ----- Enter critical section -----
+         interrupts::critical_section ics;

-         // Remove thread from the funeral list and kill it here.
-         ready_node_.unlink ();
-
-         // If the thread is waiting on an event, remove it from the list.
-         if (waiting_node_ != nullptr)
+         if (state_ != state::destroying && state_ != state::destroyed)
+         {
-             waiting_node_>unlink ();
-         }
+         we_claimed = true;
+         state_ = state::destroying;

-         // If the thread is waiting on a timeout, remove it from the list.
-         if (clock_node_ != nullptr)
-         {
-             clock_node_>unlink ();
-         }
+         // Remove thread from the funeral list and kill it here.
+         ready_node_.unlink ();

-         child_links_.unlink ();
+         // If the thread is waiting on an event, remove it from the list.
+         if (waiting_node_ != nullptr)
+         {
+             waiting_node_>unlink ();
+         }
+
+         // If the thread is waiting on a timeout, remove it from the list.
+         if (clock_node_ != nullptr)
+         {
+             clock_node_>unlink ();
+         }
+
+         child_links_.unlink ();
+     }
+     // ----- Exit critical section -----
+ }

+ if (!we_claimed)
+ {

```

```

+         // The idle reaper already claimed this thread for destruction.
+         // Wait with lock released until it is completely destroyed.
+         while (state_ != state::destroyed)
+         {
+             scheduler::uncritical_section succs;
+             this_thread::yield ();
+         }
+         instrumentation::thread::kill_retval (this, result::ok);
+         return result::ok;
+     }
+
+     // The must be no more children threads alive.
+     assert (children_.empty ());
+     parent_ = nullptr;
@@ -1837,6 +1934,11 @@ namespace os
+
+     *
+     * @warning Cannot be invoked from Interrupt Service Routines.
+     */
+
+     // NOTE: NOT 'inline'. It is declared non-inline in os-thread.h, and its
+     // body lives only here (not in a header), so other TUs (e.g. the C API
+     // wrapper os_this_thread_suspend) that odr-use it need an out-of-line
+     // definition. Marking it 'inline' meant no TU emitted one -> undefined
+     // reference unless the caller happened to be pruned by --gc-sections.
+     void
+     suspend (void)
+     {

```

smp-step(10): condvar atomic wait/timed_wait, clock member, CONDVAR suspend cause

```

diff --git a/include/cmsis-plus/diag/instrumentation.h b/include/cmsis-plus/diag/instrumentation.h
index 69e0c035..fd0b5cec 100644
--- a/include/cmsis-plus/diag/instrumentation.h
+++ b/include/cmsis-plus/diag/instrumentation.h
@@ -25,6 +25,7 @@
#define OS_INTEGER_INSTRUMENTATION_SUSPEND_CAUSE_EVENT_FLAGS (9u)
#define OS_INTEGER_INSTRUMENTATION_SUSPEND_CAUSE_JOIN (10u)
#define OS_INTEGER_INSTRUMENTATION_SUSPEND_CAUSE_USER (11u)
+#define OS_INTEGER_INSTRUMENTATION_SUSPEND_CAUSE_CONDVAR (12u)

// -----

diff --git a/include/cmsis-plus/rtos/os-c-decls.h b/include/cmsis-plus/rtos/os-c-decls.h
index 260d1e52..86dc5429 100644
--- a/include/cmsis-plus/rtos/os-c-decls.h
+++ b/include/cmsis-plus/rtos/os-c-decls.h
@@ -1065,7 +1065,7 @@ extern "C"
    const char* name;
    #if !defined(OS_USE_RTOS_PORT_CONDITION_VARIABLE)
        os_internal_threads_waiting_list_t list;
-    // void* clock;
+    void* clock;
    #endif

    /**

diff --git a/include/cmsis-plus/rtos/os-condvar.h b/include/cmsis-plus/rtos/os-condvar.h
index d9b64b22..8faa4e66 100644
--- a/include/cmsis-plus/rtos/os-condvar.h
+++ b/include/cmsis-plus/rtos/os-condvar.h
@@ -278,7 +278,7 @@ namespace os

    #if !defined(OS_USE_RTOS_PORT_CONDITION_VARIABLE)
        internal::waiting_threads_list list_;
-        // clock& clock_;
+        clock* clock_ = nullptr;
    #endif

    /**

diff --git a/src/rtos/os-condvar.cpp b/src/rtos/os-condvar.cpp
index 9c1021a3..b08ff1e7 100644
--- a/src/rtos/os-condvar.cpp
+++ b/src/rtos/os-condvar.cpp
@@ -271,8 +271,7 @@ namespace os
    * Edition](http://pubs.opengroup.org/onlinepubs/9699919799/nframe.html)).

```

```

    */
    condition_variable::condition_variable (const char* name,
-                                     const attributes& attr
-                                     __attribute__((unused)))
+                                     const attributes& attr)
        : object_named_system{ name }
    {
        instrumentation::condition_variable::create (this);
@@ -284,6 +283,10 @@ namespace os
        // Don't call this from interrupt handlers.
        os_assert_throw (!interrupts::in_handler_mode (), EPERM);

+    #if !defined(OS_USE_RTOS_PORT_CONDITION_VARIABLE)
+        clock_ = attr.clock != nullptr ? attr.clock : &sysclock;
+    #endif
+
        instrumentation::condition_variable::create_return (this);
    }

@@ -371,10 +374,20 @@ namespace os
        // Don't call this from interrupt handlers.
        os_assert_err (!interrupts::in_handler_mode (), EPERM);

+    #if defined(OS_USE_RTOS_PORT_CONDITION_VARIABLE)
+
+        result_t res = port::condition_variable::signal (this);
+        instrumentation::condition_variable::signal_retval (this, res);
+        return res;
+
+    #else
+
        list_.resume_one ();

        instrumentation::condition_variable::signal_retval (this, result::ok);
        return result::ok;
+
+    #endif
    }

    /**
@@ -447,6 +460,14 @@ namespace os
        // Don't call this from interrupt handlers.
        os_assert_err (!interrupts::in_handler_mode (), EPERM);

+    #if defined(OS_USE_RTOS_PORT_CONDITION_VARIABLE)

```

```
+         result_t res = port::condition_variable::broadcast (this);
+         instrumentation::condition_variable::broadcast_retval (this, res);
+         return res;
+
+#else
+
+         // Wake-up all threads, if any.
+         // Need not be inside the critical section,
+         // the list is protected by inner `resume_one()`.
@@ -454,6 +475,8 @@ namespace os
+
+         instrumentation::condition_variable::broadcast_retval (this, result::ok);
+         return result::ok;
+
+#endif
+     }
+
+ /**
@@ -555,6 +578,14 @@ namespace os
+         // Don't call this from critical regions.
+         os_assert_err (!scheduler::locked (), EPERM);
+
+#if defined(OS_USE_RTOS_PORT_CONDITION_VARIABLE)
+
+         result_t res = port::condition_variable::wait (this, &mutex);
+         instrumentation::condition_variable::wait_retval (this, res);
+         return res;
+
+#else
+
+         thread& crt_thread = this_thread::thread ();
+
+         // Prepare a list node pointing to the current thread.
@@ -562,32 +593,52 @@ namespace os
+         // list and guaranteed to be removed before this function returns.
+         internal::waiting_thread_node node{ crt_thread };
+
+         // TODO: validate
+
+         result_t res;
+
+         res = mutex.unlock ();
+
+         {
+             // ----- Enter critical section -----
+
+             // The link and the unlock must be one step for this CPU's scheduler.
```

```

+ // Once linked, the thread is `suspended`; a tick or an IPI taken
+ // before `mutex.unlock()` would switch it out still OWNING the mutex,
+ // and the thread that could signal it would block on that mutex
+ // forever (smp-pro-cons-test stalled this way on 4 cores, 1 run in
+ // about 20). Other CPUs are not held back: they can still take the
+ // mutex, and signal(), as soon as it is released.
+ scheduler::critical_section scs;

- if (res != result::ok)
+ {
-     instrumentation::condition_variable::wait_retval (this, res);
-     return res;
+ // ----- Enter critical section -----
+ interrupts::critical_section ics;
+
+ // Add this thread to the condition variable waiting list.
+ scheduler::internal_link_node (
+     list_, node, OS_INTEGER_INSTRUMENTATION_SUSPEND_CAUSE_CONDVAR);
+ // state::suspended set in above link().
+ // ----- Exit critical section -----
+ }

- {
- // Add this thread to the condition variable waiting list.
- list_.link (node);
- node.thread_>waiting_node_ = &node;
-
- res = mutex.lock ();
+ res = mutex.unlock ();

- // Remove the thread from the node waiting list,
- // if not already removed.
- node.thread_>waiting_node_ = nullptr;
- node.unlink ();
+ if (res != result::ok)
+ {
+     scheduler::internal_unlink_node (node);
+     instrumentation::condition_variable::wait_retval (this, res);
+     return res;
+ }
+ // ----- Exit critical section -----
+ }

+ port::scheduler::reschedule ();
+

```

```

+    // Remove the thread from the condition variable waiting list,
+    // if not already removed by signal() / broadcast().
+    scheduler::internal_unlink_node (node);
+
+    res = mutex.lock ();
+
+    instrumentation::condition_variable::wait_retval (this, res);
+    return res;
+
+endif
+    }
+
+    /**
@@ -715,6 +766,14 @@ namespace os
+    // Don't call this from critical regions.
+    os_assert_err (!scheduler::locked (), EPERM);
+
+    #if defined(OS_USE_RTOS_PORT_CONDITION_VARIABLE)
+
+    result_t res = port::condition_variable::timed_wait (this, &mutex, timeout);
+    instrumentation::condition_variable::timed_wait_retval (this, res);
+    return res;
+
+    #else
+
+    thread& crt_thread = this_thread::thread ();
+
+    // Prepare a list node pointing to the current thread.
@@ -722,32 +781,62 @@ namespace os
+    // list and guaranteed to be removed before this function returns.
+    internal::waiting_thread_node node{ crt_thread };
+
+    -    // TODO: validate
+    clock* clk = (clock_ != nullptr) ? clock_ : &sysclock;
+    internal::clock_timestamps_list& clock_list = clk->steady_list ();
+    clock::timestamp_t timeout_timestamp = clk->steady_now () + timeout;
+
+    // Prepare a timeout node pointing to the current thread.
+    internal::timeout_thread_node timeout_node{ timeout_timestamp,
+                                                crt_thread };
+
+    result_t res;
+    -    res = mutex.unlock ();
+    {
+    // ----- Enter critical section -----

```



```

+ // Link and unlock as one step for this CPU's scheduler; see wait().
+ scheduler::critical_section scs;

- if (res != result::ok)
+ {
- instrumentation::condition_variable::timed_wait_retval (this, res);
- return res;
+ // ----- Enter critical section -----
+ interrupts::critical_section ics;
+
+ // Add this thread to the condition variable waiting list,
+ // and the clock timeout list.
+ scheduler::internal_link_node (
+     list_, node, clock_list, timeout_node,
+     OS_INTEGER_INSTRUMENTATION_SUSPEND_CAUSE_CONDVAR);
+ // state::suspended set in above link().
+ // ----- Exit critical section -----
+ }

- {
- // Add this thread to the condition variable waiting list.
- list_.link (node);
- node.thread->waiting_node_ = &node;
-
- res = mutex.timed_lock (timeout);
+ res = mutex.unlock ();

- // Remove the thread from the node waiting list,
- // if not already removed.
- node.thread->waiting_node_ = nullptr;
- node.unlink ();
+ if (res != result::ok)
+ {
+     scheduler::internal_unlink_node (node, timeout_node);
+     instrumentation::condition_variable::timed_wait_retval (this, res);
+     return res;
+ }
+ // ----- Exit critical section -----
+ }

+ port::scheduler::reschedule ();

+
+ // Remove the thread from the condition variable waiting list,
+ // if not already removed by signal() / broadcast() and from the clock
+ // timeout list, if not already removed by the timer.

```

```
+ scheduler::internal_unlink_node (node, timeout_node);
+
+ res = mutex.lock ();
+
+ if (res == result::ok && clk->steady_now () >= timeout_timestamp)
+ {
+     res = ETIMEDOUT;
+ }
+
+ instrumentation::condition_variable::timed_wait_retval (this, res);
+ return res;
+
+#endif
+
+ }
```

// -----

smp-step(11): std::thread join waits for native handle, retained functor for deterministic free

```

diff --git a/include/cmsis-plus/estd/thread_internal.h b/include/cmsis-plus/estd/thread_internal.h
index 1d97128e..389498e8 100644
--- a/include/cmsis-plus/estd/thread_internal.h
+++ b/include/cmsis-plus/estd/thread_internal.h
@@ -146,6 +146,12 @@ private:
    using function_object_deleter_t = void (*) (void*);
    function_object_deleter_t function_object_deleter_ = nullptr;

+ // The bound function object, remembered here so it can be deleted even
+ // after the kernel has cleared the thread's `func_args_` on exit. Relying
+ // on `native_thread->function_args()` leaks it whenever the thread has
+ // already finished by the time `join()` runs (the common case).
+ void* function_object_ = nullptr;
+
 public:
 };

@@ -358,6 +364,10 @@ thread::thread (Callable_T&& f, Args_T&&... args)
    Function_object* funct_obj = new Function_object (std::bind (
        std::forward<Callable_T> (f), std::forward<Args_T> (args)...));

+ // Remember the object ourselves; the kernel may clear its own copy when
+ // the thread exits, before `join()` gets a chance to read it.
+ function_object_ = funct_obj;
+
    // The function to start the thread is a custom proxy that
    // knows how to get the variadic arguments.
    #pragma GCC diagnostic push
diff --git a/src/libcpp/thread-cpp.h b/src/libcpp/thread-cpp.h
index c14f5bba..af572d23 100644
--- a/src/libcpp/thread-cpp.h
+++ b/src/libcpp/thread-cpp.h
@@ -36,11 +36,14 @@ thread::delete_system_thread (void)
 {
    if (id_ != id ())
    {
-       void* args = id_.native_thread->function_args ();
-       if (args != nullptr && function_object_deleter_ != nullptr)
+       if (function_object_ != nullptr && function_object_deleter_ != nullptr)
        {
            // Manually delete the function object used to store arguments.
-            function_object_deleter_ (args);
+            function_object_deleter_ (function_object_);
+            // `function_object_` is our own copy: the kernel clears its
+            // `func_args_` on exit, so reading it here would leak whenever the

```

```

+         // thread has already finished.
+         function_object_deleter_ (function_object_);
+         function_object_ = nullptr;
+     }

    // Manually delete the system thread.
@@ -68,6 +71,7 @@ thread::swap (thread& t) noexcept
{
    std::swap (id_, t.id_);
    std::swap (function_object_deleter_, t.function_object_deleter_);
+ std::swap (function_object_, t.function_object_);
}

bool
@@ -81,7 +85,28 @@ thread::join ()
{
    os::trace::printf ("%s() @%p\n", __func__, this);

- delete_system_thread ();
+ if (id_ != id ())
+ {
+     // Wait for the thread to end, as ISO join() does, before freeing what
+     // it runs on. Deleting it straight away -- which is what this did --
+     // frees its bound arguments and kills the system thread; on one core
+     // the thread had usually finished by then, on SMP it is still running
+     // on another core.
+     id_.native_thread->join ();
+
+     if (function_object_ != nullptr && function_object_deleter_ != nullptr)
+     {
+         // Manually delete the function object used to store arguments.
+         // Use our own pointer, because the kernel clears `func_args_` when
+         // the thread exits -- which happens before `join()` returns for any
+         // short-lived thread.
+         function_object_deleter_ (function_object_);
+         function_object_ = nullptr;
+     }
+
+     // Manually delete the system thread, destroyed by now.
+     delete id_.native_thread_;
+ }

    id_ = id ();
    os::trace::printf ("%s() @%p joined\n", __func__, this);

```

smp-step(12): message queue reschedule after send/receive wakeups

```

diff --git a/src/rtos/os-mqueue.cpp b/src/rtos/os-mqueue.cpp
index 3f134fae..cd46c58a 100644
--- a/src/rtos/os-mqueue.cpp
+++ b/src/rtos/os-mqueue.cpp
@@ -961,18 +961,27 @@ namespace os

#else

+    bool sent;
+    {
+        // ----- Enter critical section -----
+        interrupts::critical_section ics;

-        if (internal_try_send_ (msg, nbytes, mprio))
-        {
-            instrumentation::message_queue::send_retval (this, result::ok);
-            return result::ok;
-        }
+        sent = internal_try_send_ (msg, nbytes, mprio);
+        // ----- Exit critical section -----
+    }

+    if (sent)
+    {
+        // internal_try_send_() resumed a waiting receiver. Rescheduling here,
+        // with the kernel lock released, lets it run at once -- as the ARM
+        // ports do when PendSV is taken on the way out of the section --
+        // instead of waiting for the next tick.
+        port::scheduler::reschedule ();

+        instrumentation::message_queue::send_retval (this, result::ok);
+        return result::ok;
+    }

+    thread& crt_thread = this_thread::thread ();

    // Prepare a list node pointing to the current thread.
@@ -1083,24 +1092,30 @@ namespace os
    // Don't call this from high priority interrupts.
    assert (port::interrupts::is_priority_valid ());

+    bool sent;
+    {
+        // ----- Enter critical section -----

```

```

        interrupts::critical_section ics;

-    if (internal_try_send_ (msg, nbytes, mprio))
-    {
-        instrumentation::message_queue::try_send_retval (this, result::ok);
-        return result::ok;
-    }
-    else
-    {
-        instrumentation::message_queue::try_send_retval (this,
-                                                            EWOULDBLOCK);
-        return EWOULDBLOCK;
-    }
+    sent = internal_try_send_ (msg, nbytes, mprio);
    // ----- Exit critical section -----
}

+    if (sent)
+    {
+        // internal_try_send_() resumed a waiting receiver. Rescheduling here,
+        // with the kernel lock released, lets it run at once -- as the ARM
+        // ports do when PendSV is taken on the way out of the section --
+        // instead of waiting for the next tick.
+        port::scheduler::reschedule ();
+
+        instrumentation::message_queue::try_send_retval (this, result::ok);
+        return result::ok;
+    }
+
+    instrumentation::message_queue::try_send_retval (this, EWOULDBLOCK);
+    return EWOULDBLOCK;
+
#endif
}

@@ -1185,19 +1200,27 @@ namespace os

    // Extra test before entering the loop, with its inherent weight.
    // Trade size for speed.
+    bool sent;
    {
        // ----- Enter critical section -----
        interrupts::critical_section ics;

-        if (internal_try_send_ (msg, nbytes, mprio))

```



```

-     {
-         instrumentation::message_queue::timed_send_retval (this,
-                                                             result::ok);
-         return result::ok;
-     }
+     sent = internal_try_send_ (msg, nbytes, mprio);
+     // ----- Exit critical section -----
+ }

+ if (sent)
+ {
+     // internal_try_send_() resumed a waiting receiver. Rescheduling here,
+     // with the kernel lock released, lets it run at once -- as the ARM
+     // ports do when PendSV is taken on the way out of the section --
+     // instead of waiting for the next tick.
+     port::scheduler::reschedule ();
+
+     instrumentation::message_queue::timed_send_retval (this, result::ok);
+     return result::ok;
+ }
+
+ thread& crt_thread = this_thread::thread ();

+     // Prepare a list node pointing to the current thread.
@@ -1338,18 +1361,26 @@ namespace os

+     // Extra test before entering the loop, with its inherent weight.
+     // Trade size for speed.
+ bool received;
+ {
+     // ----- Enter critical section -----
+     interrupts::critical_section ics;

-     if (internal_try_receive_ (msg, nbytes, mprio))
-     {
-         instrumentation::message_queue::receive_retval (this, result::ok);
-         return result::ok;
-     }
+     received = internal_try_receive_ (msg, nbytes, mprio);
+     // ----- Exit critical section -----
+ }

+ if (received)
+ {
+     // internal_try_receive_() resumed a waiting sender. Rescheduling

```

```

+         // here, with the kernel lock released, lets it run at once instead
+         // of waiting for the next tick.
+         port::scheduler::reschedule ();
+
+         instrumentation::message_queue::receive_retval (this, result::ok);
+         return result::ok;
+     }
+
+     thread& crt_thread = this_thread::thread ();

    // Prepare a list node pointing to the current thread.
@@ -1463,25 +1494,29 @@ namespace os
    // Don't call this from high priority interrupts.
    assert (port::interrupts::is_priority_valid ());

+     bool received;
+     {
+         // ----- Enter critical section -----
+         interrupts::critical_section ics;

-         if (internal_try_receive_ (msg, nbytes, mprio))
-         {
-             instrumentation::message_queue::try_receive_retval (this,
-                                                                     result::ok);
-             return result::ok;
-         }
-         else
-         {
-             instrumentation::message_queue::try_receive_retval (this,
-                                                                     EWOULDBLOCK);
-             return EWOULDBLOCK;
-         }
+         received = internal_try_receive_ (msg, nbytes, mprio);
+         // ----- Exit critical section -----
+     }

+     if (received)
+     {
+         // internal_try_receive_() resumed a waiting sender. Rescheduling
+         // here, with the kernel lock released, lets it run at once instead
+         // of waiting for the next tick.
+         port::scheduler::reschedule ();
+
+         instrumentation::message_queue::try_receive_retval (this, result::ok);
+         return result::ok;

```

```

+     }
+
+     instrumentation::message_queue::try_receive_retval (this, EWOULDBLOCK);
+     return EWOULDBLOCK;
+
+ #endif
+     }
+
@@ -1578,19 +1613,27 @@ namespace os

    // Extra test before entering the loop, with its inherent weight.
    // Trade size for speed.
+    bool received;
+    {
+        // ----- Enter critical section -----
+        interrupts::critical_section ics;

-        if (internal_try_receive_ (msg, nbytes, mprio))
-        {
-            instrumentation::message_queue::timed_receive_retval (this,
-                                                                    result::ok);
-            return result::ok;
-        }
+        received = internal_try_receive_ (msg, nbytes, mprio);
+        // ----- Exit critical section -----
+    }

+    if (received)
+    {
+        // internal_try_receive_() resumed a waiting sender. Rescheduling
+        // here, with the kernel lock released, lets it run at once instead
+        // of waiting for the next tick.
+        port::scheduler::reschedule ();
+
+        instrumentation::message_queue::timed_receive_retval (this,
+                                                                result::ok);
+        return result::ok;
+    }
+
    thread& crt_thread = this_thread::thread ();

    // Prepare a list node pointing to the current thread.

```

**smp-step(13): highres clock port sync (kernel decl+call, cortexm ICSR, posix-arch
CLOCK_MONOTONIC)**

```

diff --git a/include/cmsis-plus/rtos/os-decls.h b/include/cmsis-plus/rtos/os-decls.h
index 5d625d5b..2e39d590 100644
--- a/include/cmsis-plus/rtos/os-decls.h
+++ b/include/cmsis-plus/rtos/os-decls.h
@@ -22,6 +22,11 @@
 #endif

 // Include the non-portable portable types, enums and constants declarations.
 #include <cmsis-plus/rtos/port/os-decls.h>
+// Port compat (develop/xbpack-development): the C-ABI declarations were split
+// out of the C++ <os.h> chain, but kernel C++ sources (os_systick_handler in
+// os-clocks.cpp) and the ports (os_thread_t) still rely on them being visible
+// via <os.h>, as in v7.0.1. Needed by both the SMP and single-CPU builds.
+#include <cmsis-plus/rtos/os-c-decls.h>

#include <cmsis-plus/diag/trace.h>

@@ -1025,6 +1030,12 @@ namespace os
     static void
     start (void);

+    static constexpr bool
+    has_hardware_counter (void) noexcept;
+
+    static uint64_t
+    hardware_counter (void) noexcept;
+
+    static uint32_t
+    cycles_per_tick (void);

diff --git a/src/rtos/os-clocks.cpp b/src/rtos/os-clocks.cpp
index f91f2ae3..94b457ff 100644
--- a/src/rtos/os-clocks.cpp
+++ b/src/rtos/os-clocks.cpp
@@ -793,11 +793,19 @@ namespace os
     clock::timestamp_t
     clock_highres::now (void)
     {
-        // ----- Enter critical section -----
-        interrupts::critical_section ics;
+        if constexpr (port::clock_highres::has_hardware_counter ())
+        {
+            return port::clock_highres::hardware_counter ();
+        }
+        else
     {
         // ----- Enter critical section -----
         interrupts::critical_section ics;
     }
     if constexpr (port::clock_highres::has_hardware_counter ())
     {
         return port::clock_highres::hardware_counter ();
     }
     else

```

```

+      {
+          // Prevent inconsistent values.
+          // ----- Enter critical section -----
+          interrupts::critical_section ics;

-      return steady_count_ + port::clock_highres::cycles_since_tick ();
-      // ----- Exit critical section -----
+      return steady_count_ + port::clock_highres::cycles_since_tick ();
+      // ----- Exit critical section -----
+      }
    }

    // -----

```

Part B — Step 14 (SMP) — kernel

```

diff --git a/include/cmsis-plus/rtos/os-c-decls.h b/include/cmsis-plus/rtos/os-c-decls.h
index 86dc5429..e3cd9bb30 100644
--- a/include/cmsis-plus/rtos/os-c-decls.h
+++ b/include/cmsis-plus/rtos/os-c-decls.h
@@ -316,7 +316,6 @@ extern "C"
     * @brief Used to check reused threads.
     */
     os_thread_state_initialising = 6,
-
-    /**
     * @brief In process of being destroyed.
     */
@@ -546,6 +545,10 @@ extern "C"
     */
     bool th_enable_assert_reuse;

+#if defined(OS_USE_SMP_SCHEDULER)
+    uint32_t th_cpu_affinity;
+#endif
+
+    } os_thread_attr_t;

    /**
@@ -592,6 +595,9 @@ extern "C"
     os_thread_state_t state;
     os_thread_prio_t prio_assigned;
     os_thread_prio_t prio_inherited;
+#if defined(OS_USE_SMP_SCHEDULER)
+    uint32_t cpu_affinity;
+#endif /* defined(OS_USE_SMP_SCHEDULER) */
     bool interrupted;
     os_internal_evflags_t event_flags;
     #if defined(OS_INCLUDE_RTOS_CUSTOM_THREAD_USER_STORAGE)
diff --git a/include/cmsis-plus/rtos/os-sched.h b/include/cmsis-plus/rtos/os-sched.h
index 0a2b4e78..f3b6ab60 100644
--- a/include/cmsis-plus/rtos/os-sched.h
+++ b/include/cmsis-plus/rtos/os-sched.h
@@ -55,7 +55,11 @@ namespace os

    #if !defined(OS_USE_RTOS_PORT_SCHEDULER)
        extern bool is_preemptive_;
    #if defined(OS_USE_SMP_SCHEDULER)
+        extern thread* volatile current_thread_[OS_NCPU];
    #else

```



```

extern thread* volatile current_thread_;
#endif /* defined(OS_USE_SMP_SCHEDULER) */

extern internal::ready_threads_list ready_threads_list_;
#endif /* !defined(OS_USE_RTOS_PORT_SCHEDULER) */

diff --git a/include/cmsis-plus/rtos/os-thread.h b/include/cmsis-plus/rtos/os-thread.h
index cac6777f..385e6cb2 100644
--- a/include/cmsis-plus/rtos/os-thread.h
+++ b/include/cmsis-plus/rtos/os-thread.h
@@ -745,6 +745,14 @@ namespace os
    friend port::stack::element_t*
    port::scheduler::switch_stacks (port::stack::element_t* sp);

+#if defined(OS_USE_SMP_SCHEDULER)
+    // The SMP picker accesses state_ / context_ directly.
+    friend void
+    rtos::scheduler::internal_switch_threads (void);
+    // The idle reaper reads context_ to know a terminated thread's
+    // context is no longer live on another CPU (see os-idle.cpp).
+    friend void ::os_rtos_idle_actions (void);
+#endif /* defined(OS_USE_SMP_SCHEDULER) */

#endif

/**
@@ -880,6 +888,9 @@ namespace os

    bool th_enable_assert_reuse = false;

+#if defined(OS_USE_SMP_SCHEDULER)
+    uint32_t th_cpu_affinity = 0xFFFFFFFFu;
+#endif

    // Add more attributes here.

@@ -1198,6 +1209,13 @@ namespace os
    priority_t
    priority_inherited (void);

+#if defined(OS_USE_SMP_SCHEDULER)
+    uint32_t
+    cpu_affinity (void) const;
+
+    void
+    cpu_affinity (uint32_t mask);
+#endif /* defined(OS_USE_SMP_SCHEDULER) */

```

```

    #if 0
        // ???
@@ -1696,6 +1714,9 @@ namespace os
    priority_t volatile prio_assigned_ = priority::none;
    priority_t volatile prio_inherited_ = priority::none;

+#if defined(OS_USE_SMP_SCHEDULER)
+    uint32_t cpu_affinity_ = 0xFFFFFFFFu;
+#endif /* defined(OS_USE_SMP_SCHEDULER) */

    bool volatile interrupted_ = false;

diff --git a/src/rtos/os-c-wrapper.cpp b/src/rtos/os-c-wrapper.cpp
index 4bd9e1bb..1c37050d 100644
--- a/src/rtos/os-c-wrapper.cpp
+++ b/src/rtos/os-c-wrapper.cpp
@@ -239,6 +239,8 @@ static_assert (os_thread_state_terminated == thread::state::terminated,
    "adjust os_thread_state_terminated");
    static_assert (os_thread_state_destroyed == thread::state::destroyed,
    "adjust os_thread_state_destroyed");
+static_assert (os_thread_state_destroying == thread::state::destroying,
+    "adjust os_thread_state_destroying");

    static_assert (os_timer_once == timer::run::once, "adjust os_timer_once");
    static_assert (os_timer_periodic == timer::run::periodic,

@@ -3538,6 +3540,12 @@ osThreadCreate (const osThreadDef_t* thread_def, void* args)
    thread::attributes attr;
    attr.th_priority = thread_def->tpriority;
    attr.th_stack_size_bytes = thread_def->stacksize;
+#if defined(OS_USE_SMP_SCHEDULER)
+    // CMSIS-RTOS v1 assumes a single-core priority scheduling model;
+    // pin threads created through the CMSIS-RTOS v1 API to Core 0 at construction.
+    attr.th_cpu_affinity = (1u << 0);
+#endif
+
    // Creating thread with invalid priority should fail (validator requirement).
    if (thread_def->tpriority >= osPriorityError)
    {
@@ -3575,6 +3583,12 @@ osThreadCreate (const osThreadDef_t* thread_def, void* args)
        args, attr);

#pragma GCC diagnostic pop

+#if defined(OS_USE_SMP_SCHEDULER)
+    // CMSIS-RTOS v1 assumes a single-core priority scheduling model;

```

```

+         // pin threads created through the CMSIS-RTOS v1 API to Core 0.
+         th->cpu_affinity (1u << 0);
+
+
+         // No need to yield here, already done by constructor.
+         return reinterpret_cast<osThreadId> (th);
+     }
diff --git a/src/rtos/os-core.cpp b/src/rtos/os-core.cpp
index c9d6859e..8b88a471 100644
--- a/src/rtos/os-core.cpp
+++ b/src/rtos/os-core.cpp
@@ -24,6 +24,11 @@

// -----

+#if defined(OS_USE_SMP_SCHEDULER)
+// Provided by the SMP port (e.g. Cortex-A7); returns the current core index.
+extern "C" unsigned port_cpu_id(void);
+#endif /* defined(OS_USE_SMP_SCHEDULER) */
+
+namespace
+{
+    #if defined(OS_HAS_INTERRUPTS_STACK)
@@ -130,10 +135,21 @@ namespace os

#pragma GCC diagnostic push
#pragma GCC diagnostic ignored "-Wcast-align"
+#if defined(OS_USE_SMP_SCHEDULER)
+    // One running thread per CPU, indexed by port_cpu_id().
+    thread* volatile current_thread_[OS_NCPU]
+    = {reinterpret_cast<thread*> (&tiny_thread)};
+#else
+    thread* volatile current_thread_
+    = reinterpret_cast<thread*> (&tiny_thread);
+#endif /* defined(OS_USE_SMP_SCHEDULER) */
+    #pragma GCC diagnostic pop

+#if defined(OS_USE_SMP_SCHEDULER)
+    // One idle thread per CPU (secondary cores register theirs at boot).
+    thread* os_idle_thread_core[OS_NCPU] = {nullptr};
+#endif /* defined(OS_USE_SMP_SCHEDULER) */
+
+    #pragma GCC diagnostic push
+    #if defined(__clang__)
+    #pragma clang diagnostic ignored "-Wglobal-constructors"

```

```
@@ -462,6 +478,45 @@ namespace os
```

```
#if !defined(OS_USE_RTOS_PORT_SCHEDULER)
```

```
+#if defined(OS_USE_SMP_SCHEDULER)
```

```
+    // Affinity gate: a per-core idle thread is pinned to its own core,  
+    // everything else follows its cpu_affinity() bitmask.  
+    //  
+    // Identity against os_idle_thread_core[] is the authoritative test: it  
+    // covers every core whatever the thread is named. The name rules are  
+    // only the fallback for the boot window, between the moment a secondary  
+    // idle thread is linked into the ready list by its constructor and the  
+    // moment it is registered in os_idle_thread_core[]. Without a rule for  
+    // core OS_NCPU-1 an unregistered idle thread inherits the default  
+    // 0xFFFFFFFF affinity, so another core can pick it out of the ready list  
+    // and run it on the same stack as its own core does.
```

```
+    static bool
```

```
+    is_thread_allowed_on_cpu (thread* th, unsigned cpu)
```

```
+    {
```

```
+        for (unsigned c = 0; c < OS_NCPU; ++c)
```

```
+        {
```

```
+            if (th == scheduler::os_idle_thread_core[c])
```

```
+            {
```

```
+                return (cpu == c);
```

```
+            }
```

```
+        }
```

```
+        const char* name = th->name ();
```

```
+        if (name != nullptr && name[0] == 'i' && name[1] == 'd'
```

```
+            && name[2] == 'l' && name[3] == 'e')
```

```
+        {
```

```
+            if (name[4] == '\\0' || name[4] == '0')
```

```
+            {
```

```
+                return (cpu == 0);
```

```
+            }
```

```
+            if (name[4] >= '1' && name[4] <= '9' && name[5] == '\\0')
```

```
+            {
```

```
+                return (cpu == static_cast<unsigned> (name[4] - '0'));
```

```
+            }
```

```
+        }
```

```
+        return (th->cpu_affinity () & (1u << cpu)) != 0;
```

```
+    }
```

```
+#endif /* defined(OS_USE_SMP_SCHEDULER) */
```

```
+
```

```
    void
```

```
    internal_switch_threads (void)
```

```

{
@@ -490,7 +545,11 @@ namespace os
    scheduler::statistics::cpu_cycles_ += delta;

    // Accumulate durations to old thread.
+#if defined(OS_USE_SMP_SCHEDULER)
+    scheduler::current_thread_[port_cpu_id()->statistics_.cpu_cycles_ += delta;
+#else
    scheduler::current_thread_->statistics_.cpu_cycles_ += delta;
+#endif /* defined(OS_USE_SMP_SCHEDULER) */

    // Remember the timestamp for the next context switch.
    scheduler::statistics::switch_timestamp_ = now;
@@ -501,6 +560,77 @@ namespace os
    // current thread and return the top priority thread.
    if (!locked ())
    {
+#if defined(OS_USE_SMP_SCHEDULER)
+        unsigned cpu = port_cpu_id ();
+        thread* old_thread = scheduler::current_thread_[cpu];
+
+        instrumentation::thread::suspended (
+            old_thread, OS_INTEGER_INSTRUMENTATION_SUSPEND_CAUSE_SWITCH);
+
+        bool is_old_idle = false;
+        for (unsigned c = 0; c < OS_NCPU; ++c)
+        {
+            if (old_thread == scheduler::os_idle_thread_core[c])
+            {
+                is_old_idle = true;
+                break;
+            }
+        }
+
+        if (!is_old_idle)
+        {
+            old_thread->internal_relink_running_ ();
+        }
+        else
+        {
+            old_thread->state_ = thread::state::ready;
+        }
+
+        // Affinity-aware pick from the ready list. Skip threads whose
+        // stack_ptr is still nullptr (their context is live in another

```

```

+         // CPU's registers -- publish is deferred to the asm restore path);
+
+         // this CPU's own outgoing thread may always be re-picked.
+
+         thread* next_thread = nullptr;
+
+         auto* sentinel = reinterpret_cast<internal::waiting_thread_node*> (
+             &scheduler::ready_threads_list_);
+
+         auto* node = const_cast<internal::waiting_thread_node*> (
+             static_cast<const volatile internal::waiting_thread_node*> (
+                 scheduler::ready_threads_list_.head ())));
+
+
+         while (node != nullptr && node != sentinel)
+         {
+             thread* th = node->thread_;
+
+             if (th != nullptr && is_thread_allowed_on_cpu (th, cpu)
+                 && (th == old_thread
+                     || (th->state_ != thread::state::running
+                         && th->context_.port_.stack_ptr != nullptr)))
+             {
+                 next_thread = th;
+                 node->unlink ();
+                 next_thread->state_ = thread::state::running;
+                 break;
+             }
+
+             node = static_cast<internal::waiting_thread_node*> (
+                 node->next ());
+         }
+
+
+         if (next_thread != nullptr)
+         {
+             scheduler::current_thread_[cpu] = next_thread;
+         }
+         else
+         {
+             scheduler::current_thread_[cpu]
+                 = scheduler::os_idle_thread_core[cpu];
+             if (scheduler::os_idle_thread_core[cpu] != nullptr)
+             {
+                 scheduler::os_idle_thread_core[cpu]->state_
+                     = thread::state::running;
+             }
+         }
+
+         instrumentation::thread::active (scheduler::current_thread_[cpu]);
+
+     #else
+
+         instrumentation::thread::suspended (
+             scheduler::current_thread_,

```

```

        OS_INTEGER_INSTRUMENTATION_SUSPEND_CAUSE_SWITCH);
@@ -512,6 +642,7 @@ namespace os
        = scheduler::ready_threads_list_.unlink_head ();

        instrumentation::thread::active (scheduler::current_thread_);
+endif /* defined(OS_USE_SMP_SCHEDULER) */
    }

    // ***** Pointer switched to new thread! *****
@@ -527,7 +658,11 @@ namespace os
        scheduler::statistics::context_switches++;

        // Increment new thread context switches.
+if defined(OS_USE_SMP_SCHEDULER)
+    scheduler::current_thread_[port_cpu_id()->statistics_.context_switches++;
+else
        scheduler::current_thread_->statistics_.context_switches++;
+endif /* defined(OS_USE_SMP_SCHEDULER) */

    #endif /* defined(OS_INCLUDE_RTOS_STATISTICS_THREAD_CONTEXT_SWITCHES) */
}
diff --git a/src/rtos/os-idle.cpp b/src/rtos/os-idle.cpp
index 0db1ab35..10272366 100644
--- a/src/rtos/os-idle.cpp
+++ b/src/rtos/os-idle.cpp
@@ -80,6 +80,28 @@ os_rtos_idle_actions (void)
        node = const_cast<internal::waiting_thread_node*> (
            scheduler::terminated_threads_list_.head ());
        thread* th = node->thread_;
+if defined(OS_USE_SMP_SCHEDULER)
+    // A thread links itself here in internal_exit_() and only then
+    // switches away, so on SMP it may still be running on another CPU.
+    // Destroying it now frees the stack that CPU is executing on, and
+    // lets the joiner delete the object under it. Reap it only once no
+    // CPU has it current and its context has been saved -- the same
+    // rule internal_switch_threads() applies to the ready list -- and
+    // otherwise leave it linked for the next idle pass.
+    bool live = (__atomic_load_n (&th->context_.port_.stack_ptr,
+        __ATOMIC_ACQUIRE)
+        == nullptr);
+    for (unsigned c = 0; c < OS_NCPU && !live; ++c)
+    {
+        live = (__atomic_load_n (&scheduler::current_thread_[c],
+            __ATOMIC_ACQUIRE)
+            == th);

```

```

+         }
+         if (live)
+         {
+             break;
+         }
+     }
+ }
+
+ #endif /* defined(OS_USE_SMP_SCHEDULER) */
+
+     if (th->state_ == thread::state::destroying
+         || th->state_ == thread::state::destroyed)
+     {
+
+ diff --git a/src/rtos/os-main.cpp b/src/rtos/os-main.cpp
+ index 67782880..546c709e 100644
+ --- a/src/rtos/os-main.cpp
+ +++ b/src/rtos/os-main.cpp
+ @@ -165,6 +165,9 @@ int
+
+         thread::attributes attr = thread::initializer;
+         attr.th_stack_size_bytes = OS_INTEGER_RTOS_MAIN_STACK_SIZE_BYTES;
+
+ #if defined(OS_USE_SMP_SCHEDULER)
+ + attr.th_cpu_affinity = (1u << 0);
+ #endif
+
+ #pragma GCC diagnostic push
+ #if defined(__clang__)
+ #pragma clang diagnostic ignored "-Wcast-function-type-strict"
+ @@ -176,6 +179,12 @@ int
+
+ #endif /* defined(OS_EXCLUDE_DYNAMIC_MEMORY_ALLOCATIONS) */
+
+
+ #if defined(OS_USE_SMP_SCHEDULER)
+ + // Pin main thread to core 0 so per-core peripherals (NVIC, SysTick) initialized
+ + // on the boot core remain consistent for the main thread lifecycle.
+ + os_main_thread->cpu_affinity (1u << 0);
+ #endif
+
+
+ #if !defined(OS_USE_RTOS_PORT_SCHEDULER)
+     os_startup_create_thread_idle ();
+ #endif /* !defined(OS_USE_RTOS_PORT_SCHEDULER) */
+
+ diff --git a/src/rtos/os-thread.cpp b/src/rtos/os-thread.cpp
+ index 79b0a71c..0aff28a9 100644
+ --- a/src/rtos/os-thread.cpp
+ +++ b/src/rtos/os-thread.cpp
+ @@ -19,6 +19,16 @@
+
+ #include <memory>
+ #include <stdexcept>
+
+ #if defined(OS_USE_SMP_SCHEDULER)

```



```

+extern "C" unsigned port_cpu_id(void);
+extern "C" void port_smp_ipi(unsigned cpu) __attribute__((weak));
+
+extern "C" void __attribute__((weak))
+port_smp_ipi (unsigned cpu)
+{
+ (void) cpu;
+}
+#endif /* defined(OS_USE_SMP_SCHEDULER) */

// -----

@@ -272,6 +282,9 @@ namespace os
    // Must be explicit here, since they are not done in the members
    // declarations to allow th_enable_assert_reuse.
    state_ = state::initializing;
+#if defined(OS_USE_SMP_SCHEDULER)
+    cpu_affinity_ = 0xFFFFFFFFu;
+#endif /* defined(OS_USE_SMP_SCHEDULER) */

    instrumentation::thread::create_return (this);
}

@@ -286,6 +299,9 @@ namespace os
    // Must be explicit here, since they are not done in the members
    // declarations to allow th_enable_assert_reuse.
    state_ = state::initializing;
+#if defined(OS_USE_SMP_SCHEDULER)
+    cpu_affinity_ = 0xFFFFFFFFu;
+#endif /* defined(OS_USE_SMP_SCHEDULER) */

    instrumentation::thread::create_return (this);
}

@@ -440,6 +456,9 @@ namespace os
    #endif /* DEBUG */

    state_ = state::initializing;
+#if defined(OS_USE_SMP_SCHEDULER)
+    cpu_affinity_ = attr.th_cpu_affinity;
+#endif /* defined(OS_USE_SMP_SCHEDULER) */

    allocator_ = &allocator;

@@ -578,7 +597,11 @@ namespace os

    if (!scheduler::started ())

```

```

    {
+ #if defined(OS_USE_SMP_SCHEDULER)
+     scheduler::current_thread_[port_cpu_id()] = this;
+ #else
+     scheduler::current_thread_ = this;
+ #endif /* defined(OS_USE_SMP_SCHEDULER) */
    }

    // Add to ready list, but do not yield yet.
@@ -697,6 +720,23 @@ namespace os

    port::scheduler::reschedule ();

+ #if defined(OS_INTEGER_RTOS_PORT_NCPU) && (OS_INTEGER_RTOS_PORT_NCPU > 1)
+     // If the waking CPU is not in this thread's affinity mask,
+     // actively trigger a reschedule IPI to an eligible core so it
+     // doesn't wait up to 1 ms for its next local clock tick.
+     unsigned this_cpu = port_cpu_id ();
+     if ((cpu_affinity () & (1u << this_cpu)) == 0)
+     {
+         for (unsigned c = 0; c < OS_NCPU; ++c)
+         {
+             if (c != this_cpu && (cpu_affinity () & (1u << c)) != 0)
+             {
+                 port_smp_ipi (c);
+                 break;
+             }
+         }
+     }
+ #endif

+ #endif

@@ -709,6 +749,19 @@ namespace os
    *
    * @note Can be invoked from Interrupt Service Routines.
    */
+ #if defined(OS_USE_SMP_SCHEDULER)
+     uint32_t
+     thread::cpu_affinity (void) const
+     {
+         return cpu_affinity_;
+     }
+
+     void

```

```

+   thread::cpu_affinity (uint32_t mask)
+   {
+       cpu_affinity_ = mask;
+   }
+ #endif /* defined(OS_USE_SMP_SCHEDULER) */

    thread::priority_t
    thread::priority (void)
@@ -1064,6 +1117,30 @@ namespace os
        port::scheduler::reschedule ();
    }

+ #if defined(OS_USE_SMP_SCHEDULER)
+     // On SMP, the joined thread may still be running on another CPU
+     // completing its final reschedule() context switch. Wait until it is
+     // no longer current on any CPU before returning, so the caller can
+     // safely reclaim the thread object and stack memory.
+     for (;;)
+     {
+         bool still_running = false;
+         for (unsigned c = 0; c < OS_NCPU; ++c)
+         {
+             if (__atomic_load_n (&scheduler::current_thread_[c],
+                                   __ATOMIC_ACQUIRE) == this)
+             {
+                 still_running = true;
+                 break;
+             }
+         }
+         if (!still_running)
+         {
+             break;
+         }
+         this_thread::yield ();
+     }
+ #endif

    #if defined(OS_TRACE_RTOS_THREAD)
        trace::printf ("%s() @%p %s joined\n", __func__, this, name ());
@@ -1384,6 +1461,69 @@ namespace os
        // ----- Enter critical section -----
        scheduler::critical_section scs;

+ #if defined(OS_USE_SMP_SCHEDULER)
+     // On SMP the thread may be running on another CPU -- typically

```

```

+ // still inside its own internal_exit_(), right after the event that
+ // let this caller go on -- or be terminated and already taken off
+ // the funeral list by an idle reaper, which destroys it outside the
+ // lock. Destroying it here in either case frees the stack another
+ // CPU runs on, or destroys it twice. So wait, with the lock
+ // released, until it is off every CPU and unclaimed; from then on
+ // the lock held keeps it so.
+ for (;;)
+ {
+     if (__atomic_load_n (&state_, __ATOMIC_ACQUIRE) == state::destroyed)
+     {
+         break;
+     }
+
+     bool busy = (__atomic_load_n (&context_.port_.stack_ptr,
+                                   __ATOMIC_ACQUIRE)
+                 == nullptr);
+     unsigned busy_cpu = OS_NCPU;
+     for (unsigned c = 0; c < OS_NCPU; ++c)
+     {
+         if (__atomic_load_n (&scheduler::current_thread_[c],
+                               __ATOMIC_ACQUIRE)
+             == this)
+         {
+             busy = true;
+             busy_cpu = c;
+             break;
+         }
+     }
+
+     thread::state_t st = __atomic_load_n (&state_, __ATOMIC_ACQUIRE);
+     if (st == state::destroying)
+     {
+         busy = true;
+     }
+     else if (st == state::terminated && ready_node_.unlinked ())
+     {
+         // Idle reaper claimed it and will destroy it.
+         busy = true;
+     }
+
+     if (!busy)
+     {
+         break;
+     }
+ }

```

```

+
+         // Actively trigger reschedule IPI if running on another core
+         if (busy_cpu < OS_NCPU && busy_cpu != port_cpu_id ())
+         {
+             port_smp_ipi (busy_cpu);
+         }
+
+         {
+             // ----- Enter uncritical section -----
+             scheduler::uncritical_section sucs;
+
+             this_thread::yield ();
+             // ----- Exit uncritical section -----
+         }
+     }
+
+#endif /* defined(OS_USE_SMP_SCHEDULER) */

    if (state_ == state::destroyed)
    {
@@ -1845,7 +1985,22 @@ namespace os

    #else

+#if defined(OS_USE_SMP_SCHEDULER)
+    {
+        // Read the core id and that core's current thread with this core's
+        // interrupts masked. With them enabled, a preemption between the
+        // two reads can move the caller to another core, and it would get
+        // back the thread now running on the core it left -- as a mutex
+        // owner, a waiter, or the errno it writes. The IRQ critical section
+        // is the per-core mask every port provides; it also takes the
+        // kernel lock, recursively, so a caller already inside one pays
+        // nothing more.
+        interrupts::critical_section ics;
+        th = scheduler::current_thread_[port_cpu_id ()];
+    }
+#else
+        th = scheduler::current_thread_;
+#endif /* defined(OS_USE_SMP_SCHEDULER) */

    #endif

    return th;

```



```

diff --git a/include/cmsis-plus/rtos/port/os-inlines.h b/include/cmsis-plus/rtos/port/os-inlines.h
index 93944a5..0ec7ab7 100644
--- a/include/cmsis-plus/rtos/port/os-inlines.h
+++ b/include/cmsis-plus/rtos/port/os-inlines.h
@@ -129,6 +129,21 @@ namespace os

    // =====

+    inline constexpr bool __attribute__((always_inline))
+    clock_highres::has_hardware_counter (void) noexcept
+    {
+        return true;
+    }
+
+    inline uint64_t __attribute__((always_inline))
+    clock_highres::hardware_counter (void) noexcept
+    {
+        timespec tp;    // NB: no 'struct' – baseline gate has -Werror=redundant-tags
+        ::clock_gettime (CLOCK_MONOTONIC, &tp);
+        return static_cast<uint64_t> (tp.tv_sec) * 1000000ULL
+            + static_cast<uint64_t> (tp.tv_nsec) / 1000ULL;
+    }
+
+    } /* namespace port */
+    } /* namespace rtos */
+    } /* namespace os */
diff --git a/CMakeLists.txt b/CMakeLists.txt
index ae85397..ba48588 100644
--- a/CMakeLists.txt
+++ b/CMakeLists.txt
@@ -49,6 +49,9 @@ target_include_directories(micro-os-plus-iii-posix-arch-interface INTERFACE
    target_sources(micro-os-plus-iii-posix-arch-interface INTERFACE
        src/diag/trace-posix.cpp
        src/rtos/os-core.cpp
+    + src/host_cpu.cpp
+    + src/free-store.cpp
+    + src/exception_handler.cpp
    )

    target_compile_definitions(micro-os-plus-iii-posix-arch-interface INTERFACE
diff --git a/include/cmsis-plus/rtos/port/os-c-decls.h b/include/cmsis-plus/rtos/port/os-c-decls.h
index c989c07..a274f67 100644
--- a/include/cmsis-plus/rtos/port/os-c-decls.h
+++ b/include/cmsis-plus/rtos/port/os-c-decls.h

```

```

@@ -1,29 +1,15 @@
/*
+ * os-c-decls.h - the width-dependent half of the POSIX port contract.
+ *
+ * This file is part of the µOS++ project (https://micro-os-plus.github.io/).
+ * Copyright (c) 2016-2025 Liviu Ionescu. All rights reserved.
+ * Copyright (c) 2026 Dan. All rights reserved.
+ *
+ * Permission to use, copy, modify, and/or distribute this software
+ * for any purpose is hereby granted, under the terms of the MIT license.
+ *
- * If a copy of the license was not distributed with this file, it can
- * be obtained from https://opensource.org/licenses/mit/.
- */
-
-/*
- * The initial CMSIS++ RTOS API was inspired by CMSIS RTOS API v1.x,
- * Copyright (c) 2013 ARM LIMITED
- */
-
-/*
- * This file is part of the CMSIS++ proposal, intended as a CMSIS
- * replacement for C++ applications.
- *
- * It is included in `cmsis-plus/rtos/os-c-decls.h` to customise
- * it with port specific declarations.
- *
- * These structures (which basically contain handlers)
- * are conditionally included in the system objects
- * when they are implemented using the port native objects.
+ * Derived from upstream micro-os-plus-iii-posix-arch v1.0.1. What changed and
+ * why is in os-decls.h, beside the declarations it changed.
+ */

#ifdef CMSIS_PLUS_RTOS_PORT_OS_C_DECLS_H_
@@ -69,17 +55,39 @@ typedef uint32_t os_port_clock_duration_t;
// Must match port::clock::offset_t
typedef uint64_t os_port_clock_offset_t;

+typedef uint64_t os_port_thread_stack_element_t;
+typedef uint64_t os_port_thread_stack_allocation_element_t;
+
+/*
+ * The thread context.
+ *

```



```

+ * `stack_ptr` FIRST and `ucontext` second, and the order is not cosmetic.
+ *
+ * Upstream's context was the ucontext alone, because upstream had one CPU and
+ * therefore never had to ask whether a saved context was safe to resume. The
+ * SMP scheduler does ask, in the kernel, in a line no port may edit:
+ *
+ *   src/rtos/os-core.cpp, internal_switch_threads():
+ *       && (th == old_thread || th->context_.port_.stack_ptr != nullptr)
+ *
+ * On the ARM ports that field is the outgoing stack pointer, published by the
+ * assembly restore path only once the CPU has left the outgoing thread's
+ * stack. Here it is the same gate carrying the same meaning -- "this context
+ * is fully saved, another CPU may claim it" -- and nothing more: the register
+ * state lives in the ucontext. A CPU clears it when it claims a thread and
+ * the NEXT context to run on that CPU publishes it, which is the whole of the
+ * deferred-publish rule. See host_cpu.cpp.
+ */
+
+ typedef struct
+ {
+     - os_impl_ucontext_t ucontext; //
+     + os_port_thread_stack_element_t* stack_ptr;
+     + os_impl_ucontext_t ucontext;
+ } os_port_thread_context_t;
+
+ typedef bool os_port_scheduler_state_t;
+
+ -// Signal set (true if signal blocked)
+ +/* True when this CPU's interrupt signals are blocked. A per-thread signal
+ + * mask IS per-CPU interrupt masking, because a host thread IS a CPU. */
+
+ typedef bool os_port_irq_state_t;
+
+ -typedef uint64_t os_port_thread_stack_element_t;
+ -typedef uint64_t os_port_thread_stack_allocation_element_t;
+ -
+ #endif /* CMSIS_PLUS_RTOS_PORT_OS_C_DECLS_H_ */
diff --git a/include/cmsis-plus/rtos/port/os-decls.h b/include/cmsis-plus/rtos/port/os-decls.h
index 4ee273a..9cc63ca 100644
--- a/include/cmsis-plus/rtos/port/os-decls.h
+++ b/include/cmsis-plus/rtos/port/os-decls.h
@@ -1,51 +1,56 @@
+ /*
+ + * os-decls.h - the C++ half of the POSIX port contract.
+ + *
+ + * This file is part of the µOS++ project (https://micro-os-plus.github.io/).
+ + * Copyright (c) 2016-2025 Liviu Ionescu. All rights reserved.

```

```

+ * Copyright (c) 2026 Dan. All rights reserved.
+ *
+ * Permission to use, copy, modify, and/or distribute this software
+ * for any purpose is hereby granted, under the terms of the MIT license.
+ *
- * If a copy of the license was not distributed with this file, it can
- * be obtained from https://opensource.org/licenses/mit/.
- */
-
-/*
- * This file is part of the CMSIS++ proposal, intended as a CMSIS
- * replacement for C++ applications.
- *
- * It is included in `cmsis-plus/rtos/os.h` to customise
- * it with POSIX specific declarations.
+ * This port does NOT link micro-os-plus::port-smp-decls, and that target's
+ * own README says why: the shared copy is written for ports whose state types
+ * are integers and whose thread context is a bare stack pointer. Neither is
+ * true here -- the interrupt state is a signal-mask bit, the scheduler state
+ * is a bool, and the context carries a ucontext. The SMP declarations the
+ * kernel actually reads (lock_state[], _smp_klock, _port_ctx_pending[]) are
+ * reproduced below with the same names, the same types and the same
+ * volatility, so the kernel cannot tell the two apart.
+ */

#ifndef CMSIS_PLUS_RTOS_PORT_OS_DECLS_H_
#define CMSIS_PLUS_RTOS_PORT_OS_DECLS_H_

-// -----
-
-#if defined(OS_USE_OS_APP_CONFIG_H)
-#include <cmsis-plus/os-app-config.h>
-#endif

#include <cmsis-plus/rtos/port/os-c-decls.h>

-// -----
+#if !defined(OS_NCPU)
+#define OS_NCPU (1)
+#endif

-#if !defined(OS_INTEGER_SYSTICK_FREQUENCY_HZ)
-#define OS_INTEGER_SYSTICK_FREQUENCY_HZ (1000)
-#endif

```

```

+/*
+ * Host stacks are large because a signal frame lands on them.
+ *
+ * Preemption here is a signal delivered on the running thread's own stack --
+ * deliberately, with no SA_ONSTACK, so that the whole signal frame migrates
+ * with the thread when another CPU resumes it (see host_cpu.cpp). That frame
+ * is several kilobytes of ucontext plus the FPU state, so upstream's 32 KiB
+ * minimum is kept rather than the 2 KiB the ARM ports use.
+ */
+
+if !defined(OS_INTEGER_RTOS_MIN_STACK_SIZE_BYTES)
+define OS_INTEGER_RTOS_MIN_STACK_SIZE_BYTES (32 * 1024)
+endif

+if !defined(OS_INTEGER_RTOS_DEFAULT_STACK_SIZE_BYTES)
+if defined(__linux__)
+define OS_INTEGER_RTOS_DEFAULT_STACK_SIZE_BYTES \
+    (2 * OS_INTEGER_RTOS_MIN_STACK_SIZE_BYTES)
+else
+define OS_INTEGER_RTOS_DEFAULT_STACK_SIZE_BYTES \
+    (OS_INTEGER_RTOS_MIN_STACK_SIZE_BYTES)
+endif /* defined(__linux__) */
+endif

+if !defined(OS_INTEGER_RTOS_MAIN_STACK_SIZE_BYTES)
@@ -58,59 +63,52 @@
+    (OS_INTEGER_RTOS_DEFAULT_STACK_SIZE_BYTES)
+endif

+// -----
+if !defined(OS_INTEGER_RTOS_STACK_FILL_MAGIC)
+define OS_INTEGER_RTOS_STACK_FILL_MAGIC (0xEFBEADDEEFBEADDEULL)
+endif

+ifndef __cplusplus
+/* Kept identical to the ARM ports so the same code reads on both. */
+if !defined(SMP_NO_OWNER)
+define SMP_NO_OWNER (0xFFFFFFFFu)
+endif

+// -----
+ifndef __cplusplus

#include <signal.h>
-// Platform definitions
#include <sys/time.h>

```

```

#include <cstdint>
#include <cstddef>

-// -----
-
#pragma GCC diagnostic push

#if defined(__clang__)
#pragma clang diagnostic ignored "-Wc++98-compat"
#endif

-// -----
-
namespace os
{
    namespace rtos
    {
        namespace port
        {
-            // -----

            namespace stack
            {
-                // Assume 64-bits core.
                using element_t = os_port_thread_stack_element_t;

-                // Align stack to 8 bytes.
                using allocation_element_t = os_port_thread_stack_allocation_element_t;

-                // Initial value for the minimum stack size in bytes.
                constexpr std::size_t min_size_bytes
                    = OS_INTEGER_RTOS_MIN_STACK_SIZE_BYTES;

-                // Initial value for the default stack size in bytes.
                constexpr std::size_t default_size_bytes
                    = OS_INTEGER_RTOS_DEFAULT_STACK_SIZE_BYTES;

-                constexpr element_t magic = 0xEFBEADDEEFBEADDE;
+                constexpr element_t magic = OS_INTEGER_RTOS_STACK_FILL_MAGIC;
            } /* namespace stack */

            namespace interrupts
            {
-                // True if signal blocked

```

```

        using state_t = os_port_irq_state_t;

        namespace state
        @@ -118,8 +116,19 @@ namespace os
            constexpr state_t init = false;
            } /* namespace state */

-        extern sigset_t clock_set;
+        /* The signals this port treats as interrupts: the per-CPU tick and
+         * the IPI. Blocking them IS masking interrupts on this CPU. */
+        extern sigset_t irq_set;
+
+        /* Per-CPU handler-mode flag. AArch64 has the same array for the same
+         * reason: there are no banked modes to ask. */
+        extern "C" volatile bool _in_isr[OS_NCPU];

+        /* The Arm CMSIS RTOS validator's host shim raises its "interrupt" with
+         * kill(SIGUSR1) and increments this around the handler. The port has
+         * no banked mode to ask either, so a non-zero value also means handler
+         * mode (see in_handler_mode()). Defined in src/rtos/os-core.cpp. */
+        extern "C" volatile uint32_t signal_nesting;
    } /* namespace interrupts */

    namespace scheduler
    @@ -133,23 +142,46 @@ namespace os
        constexpr state_t init = unlocked;
        } /* namespace state */

-        extern state_t lock_state;
+        extern volatile state_t lock_state[OS_NCPU];

+        /* Byte-for-byte the ARM ports' recursive kernel lock (lock word,
+         * owner CPU, nesting depth). The ARM timer leaf lock (smp_tlock_t,
+         * port_tmr_lock/unlock) is deliberately NOT carried here: it exists
+         * for a kernel timer back-end that takes it, and this port's timers
+         * are POSIX timers driven through host_cpu, so it would be dead. */
+        struct smp_klock_t
+        {
+            volatile uint32_t lock;
+            volatile uint32_t owner;
+            volatile uint32_t depth;
+        };

+        extern smp_klock_t _smp_klock;
+        extern volatile unsigned _port_ctx_pending[OS_NCPU];

```

```

} /* namespace scheduler */

namespace clock
{
-   constexpr unsigned int signal_number = SIGALRM;
-   } /* namespace clock */
+   /* Not SIGALRM. ITIMER_REAL and SIGALRM are process-wide; this port
+    * gives every CPU its own timer_create() timer, and a real-time
+    * signal because those queue rather than coalesce.
+    *
+    * Functions, not constants: glibc's SIGRTMIN expands to a call to
+    * __libc_current_sigrtmin(), so it is not a constant expression. */
+   inline int
+   signal_number (void)
+   {
+       return SIGRTMIN;
+   }

-   using thread_context_t = struct thread_context_s
-   {
-       // On POSIX, the context is saved on standard (although deprecated)
-       // ucontext_t structures. It requires _XOPEN_SOURCE=700L to compile.
-       os_impl_ucontext_t ucontext; //
-   };
+   inline int
+   ipi_signal_number (void)
+   {
+       return SIGRTMIN + 1;
+   }
+   } /* namespace clock */

-   // -----
+   using thread_context_t = os_port_thread_context_t;

    } /* namespace port */
} /* namespace rtos */
@@ -157,10 +189,6 @@ namespace os

#pragma GCC diagnostic pop

-// -----
-
#endif /* __cplusplus */

-// -----

```

```

-
#endif /* CMSIS_PLUS_RTOS_PORT_OS_DECLS_H_ */
diff --git a/include/cmsis-plus/rtos/port/os-inlines.h b/include/cmsis-plus/rtos/port/os-inlines.h
index 0ec7ab7..943da20 100644
--- a/include/cmsis-plus/rtos/port/os-inlines.h
+++ b/include/cmsis-plus/rtos/port/os-inlines.h
@@ -1,53 +1,44 @@
/*
+ * os-inlines.h - the hot half of the POSIX port: masking and the kernel lock.
+ *
+ * This file is part of the µOS++ project (https://micro-os-plus.github.io/).
+ * Copyright (c) 2016-2025 Liviu Ionescu. All rights reserved.
+ * Copyright (c) 2026 Dan. All rights reserved.
+ *
+ * Permission to use, copy, modify, and/or distribute this software
+ * for any purpose is hereby granted, under the terms of the MIT license.
+ *
- * If a copy of the license was not distributed with this file, it can
- * be obtained from https://opensource.org/licenses/mit/.
- */
-
-/*
- * This file is part of the CMSIS++ proposal, intended as a CMSIS
- * replacement for C++ applications.
- *
- * If contains the scheduler implementation that uses
- * functions from the POSIX API (macOS and GNU/Linux).
+ * Read this beside micro-os-plus-iii-aarch64/include/cmsis-plus/rtos/port/
+ * os-inlines.h. The two files have the same shape function for function; only
+ * the mechanism differs, and that is the whole point of the port seam:
+ *
- * This file is included in all src/os-*.cpp files.
+ * concern          AArch64                      here
+ * -----
+ * CPU id           MRS MPIDR_EL1, & 3            thread_local, set at CPU start
+ * IRQ mask         MSR DAIFSET/DAIFCLR, #2       pthread_sigmask(irq_set)
+ * IRQ state        MRS DAIF                      sigismember(old, tick)
+ * atomics          LDAXR/STLXR + DMB ISH         __atomic_* , seq_cst
+ * in-handler       _in_isr[]                     _in_isr[] (identical)
+ */

#ifndef CMSIS_PLUS_RTOS_PORT_OS_INLINE_H_
#define CMSIS_PLUS_RTOS_PORT_OS_INLINE_H_

-// -----

```

```

-
#if defined(OS_USE_OS_APP_CONFIG_H)
#include <cmsis-plus/os-app-config.h>
#endif

#include <cmsis-plus/rtos/os-c-decls.h>

-// -----
-
#ifdef __cplusplus

-// -----
-
-#include <stdlib.h>
-#include <string.h>
-#include <sys/utsname.h>
-#include <sys/time.h>
-
-// For Linux
+#include <pthread.h>
+#include <signal.h>
#include <unistd.h>
+#include <time.h>

#include <cmsis-plus/diag/trace.h>

-// -----
-
#pragma GCC diagnostic push
#pragma GCC diagnostic ignored "-Wunused-parameter"

@@ -55,9 +46,9 @@
#pragma clang diagnostic ignored "-Wc++98-compat"
#endif

-// -----
-
-extern "C" uint32_t signal_nesting;
+// Out of line on purpose; see _this_cpu below.
+extern "C" unsigned
+port_cpu_id (void);

namespace os
{
@@ -65,10 +56,44 @@ namespace os

```



```

{
    namespace port
    {
-        // -----
+        /*
+         * Which CPU is running this code.
+         *
+         * This is the one deliberate use of native thread-local storage in the
+         * whole port, and it is the one use that is correct by construction: a
+         * host thread IS a CPU, so storage private to a host thread is storage
+         * private to a CPU. It answers "where am I", never "what was I doing".
+         *
+         * Everything else must obey the opposite rule. A µOS++ thread migrates
+         * between CPUs, so `errno` and any `thread_local` it touches belong to
+         * whichever host thread happens to be running it at that instant. No
+         * port or application state may live there across a switch point --
+         * read errno only inside the critical section that made the call.
+         *
+         * That rule covers the ADDRESS of `_this_cpu` too, and the compiler does
+         * not know it. The thread pointer is constant for the life of a host
+         * thread, so clang computes `&_this_cpu` once per function and keeps it
+         * in a callee-saved register. A function that blocks -- semaphore:::
+         * wait() loops around reschedule() -- then resumes on another host
+         * thread and reads the CPU id of the one it left. It takes the kernel
+         * lock as that CPU, skips the acquire when that CPU already owns it,
+         * and two CPUs run inside the lock; the depth count strands it held and
+         * every CPU spins for ever (clang 16/17 -O2: smp_test3, smp-pipeline).
+         * GCC reloads the thread pointer at each access and never showed it.
+         *
+         * So the read is never inlined: each call of port_cpu_id() computes the
+         * address afresh, and the empty asm makes every call a side effect that
+         * can be neither merged with another nor hoisted, LTO included.
+         */
+        extern thread_local unsigned _this_cpu;

        namespace scheduler
        {
+            inline unsigned __attribute__((always_inline))
+            port_cpu_id_inline (void)
+            {
+                return ::port_cpu_id ();
+            }

            inline port::scheduler::state_t __attribute__((always_inline))
            lock (void)

```

```

@@ -85,27 +110,54 @@ namespace os
    inline bool __attribute__ ((always_inline))
    locked (void)
    {
-       return lock_state != state::unlocked;
+       return lock_state[port_cpu_id_inline ()] != state::unlocked;
    }

+   /* SMP: acquire the scheduler lock word.
+   *
+   * A spin, exactly as on ARM, and for the same reason: this lock is
+   * held for a handful of instructions and is taken from inside signal
+   * handlers, where a pthread_mutex would be a function call into code
+   * that is not async-signal-safe. The pause hint keeps a spinning CPU
+   * from starving the one that holds it when the host oversubscribes. */
    inline void __attribute__ ((always_inline))
    wait_for_interrupt (void)
+   _smp_klock_raw_acquire (void)
    {
-#if defined(OS_TRACE_RTOS_THREAD_CONTEXT)
-       trace::printf ("%s() \n", __func__);
+       while (__atomic_exchange_n (&_smp_klock.lock, 1u, __ATOMIC_ACQUIRE)
+           != 0u)
+       {
+#if defined(__x86_64__) || defined(__i386__)
+           __builtin_ia32_pause ();
+#elif defined(__aarch64__) || defined(__arm__)
+           __asm__ volatile("yield" ::: "memory");
+       #endif
-       pause ();
+       }
+   }

+   inline void __attribute__ ((always_inline))
+   _smp_klock_raw_release (void)
+   {
+       __atomic_store_n (&_smp_klock.lock, 0u, __ATOMIC_RELEASE);
+   }

+   inline unsigned __attribute__ ((always_inline))
+   port_smp_depth (void)
+   {
+       return _smp_klock.depth;
+   }
+

```

```

+     void
+     wait_for_interrupt (void);
+
+     } /* namespace scheduler */

namespace interrupts
{
-
+     inline bool __attribute__ ((always_inline))
+     in_handler_mode (void)
+     {
-         return (signal_nesting > 0);
+         const unsigned cpu = scheduler::port_cpu_id_inline ();
+         return ((cpu < OS_NCPU) && _in_isr[cpu]) || (signal_nesting != 0);
+     }

+     inline bool __attribute__ ((always_inline))
@@ -114,9 +166,71 @@ namespace os
+         return true;
+     }

-     } /* namespace interrupts */
+     /*
+      * Enter an IRQ critical section: mask this CPU's interrupts and take
+      * the kernel lock. Same two steps, same order, as AArch64's.
+      *
+      * pthread_sigmask, not sigprocmask. In a multithreaded process the
+      * behaviour of sigprocmask is unspecified (POSIX.1-2017, and glibc
+      * documents it as such), and upstream's port used it throughout
+      * because upstream had exactly one thread. The replacement is not a
+      * workaround but the better abstraction: a per-thread signal mask is
+      * per-CPU interrupt masking, which is what a critical section has
+      * always meant.
+      */
+     inline rtos::interrupts::state_t __attribute__ ((always_inline))
+     critical_section::enter (void)
+     {
+         sigset_t old;
+         ::pthread_sigmask (SIG_BLOCK, &irq_set, &old);

+         const unsigned cpu = scheduler::port_cpu_id_inline ();
+         if (scheduler::_smp_klock.owner != cpu)
+         {
+             scheduler::_smp_klock_raw_acquire ();
+             scheduler::_smp_klock.owner = cpu;

```

```

+     }
+     scheduler::_smp_klock.depth = scheduler::_smp_klock.depth + 1;
+
+     return ::sigismember (&old, clock::signal_number ()) != 0;
+ }
+
+ inline void __attribute__ ((always_inline))
+ critical_section::exit (rtos::interrupts::state_t state)
+ {
+     const unsigned cpu = scheduler::port_cpu_id_inline ();
+     if (scheduler::_smp_klock.owner == cpu
+         && scheduler::_smp_klock.depth > 0)
+     {
+         const uint32_t d = scheduler::_smp_klock.depth - 1;
+         scheduler::_smp_klock.depth = d;
+         if (d == 0)
+         {
+             scheduler::_smp_klock.owner = SMP_NO_OWNER;
+             scheduler::_smp_klock_raw_release ();
+         }
+     }
+
+     ::pthread_sigmask (state ? SIG_BLOCK : SIG_UNBLOCK, &irq_set,
+                        nullptr);
+ }
+
+ inline rtos::interrupts::state_t __attribute__ ((always_inline))
+ uncritical_section::enter (void)
+ {
+     sigset_t old;
+     ::pthread_sigmask (SIG_UNBLOCK, &irq_set, &old);
+     return ::sigismember (&old, clock::signal_number ()) != 0;
+ }
+
+ inline void __attribute__ ((always_inline))
+ uncritical_section::exit (rtos::interrupts::state_t state)
+ {
+     ::pthread_sigmask (state ? SIG_BLOCK : SIG_UNBLOCK, &irq_set,
+                        nullptr);
+ }
+
+ // =====
+ } /* namespace interrupts */
+
+ namespace this_thread

```

```

    {
@@ -124,11 +238,8 @@ namespace os
        prepare_suspend (void)
        {
        }

-
        } /* namespace this_thread */

-
        // =====
-
        inline constexpr bool __attribute__ ((always_inline))
        clock_highres::has_hardware_counter (void) noexcept
        {
@@ -138,7 +249,7 @@ namespace os
        inline uint64_t __attribute__ ((always_inline))
        clock_highres::hardware_counter (void) noexcept
        {
-        timespec tp;    // NB: no 'struct' – baseline gate has -Werror=redundant-tags
+        timespec tp;
            ::clock_gettime (CLOCK_MONOTONIC, &tp);
            return static_cast<uint64_t> (tp.tv_sec) * 1000000ULL
                + static_cast<uint64_t> (tp.tv_nsec) / 1000ULL;
@@ -150,10 +261,6 @@ namespace os

        #pragma GCC diagnostic pop

-// -----
-
        #endif /* __cplusplus */

-// -----
-
        #endif /* CMSIS_PLUS_RTOS_PORT_OS_INLINES_H_ */
diff --git a/include/exception_handler.hpp b/include/exception_handler.hpp
new file mode 100644
index 00000000..27423a1
--- /dev/null
+++ b/include/exception_handler.hpp
@@ -0,0 +1,29 @@
+/**
+ * @file exception_handler.hpp
+ * @brief Synchronous-fault reporting for the POSIX port.
+ *
+ * On the ARM ports a synchronous exception lands in a vector installed by
+ * startup.S and is reported by port_fatal_exception(). The host equivalent of

```

```

+ * a synchronous exception is a signal -- SIGSEGV, SIGBUS, SIGFPE, SIGILL --
+ * so exception::init() installs handlers for those and reports through the
+ * same function, with the same name, so a test that faults says so instead of
+ * dying silently with no output.
+ *
+ * This matters more here than on the silicon, not less: a host process that
+ * segfaults under a test runner produces an exit status and nothing else, and
+ * with several CPUs running there is no way afterwards to tell which one
+ * faulted or in which thread.
+ */
+#pragma once
+
+#include <cstdint>
+
+namespace exception {
+
+void init();
+
+} // namespace exception
+
+extern "C" void port_fatal_exception(std::uint64_t type, std::uint64_t esr,
+                                     std::uint64_t elr, std::uint64_t far,
+                                     std::uint64_t spsr);
diff --git a/include/host_cpu.hpp b/include/host_cpu.hpp
new file mode 100644
index 00000000..55861b6
--- /dev/null
+++ b/include/host_cpu.hpp
@@ -0,0 +1,129 @@
+/*
+ * host_cpu.hpp - the CPU model of the POSIX port: a host thread IS a CPU.
+ *
+ * This file is part of the µOS++ project (https://micro-os-plus.github.io/).
+ * Copyright (c) 2026 Dan. All rights reserved.
+ *
+ * Permission to use, copy, modify, and/or distribute this software
+ * for any purpose is hereby granted, under the terms of the MIT license.
+ *
+ * On the ARM ports the equivalent of this header is split between the silicon
+ * (the generic timer, the GIC/mailbox IPI, MPIDR) and the board (the spin
+ * table that releases the secondary cores). Here the silicon is the host
+ * kernel, so it is all one file, and it is port code rather than board code.
+ */
+
+#ifndef MICRO_OS_PLUS_III_POSIX_ARCH_HOST_CPU_HPP_

```

```

#define MICRO_OS_PLUS_III_POSIX_ARCH_HOST_CPU_HPP_
+
#include <cstdint>
+
#include <cmsis-plus/rtos/os.h>
#include <cmsis-plus/rtos/port/os-decls.h>
+
namespace host_cpu
+{
+ using element_t = os::rtos::port::stack::element_t;
+
+ /*
+  * "Which thread is running on this CPU."
+  *
+  * The kernel spells this two different ways: under the SMP scheduler
+  * `scheduler::current_thread_` is an array indexed by CPU, and without it
+  * a single pointer (os-sched.h, guarded on OS_USE_SMP_SCHEDULER, which
+  * uos_add_app() derives from NCPU > 1).
+  *
+  * One accessor keeps the difference in one place, so the rest of the port
+  * is written once and serves both. This is the same dual-branch arrangement
+  * the cortexm port uses to run its three STM32 boards at OS_NCPU=1 off the
+  * RP2350's SMP core.
+  */
+ inline os::rtos::thread* volatile&
+ current_thread (unsigned cpu)
+ {
+ #if defined(OS_USE_SMP_SCHEDULER)
+ return os::rtos::scheduler::current_thread_[cpu];
+ #else
+ (void)cpu;
+ return os::rtos::scheduler::current_thread_;
+ #endif
+ }
+
+ /* Install the tick and IPI signal handlers, process-wide. Called once,
+  * from port::scheduler::initialize(). */
+ void
+ install_handlers (void);
+
+ /* Give the calling CPU an alternate stack for fault reporting. Per host
+  * thread, because sigaltstack() is, and that is correct here: reporting a
+  * fault does not migrate, so it may use storage the CPU owns. */
+ void
+ install_fault_stack (void);

```

```

+
+ /* Arm the calling CPU's own periodic timer, at
+  * OS_INTEGER_SYSTICK_FREQUENCY_HZ. Every CPU has one; only CPU 0 advances
+  * the kernel clock with it. */
+ void
+ start_this_cpu_tick (void);
+
+ /* Create the OS_NCPU-1 secondary host threads. Each sets its own CPU index,
+  * arms its own tick and enters the scheduler, which is what a secondary
+  * core does after the spin table releases it. */
+ void
+ start_secondary_cpus (void);
+
+ /* Ask another CPU to re-pick. pthread_kill() of the IPI signal, which is
+  * this port's SGI / mailbox doorbell. */
+ void
+ send_ipi (unsigned cpu);
+
+ /* Build a never-run context's entry point. Wraps the kernel's thread body
+  * in the trampoline that discharges this CPU's deferred publish before the
+  * body runs -- a context that has never run still arrives on a CPU that
+  * has just left another thread behind. */
+ void
+ make_entry (os_impl_ucontext_t* ctx, void* func, void* args);
+
+ /* Leave `*addr = val` for whoever runs next on this CPU. See the long
+  * comment on port::scheduler::switch_stacks(). */
+ void
+ defer_publish (unsigned cpu, element_t** addr, element_t* val);
+
+ /* Discharge this CPU's deferred publish, if it has one. Called on every
+  * arrival into a context: after swapcontext(), and in the trampoline. */
+ void
+ publish_pending (void);
+
+ /* -----
+  * AddressSanitizer and the context switch.
+  *
+  * ASan keeps a shadow record of which stack is live so that it can tell a
+  * genuine overflow from an ordinary function return. swapcontext() moves
+  * the stack pointer to somewhere ASan has never seen, and without being
+  * told, ASan reports the first thing the resumed thread touches as a
+  * stack-buffer-overflow -- a false positive on every single switch, which
+  * makes the tool useless rather than merely noisy.
+  *
+  */

```



```

+ * The fix is ASan's own fiber interface, which exists for exactly this:
+ *
+ *   start_switch (&save, bottom, size)   before swapcontext / setcontext
+ *   finish_switch (save)                 first thing on arrival
+ *
+ * `save` is where ASan parks the outgoing fibre's "fake stack" (its
+ * use-after-return bookkeeping). It must survive until that context is
+ * resumed, so switch_stacks() keeps it in a local -- which lives on the
+ * outgoing thread's own stack and is therefore still there, and still
+ * correct, whenever and on whichever CPU that thread comes back.
+ *
+ * Both compile to nothing when ASan is off, so the calls stay unguarded at
+ * the call sites and the switch code reads the same either way.
+ * ----- */
+ void
+ asan_start_switch (void** save, const void* bottom, std::size_t size);
+
+ void
+ asan_finish_switch (void* save);
+
+} /* namespace host_cpu */
+
+#endif /* MICRO_OS_PLUS_III_POSIX_ARCH_HOST_CPU_HPP_ */
diff --git a/include/hw_result.hpp b/include/hw_result.hpp
new file mode 100644
index 00000000..2102b90
--- /dev/null
+++ b/include/hw_result.hpp
@@ -0,0 +1,53 @@
+/**
+ * @file hw_result.hpp
+ * @brief Bounded "done" helper, POSIX host.
+ *
+ * The sibling of micro-os-plus-iii-aarch64/include/hw_result.hpp. A test
+ * prints its RESULT line and then calls ok()/fail().
+ *
+ * On the silicon ports these compile to a semihosted SYS_EXIT under SEMIHOST
+ * and to nothing otherwise, so a plain image keeps its idle-forever
+ * behaviour. Here there is no "otherwise": the host always has an exit
+ * status, and a test process that idled for ever would have to be killed by
+ * the runner and could not report anything. So they always end the run, and
+ * the verdict reaches the shell as the exit code.
+ *
+ * _exit(), not exit(). Other CPUs are still running threads; exit() would run
+ * static destructors underneath them and turn a clean PASS into a crash in

```

```

+ * the teardown.
+ *
+ * But _exit() does not flush stdio either, so these flush first. Most tests
+ * here print through uart::uart1, which is write(2) and therefore already on
+ * its way out; a test carried from upstream's own suite prints with printf(),
+ * and to a pipe -- which is how the runner captures it -- stdout is fully
+ * buffered. Without the flush its entire output, verdict included, is
+ * discarded at the exit. (Found exactly that way: mutex-stress ran, passed and
+ * printed nothing.)
+ *
+ * Pattern in a test:
+ *
+ *     uart1 << "\nRESULT: " << (ok ? "PASS" : "FAIL") << "\n";
+ *     if (ok) hw_result::ok (); else hw_result::fail ();
+ *     for (;;) sysclock.sleep_for (1000);    // never reached
+ */
+
+#pragma once
+
+#include <cstdio>
+#include <unistd.h>
+
+namespace hw_result
+{
+    [[noreturn]] inline void
+    ok () noexcept
+    {
+        std::fflush (nullptr);
+        ::_exit (0);
+    }
+
+    [[noreturn]] inline void
+    fail () noexcept
+    {
+        std::fflush (nullptr);
+        ::_exit (1);
+    }
+} // namespace hw_result
diff --git a/src/board-contract.cpp b/src/board-contract.cpp
new file mode 100644
index 0000000..40805d4
--- /dev/null
+++ b/src/board-contract.cpp
@@ -0,0 +1,43 @@
+/*
+ * board-contract.cpp - no code. It fails the build the moment a board has

```

```

+ * not stated something the shared sources need.
+ *
+ * This file is part of the µOS++ project (https://micro-os-plus.github.io/).
+ * Copyright (c) 2026 Dan. All rights reserved.
+ *
+ * Permission to use, copy, modify, and/or distribute this software
+ * for any purpose is hereby granted, under the terms of the MIT license.
+ *
+ * The same file exists in the three cross-compiled projects and exists here
+ * for the same reason: a board that leaves something unset must produce a
+ * compile error naming the board and the thing, not a silent inheritance of
+ * another board's answer -- or, worse, a link that succeeds and a test that
+ * proves nothing.
+ */
+
+#if defined(__APPLE__) || defined(__linux__)
+
+#include <cmsis-plus/rtos/port/os-decls.h>
+
+#if !defined(OS_NCPU)
+#error "The board does not set OS_NCPU."
+#endif
+
+#if OS_NCPU < 1
+#error "OS_NCPU must be at least 1."
+#endif
+
+#if !defined(PORT_GREETING)
+#error "The board does not define PORT_GREETING; the test banners print it."
+#endif
+
+// A host CPU is a host thread, so OS_NCPU above the number the machine can
+// actually run in parallel is not an error -- it oversubscribes, which is a
+// legitimate and useful way to shake out races. It IS worth saying out loud,
+// because a suite that suddenly takes ten times as long has usually done this
+// by accident.
+#if OS_NCPU > 16
+#warning "OS_NCPU above 16 oversubscribes almost any host; expect the tick to drift."
+#endif
+
+#endif /* defined(__APPLE__) || defined(__linux__) */
diff --git a/src/exception_handler.cpp b/src/exception_handler.cpp
new file mode 100644
index 00000000..2a09db9
--- /dev/null

```

```

+++ b/src/exception_handler.cpp
@@ -0,0 +1,224 @@
+/*
+ * exception_handler.cpp - synchronous faults, reported rather than silent.
+ *
+ * This file is part of the µOS++ project (https://micro-os-plus.github.io/).
+ * Copyright (c) 2026 Dan. All rights reserved.
+ *
+ * Permission to use, copy, modify, and/or distribute this software
+ * for any purpose is hereby granted, under the terms of the MIT license.
+ */
+
+#if defined(__APPLE__) || defined(__linux__)
+
+#include <csignal>
+#include <cstring>
+#include <initializer_list>
+#include <dlfcn.h>
+#include <execinfo.h>
+#include <ucontext.h>
+#include <unistd.h>
+
+#include <cmsis-plus/rtos/os.h>
+#include <cmsis-plus/rtos/port/os-decls.h>
+#include <cmsis-plus/rtos/port/os-inlines.h>
+#include <exception_handler.hpp>
+#include <host_cpu.hpp>
+
+namespace
+{
+    // Everything below runs in a signal handler after the process is already
+    // broken, so it uses write(2) and nothing else: no printf, no malloc, no
+    // locks. A reporting path that can itself deadlock reports nothing.
+    void
+    emit (const char* s)
+    {
+        std::size_t n = 0;
+        while (s[n] != '\0')
+        {
+            ++n;
+        }
+        ssize_t r = ::write (2, s, n);
+        (void)r;
+    }
+}

```

```

+ void
+ emit_hex (std::uint64_t v)
+ {
+     static const char digits[] = "0123456789ABCDEF";
+     char out[19] = "0x";
+     int n = 2;
+     for (int i = 15; i >= 0; --i)
+     {
+         out[n++] = digits[(v >> (i * 4)) & 0xF];
+     }
+     out[n] = '\0';
+     emit (out);
+ }
+
+ /*
+  * The program counter of the faulting instruction, from the machine context
+  * the kernel handed the handler. The ARM ports report ELR for the same
+  * reason: on an SMP fault the faulting address alone rarely identifies the
+  * bug, and `addr2line -e <image> <pc>` names the line outright.
+  */
+
+ /*
+  * The load bias of this image, captured in exception::init() -- i.e. in a
+  * sane context, because dladdr() is not async-signal-safe. Reported
+  * alongside the raw pc so that a position-independent executable (which is
+  * the default here) can still be looked up with the static addresses in the
+  * ELF file: `addr2line -e <image> <pc-bias>`.
+  */
+ std::uint64_t g_load_bias = 0;
+
+ std::uint64_t
+ fault_pc (void* ucontext)
+ {
+     if (ucontext == nullptr)
+     {
+         return 0;
+     }
+     ucontext_t* uc = static_cast<ucontext_t*> (ucontext);
+ #if defined(__x86_64__)
+     return static_cast<std::uint64_t> (uc->uc_mcontext.gregs[REG_RIP]);
+ #elif defined(__i386__)
+     return static_cast<std::uint64_t> (uc->uc_mcontext.gregs[REG_EIP]);
+ #elif defined(__aarch64__)
+     return static_cast<std::uint64_t> (uc->uc_mcontext.pc);
+ #elif defined(__arm__)
+     return static_cast<std::uint64_t> (uc->uc_mcontext.arm_pc);

```

```

+ #else
+     (void)uc;
+     return 0;
+ #endif
+ }
+
+ void
+ fault_handler (int sig, siginfo_t* info, void* ucontext)
+ {
+     port_fatal_exception (static_cast<std::uint64_t> (sig), 0,
+                           fault_pc (ucontext),
+                           reinterpret_cast<std::uint64_t> (
+                               info != nullptr ? info->si_addr : nullptr),
+                           0);
+ }
+ } /* anonymous namespace */
+
+ extern "C" void
+ port_fatal_exception (std::uint64_t type, std::uint64_t esr,
+                      std::uint64_t elr, std::uint64_t far,
+                      std::uint64_t spsr)
+ {
+     (void)esr;
+     (void)spsr;
+
+     emit ("\n!!! FATAL EXCEPTION on CPU ");
+     {
+         const unsigned cpu = port_cpu_id ();
+         char c = static_cast<char> ('0' + (cpu % 10));
+         ssize_t r = ::write (2, &c, 1);
+         (void)r;
+     }
+     emit (" -- signal ");
+     emit (::strsignal (static_cast<int> (type)));
+     emit (" at ");
+     emit_hex (far);
+     emit (" , pc ");
+     emit_hex (elr);
+     emit (" (static ");
+     emit_hex (elr - g_load_bias);
+     emit (" )\n");
+
+     /*
+      * The scheduler state, which is the whole point of reporting at all: on a
+      * machine with several CPUs, "it crashed" says nothing without knowing

```

```

+  * which thread each CPU was running and who held the kernel lock. The ARM
+  * ports print the same two facts from their claim tripwire.
+  */
+  for (unsigned c = 0; c < OS_NCPU; ++c)
+  {
+      emit ("  cpu");
+      {
+          char d = static_cast<char> ('0' + (c % 10));
+          ssize_t r = ::write (2, &d, 1);
+          (void)r;
+      }
+      emit (": ");
+      os::rtos::thread* th = host_cpu::current_thread (c);
+      emit (th == nullptr ? "(null)"
+          : (th->name () != nullptr ? th->name () : "(anon)"));
+      emit ("\n");
+  }
+  emit ("  klock: owner=");
+  emit_hex (os::rtos::port::scheduler::_smp_klock.owner);
+  emit (" depth=");
+  emit_hex (os::rtos::port::scheduler::_smp_klock.depth);
+  emit (" lock=");
+  emit_hex (os::rtos::port::scheduler::_smp_klock.lock);
+  emit ("\n");
+
+  /*
+   * The call chain. Not async-signal-safe in the strict sense (backtrace()
+   * may take the loader lock on its first call), but the process is already
+   * dead and a chain of return addresses is what turns "a null pointer was
+   * dereferenced" into "this line dereferenced it".
+   */
+  {
+      void* frames[24];
+      int n = ::backtrace (frames, 24);
+      emit ("  backtrace (static):\n");
+      for (int i = 0; i < n; ++i)
+      {
+          emit ("    ");
+          emit_hex (reinterpret_cast<std::uint64_t> (frames[i]) - g_load_bias);
+          emit ("\n");
+      }
+  }
+
+  // Re-raise with the handler removed, so the shell sees the real cause and a
+  // core file is still produced.

```

```

+ ::signal (static_cast<int> (type), SIG_DFL);
+ ::raise (static_cast<int> (type));
+ ::_exit (128 + static_cast<int> (type));
+}
+
+namespace exception
+{
+ void
+ init ()
+ {
+ {
+     DL_info info;
+     if (::dladdr (reinterpret_cast<void*> (&port_fatal_exception), &info)
+         != 0)
+     {
+         g_load_bias = reinterpret_cast<std::uint64_t> (info.dli_fbase);
+     }
+ }
+
+ struct sigaction sa; // `sigaction` is also a function: the tag is required
+ std::memset (&sa, 0, sizeof (sa));
+ sa.sa_flags = SA_SIGINFO | SA_ONSTACK;
+ ::sigemptyset (&sa.sa_mask);
+ sa.sa_sigaction = fault_handler;
+
+ // SA_ONSTACK here, and NOT on the tick -- the two are opposite cases and
+ // the difference matters.
+ //
+ // The tick must run on the thread's own stack, so its frame migrates with
+ // the thread when another CPU resumes it (host_cpu.cpp). A fault handler
+ // never migrates: it reports and the process ends. And it must not need
+ // the faulting stack, because a blown or wild stack is precisely one of
+ // the faults worth reporting -- without an alternate stack that case
+ // cannot be reported at all, which is how the first SMP fault found here
+ // came out as a silent exit 139.
+ //
+ // The stack itself is per CPU and installed by host_cpu when the CPU
+ // starts, because sigaltstack() is per host thread.
+ for (int sig : { SIGSEGV, SIGBUS, SIGFPE, SIGILL })
+ {
+     ::sigaction (sig, &sa, nullptr);
+ }
+ }
+} /* namespace exception */
+

```



```

+#endif /* defined(__APPLE__) || defined(__linux__) */
diff --git a/src/free-store.cpp b/src/free-store.cpp
new file mode 100644
index 00000000..acc7e95
--- /dev/null
+++ b/src/free-store.cpp
@@ -0,0 +1,133 @@
+/*
+ * free-store.cpp - the application free store on the POSIX host.
+ *
+ * This file is part of the µOS++ project (https://micro-os-plus.github.io/).
+ * Copyright (c) 2016-2025 Liviu Ionescu. All rights reserved.
+ * Copyright (c) 2026 Dan. All rights reserved.
+ *
+ * Permission to use, copy, modify, and/or distribute this software
+ * for any purpose is hereby granted, under the terms of the MIT license.
+ *
+ * -----
+ * Why this file exists at all, and why it does NOT just call malloc().
+ *
+ * The kernel's own src/startup/initialise-free-store.cpp is guarded by
+ * `#if defined(__ARM_EABI__)` from top to bottom, so on a host it compiles to
+ * nothing and os_startup_initialize_free_store() is undefined -- which is
+ * exactly what the carried tests call. Relaxing that guard would mean editing
+ * the kernel, which is kept merge-clean against upstream; so the port
+ * supplies the hook, which is what a port is for.
+ *
+ * Upstream's posix-arch had no such hook and let the system malloc() serve,
+ * and its NOTES.md says so. That was correct for upstream, which had ONE host
+ * thread. It is not correct here, and the difference is the whole subject of
+ * this port:
+ *
+ * A µOS++ thread may be preempted inside an allocation and resumed on a
+ * DIFFERENT CPU -- that is, a different host thread. glibc's malloc is
+ * thread-safe by taking an arena lock, and that lock would then be released
+ * by a host thread that did not take it. Nothing in the C library promises
+ * that works.
+ *
+ * Using µOS++'s own memory resource removes the question rather than betting
+ * on it: the allocator is the kernel's, its mutual exclusion is the kernel's
+ * scheduler lock, and the scheduler lock already migrates correctly because
+ * making it do so is what the rest of this port is about. It also makes the
+ * host behave like the three silicon ports, which is the point of testing on
+ * it -- a first_fit_top over a fixed block is what every board does, so an
+ * allocation pattern that exhausts the heap here exhausts it there too,

```

```

+ * instead of being quietly absorbed by a host with gigabytes to spare.
+ * -----
+ */
+
+#if defined(__APPLE__) || defined(__linux__)
+
+#include <stddef>
+#include <new>
+
+#include <cmsis-plus/rtos/os.h>
+#include <cmsis-plus/rtos/os-hooks.h>
+#include <cmsis-plus/memory/first-fit-top.h>
+#include <cmsis-plus/estd/memory_resource>
+
+using namespace os;
+
+#if defined(OS_TYPE_APPLICATION_MEMORY_RESOURCE)
+using application_memory_resource = OS_TYPE_APPLICATION_MEMORY_RESOURCE;
+#else
+using application_memory_resource = os::memory::first_fit_top;
+#endif
+
+namespace
+{
+ // Same storage trick as the kernel's ARM version: the resource itself must
+ // outlive everything and must not need the free store to exist first.
+ alignas (application_memory_resource) char
+     application_free_store[sizeof (application_memory_resource)];
+} // namespace
+
+/*
+ * The out-of-memory hooks, for the same reason as the hook above: the
+ * kernel's definitions live in the __ARM_EABI__-guarded file. Weak, so an
+ * application that wants to reset the machine, dump the heap or coalesce
+ * before giving up replaces them by defining its own -- which is exactly the
+ * contract they have on the silicon ports.
+ */
+void __attribute__ ((weak))
+os_rtos_application_out_of_memory_hook (void)
+{
+     estd::__throw_bad_alloc ();
+}
+
+#if defined(OS_INTEGER_RTOS_DYNAMIC_MEMORY_SIZE_BYTES)
+

```

```

+void __attribute__ ((weak))
+os_rtos_system_out_of_memory_hook (void)
+{
+  estd::__throw_bad_alloc ();
+}
+
+
+#endif /* defined(OS_INTEGER_RTOS_DYNAMIC_MEMORY_SIZE_BYTES) */
+
+/*
+ * Weak no-ops, so an application that brings up no hardware still links.
+ * Every carried test defines both, and its definitions win.
+ */
+void __attribute__ ((weak))
+os_startup_initialize_hardware_early (void)
+{
+}
+
+void __attribute__ ((weak))
+os_startup_initialize_hardware (void)
+{
+}
+
+void __attribute__ ((weak))
+os_startup_initialize_free_store (void* heap_address,
+                                std::size_t heap_size_bytes)
+{
+#if !defined(OS_EXCLUDE_DYNAMIC_MEMORY_ALLOCATIONS)
+
+  new (&application_free_store)
+    application_memory_resource{ "app-heap", heap_address,
+                                heap_size_bytes };
+
+  reinterpret_cast<rtos::memory::memory_resource*> (&application_free_store)
+    ->out_of_memory_handler (os_rtos_application_out_of_memory_hook);
+
+  estd::pmr::set_default_resource (
+    reinterpret_cast<estd::pmr::memory_resource*> (&application_free_store));
+
+  // No sbrk() adjustment. On a bare-metal target the kernel's version pushes
+  // sbrk past the free store so newlib's malloc cannot collide with it; here
+  // the block is ordinary static storage the host already owns, so there is
+  // nothing to push it past.
+
+
+#else
+  (void)heap_address;

```

```

+ (void)heap_size_bytes;
+#endif /* !defined(OS_EXCLUDE_DYNAMIC_MEMORY_ALLOCATIONS) */
+}
+
+#endif /* defined(__APPLE__) || defined(__linux__) */
diff --git a/src/host_cpu.cpp b/src/host_cpu.cpp
new file mode 100644
index 00000000..87bd73f
--- /dev/null
+++ b/src/host_cpu.cpp
@@ -0,0 +1,530 @@
+/*
+ * host_cpu.cpp - host threads as CPUs: bring-up, per-CPU tick, IPI.
+ *
+ * This file is part of the µOS++ project (https://micro-os-plus.github.io/).
+ * Copyright (c) 2026 Dan. All rights reserved.
+ *
+ * Permission to use, copy, modify, and/or distribute this software
+ * for any purpose is hereby granted, under the terms of the MIT license.
+ */
+
+#if defined(__APPLE__) || defined(__linux__)
+
+#include <cerrno>
+#include <cstdio>
+#include <cstdlib>
+#include <cstring>
+
+#include <pthread.h>
+#include <signal.h>
+#include <sys/syscall.h>
+#include <time.h>
+#include <unistd.h>
+
+#include <cmsis-plus/rtos/os.h>
+#include <cmsis-plus/rtos/port/os-inlines.h>
+
+#include <host_cpu.hpp>
+
+/* Defined by the board (test/boards/<board>/src/smp.cpp), with the same
+ * meaning as on the ARM boards: 0 = not started, 3 = scheduler entered. The
+ * tests wait on it before they start timing anything. A board that does not
+ * define it fails to link, which is the point. */
+extern "C" volatile uint32_t g_core_stage[OS_NCPU];
+

```

```

+// Port-side weak default so a build that does not link the SMP test board still
+// resolves g_core_stage[]. The SMP test board (boards/native/src/smp.cpp)
+// provides a strong definition that overrides this one when present.
+extern "C"
+{
+  + __attribute__((weak)) volatile uint32_t g_core_stage[OS_NCPU] = {};
+}
+
+namespace
+{
+  + using namespace os::rtos::port;
+
+  + /* Every CPU's host thread, so one can be signalled by another. */
+  + pthread_t g_cpu_thread[OS_NCPU];
+  + volatile bool g_cpu_up[OS_NCPU];
+
+  + /* The deferred-publish slot, one per CPU. */
+  + struct publish_slot
+  + {
+  +   host_cpu::element_t** addr;
+  +   host_cpu::element_t* val;
+  + };
+
+  + publish_slot g_publish[OS_NCPU];
+
+  + /* What a never-run context must call. Recovered by the trampoline from the
+  +   * two pointers makecontext() carries for it. */
+  + using body_t = void (*) (void*);
+
+  + [[noreturn]] void
+  + fatal (const char* what)
+  + {
+  +   std::fprintf (stderr, "\n!!! posix-arch: %s: %s\n", what,
+  +               std::strerror (errno));
+  +   std::abort ();
+  + }
+
+  + /*
+  +   * The interrupt path.
+  +   *
+  +   * Deliberately the same shape as boards/rpi-zero-2w/src/rtos/port_isr.cpp:
+  +   * flag handler mode, do the work, mark a switch pending, clear the flag,
+  +   * and take the switch on the way out.
+  +   *
+  +   * Two things are deliberate and neither is an oversight:

```

```

+  *
+  * - NO SA_ONSTACK. The handler runs on the µOS++ thread's own stack, so
+  *   the whole signal frame is part of the context that swapcontext() saves
+  *   and therefore migrates with the thread when another CPU resumes it. An
+  *   alternate stack belongs to the host thread and would be left behind.
+  *   This is why OS_INTEGER_RTOS_MIN_STACK_SIZE_BYTES is 32 KiB here.
+  *
+  * - The switch happens INSIDE the handler, not after sigreturn. Returning
+  *   first would mean the switch only ever happened at a cooperative point,
+  *   which is exactly the limitation upstream's NOTES.md records ("the
+  *   scheduler runs in cooperative mode only"). swapcontext() from a
+  *   handler is well defined here because the handler frame lives on the
+  *   thread's stack: resuming the context resumes the handler, which then
+  *   returns through sigreturn normally.
+  */
+ void
+ irq_epilogue (unsigned cpu)
+ {
+   if (scheduler::_port_ctx_pending[cpu] == 0)
+   {
+     return;
+   }
+
+   if (os::rtos::scheduler::locked ())
+   {
+     // Still masked by the kernel; the next tick will find it pending.
+     return;
+   }
+
+   if (scheduler::_smp_klock.owner == cpu && scheduler::_smp_klock.depth > 0)
+   {
+     return;
+   }
+
+   scheduler::_port_ctx_pending[cpu] = 0;
+   scheduler::switch_stacks (nullptr);
+ }
+
+ /* Put `errno` back, out of line.
+  *
+  * Out of line on purpose. `errno` is `*__errno_location()`, which is native
+  * TLS, and the caller read it BEFORE a swapcontext() that may have resumed
+  * this thread on a different CPU -- that is, on a different host thread,
+  * with a different errno. A compiler that cached the address across the
+  * switch would write the value into the host thread the thread LEFT. This

```

```

+  * port bans native TLS across a switch point for exactly that reason, and a
+  * noline call is how the ban is honoured here: the address is resolved
+  * inside this function, after the switch, on whichever CPU is running now.
+  */
+  [[gnu::noline]] void
+  restore_errno (int value)
+  {
+      errno = value;
+  }
+
+  void
+  tick_handler (int, siginfo_t*, void*)
+  {
+      /* errno belongs to the THREAD, and on this port the thread is the uOS++
+       * one, not the host thread it is borrowing.
+       *
+       * This handler does not, in general, return to where it was raised: its
+       * epilogue switches contexts, so control leaves on one thread's stack and
+       * comes back -- possibly on another CPU, possibly much later -- when THIS
+       * thread is resumed. Everything the handler and every thread scheduled in
+       * between did to errno is therefore visible to the interrupted code
+       * unless it is put back. On silicon there is no errno to spoil; here
+       * there is.
+       *
+       * `saved` is a local, so like the ASan fibre save it rides the
+       * interrupted thread's own stack and is still correct wherever that
+       * thread comes back. */
+      const int saved = errno;
+
+      const unsigned cpu = port_cpu_id ();
+      if (cpu >= OS_NCPU)
+      {
+          restore_errno (saved);
+          return;
+      }
+
+      interrupts::_in_isr[cpu] = true;
+
+#if defined(__linux__)
+      // Only CPU 0 advances the kernel clock. Every CPU tickles itself into
+      // re-picking. This is the BCM2837 arrangement exactly: four cores take
+      // the 1 ms PPI, one calls os_systick_handler().
+      if (cpu == 0)
+      {
+          os_systick_handler ();

```

```

+     }
+
+ #else
+     // On non-Linux where per-thread timers are not available (SIGEV_SIGNAL
+     // process-directed delivery), guarantee os_systick_handler() is called
+     // regardless of which host thread received the tick signal so the clock
+     // does not stall.
+     os_systick_handler ();
+ #endif
+
+
+     scheduler::_port_ctx_pending[cpu] = 1;
+
+
+     interrupts::_in_isr[cpu] = false;
+
+
+     irq_epilogue (cpu);
+
+
+     // Resumed. See the comment at the top of this function.
+     restore_errno (saved);
+ }
+
+
+ void
+ ipi_handler (int, siginfo_t*, void*)
+ {
+     // Same reasoning as tick_handler(); this one switches contexts too.
+     const int saved = errno;
+
+
+     const unsigned cpu = port_cpu_id ();
+     if (cpu >= OS_NCPU)
+     {
+         restore_errno (saved);
+         return;
+     }
+
+
+     interrupts::_in_isr[cpu] = true;
+     scheduler::_port_ctx_pending[cpu] = 1;
+     interrupts::_in_isr[cpu] = false;
+
+
+     irq_epilogue (cpu);
+
+
+     restore_errno (saved);
+ }
+
+
+ /* One per CPU, never shared: a CPU reporting a fault is not going anywhere
+  * else while it does so. SIGSTKSZ is a minimum, and the reporting path
+  * formats numbers, so it gets room. */
+ char g_fault_stack[OS_NCPU][64 * 1024];

```



```

+
+ void
+ arm_tick (void)
+ {
+     /* struct */ sigevent sev;
+     std::memset (&sev, 0, sizeof (sev));
+
+ #if defined(__linux__)
+     // SIGEV_THREAD_ID, not ITIMER_REAL.
+     //
+     // ITIMER_REAL delivers SIGALRM to an ARBITRARY thread of the process, so
+     // with N CPUs the tick would land on whichever host thread the kernel
+     // happened to choose -- a tick is not a per-CPU event any more, and the
+     // preemption of CPU 2 could be charged to CPU 0. A per-thread timer is
+     // what a per-core timer actually is.
+     sev.sigev_notify = SIGEV_THREAD_ID;
+     sev._sigev_un._tid = static_cast<int> (::syscall (SYS_gettid));
+ #else
+     // On non-Linux (e.g. Darwin/BSD), per-thread timer creation is not available.
+     // Arm the process timer only on CPU 0 to prevent N timers firing concurrently.
+     if (port_cpu_id () != 0)
+     {
+         return;
+     }
+     sev.sigev_notify = SIGEV_SIGNAL;
+ #endif
+     sev.sigev_signo = clock::signal_number ();
+
+     timer_t timer;
+     if (::timer_create (CLOCK_MONOTONIC, &sev, &timer) != 0)
+     {
+         fatal ("timer_create");
+     }
+
+     const long period_ns = 1000000000L / OS_INTEGER_SYSTICK_FREQUENCY_HZ;
+
+     /* struct */ itimerspec its;
+     its.it_value.tv_sec = 0;
+     its.it_value.tv_nsec = period_ns;
+     its.it_interval.tv_sec = 0;
+     its.it_interval.tv_nsec = period_ns;
+
+     if (::timer_settime (timer, 0, &its, nullptr) != 0)
+     {
+         fatal ("timer_settime");
+     }

```

```

+     }
+ }
+
+ void*
+ secondary_cpu_body (void* arg)
+ {
+     const unsigned cpu
+         = static_cast<unsigned> (reinterpret_cast<uintptr_t> (arg));
+
+     _this_cpu = cpu;
+
+     host_cpu::install_fault_stack ();
+
+     ::pthread_sigmask (SIG_BLOCK, &interrupts::irq_set, nullptr);
+
+     // A context to switch away from, as port::scheduler::start() builds for
+     // CPU 0. A secondary core does exactly this after the spin table
+     // releases it: give itself somewhere to save, then reschedule.
+     static os_thread_t fake_thread[OS_NCPU];
+     std::memset (&fake_thread[cpu], 0, sizeof (os_thread_t));
+     fake_thread[cpu].name = "fake_thread_sec";
+     host_cpu::current_thread (cpu)
+         = reinterpret_cast<os::rtos::thread*> (&fake_thread[cpu]);
+
+     g_cpu_up[cpu] = true;
+
+     host_cpu::start_this_cpu_tick ();
+
+     g_core_stage[cpu] = 3;
+
+     scheduler::reschedule ();
+
+     // Only if this CPU never found anything to run.
+     ::pthread_sigmask (SIG_UNBLOCK, &interrupts::irq_set, nullptr);
+     for (;;)
+     {
+         ::pause ();
+     }
+
+     return nullptr;
+ }
+
+ [[noreturn]] void
+ trampoline (void* func, void* args)
+ {

```

```

+ // libucontext contexts carry no uc_sigmask, so guarantee the masked
+ // start here, before the deferred publish (unmasked again below).
+ ::pthread_sigmask (SIG_BLOCK, &interrupts::irq_set, nullptr);
+ // Arrival. The switch that brought us here started on another stack; this
+ // tells ASan the move is complete and that THIS stack is the live one.
+ // nullptr because a context that has never run has no outgoing fibre
+ // bookkeeping of its own to restore.
+ host_cpu::asan_finish_switch (nullptr);
+
+ // A context that has never run still arrives on a CPU that has just left
+ // another thread behind, so the publish is owed here too -- and it is
+ // owed BEFORE this thread can be interrupted, or the tick's own switch
+ // would overwrite the slot and strand the outgoing thread. context::create
+ // starts every new context with this CPU's interrupts masked for exactly
+ // this window; it ends here.
+ host_cpu::publish_pending ();
+
+ ::pthread_sigmask (SIG_UNBLOCK, &interrupts::irq_set, nullptr);
+
+ reinterpret_cast<body_t> (func) (args);
+
+ // The kernel's thread body does not return.
+ std::fprintf (stderr, "\n!!! posix-arch: thread body returned\n");
+ std::abort ();
+ }
+
+} /* anonymous namespace */
+
+namespace host_cpu
+{
+ void
+ install_handlers (void)
+ {
+   struct sigaction sa; // `sigaction` is also a function: the tag is required
+   std::memset (&sa, 0, sizeof (sa));
+
+   // SA_RESTART so a preempted read()/write() resumes rather than failing
+   // with EINTR; the tests print through write(), a thousand times a second.
+   // No SA_ONSTACK -- see irq_epilogue() above.
+   sa.sa_flags = SA_SIGINFO | SA_RESTART;
+   ::sigemptyset (&sa.sa_mask);
+   ::sigaddset (&sa.sa_mask, clock::signal_number ());
+   ::sigaddset (&sa.sa_mask, clock::ipi_signal_number ());
+
+   sa.sa_sigaction = tick_handler;

```

```

+   if (::sigaction (clock::signal_number (), &sa, nullptr) != 0)
+   {
+       fatal ("sigaction(tick)");
+   }
+
+   sa.sa_sigaction = ipi_handler;
+   if (::sigaction (clock::ipi_signal_number (), &sa, nullptr) != 0)
+   {
+       fatal ("sigaction(ipi)");
+   }
+
+   g_cpu_thread[0] = ::pthread_self ();
+   g_cpu_up[0] = true;
+   g_core_stage[0] = 3;
+
+   install_fault_stack ();
+ }
+
+ void
+ install_fault_stack (void)
+ {
+     const unsigned cpu = port_cpu_id ();
+     if (cpu >= OS_NCPU)
+     {
+         return;
+     }
+
+     stack_t ss;
+     std::memset (&ss, 0, sizeof (ss));
+     ss.ss_sp = g_fault_stack[cpu];
+     ss.ss_size = sizeof (g_fault_stack[cpu]);
+     ss.ss_flags = 0;
+     ::sigaltstack (&ss, nullptr);
+ }
+
+ void
+ start_this_cpu_tick (void)
+ {
+     arm_tick ();
+ }
+
+ void
+ start_secondary_cpus (void)
+ {
+     for (unsigned cpu = 1; cpu < OS_NCPU; ++cpu)

```

```

+     {
+         pthread_attr_t attr;
+         ::pthread_attr_init (&attr);
+
+         if (::pthread_create (
+             &g_cpu_thread[cpu], &attr, secondary_cpu_body,
+             reinterpret_cast<void*> (static_cast<uintptr_t> (cpu)))
+             != 0)
+         {
+             fatal ("pthread_create(cpu)");
+         }
+
+         ::pthread_attr_destroy (&attr);
+     }
+
+     // Wait for every CPU to have an index and a context to save into, the
+     // way the ARM ports wait on the per-core stage flags before proceeding.
+     for (unsigned cpu = 1; cpu < OS_NCPU; ++cpu)
+     {
+         while (!g_cpu_up[cpu])
+         {
+             ::usleep (100);
+         }
+     }
+ }
+
+ void
+ send_ipi (unsigned cpu)
+ {
+     if (cpu < OS_NCPU && g_cpu_up[cpu])
+     {
+         ::pthread_kill (g_cpu_thread[cpu], clock::ipi_signal_number ());
+     }
+ }
+
+ void
+ make_entry (os_impl_ucontext_t* ctx, void* func, void* args)
+ {
+     #pragma GCC diagnostic push
+     #if defined(__clang__)
+     #pragma clang diagnostic ignored "-Wc++98-compat-pedantic"
+     #endif
+
+     // Two pointer arguments. glibc carries them whole -- verified on this
+     // host before it was relied on, because the interface is declared with
+     // int arguments and a truncated `args` would be a pointer to nothing.

```

```

+   os_impl_makecontext (
+       ctx, reinterpret_cast<void (*) (void)> (trampoline), 2, func, args);
+ #pragma GCC diagnostic pop
+ }
+
+ void
+ defer_publish (unsigned cpu, element_t** addr, element_t* val)
+ {
+     g_publish[cpu].addr = addr;
+     g_publish[cpu].val = val;
+ }
+
+ /* See the long comment in host_cpu.hpp. Declared by hand rather than through
+  * <sanitizer/asan_interface.h>, so that a toolchain without the header still
+  * builds the port -- the symbols come from the ASan runtime, which is only
+  * linked when -fsanitize=address is on, and the calls are compiled out
+  * otherwise.
+  *
+  * __has_feature is clang's (and GCC >= 14's): it is tested in its own #if,
+  * because an older GCC rejects `__has_feature (x)` even behind a
+  * `defined (__has_feature) &&`. */
+ #if defined(__SANITIZE_ADDRESS__)
+ #define UOS_HAVE_ASAN 1
+ #elif defined(__has_feature)
+ #if __has_feature(address_sanitizer)
+ #define UOS_HAVE_ASAN 1
+ #endif
+ #endif
+
+ #if defined(UOS_HAVE_ASAN)
+ extern "C" void
+ __sanitizer_start_switch_fiber (void** fake_stack_save, const void* bottom,
+                                 std::size_t size);
+
+ extern "C" void
+ __sanitizer_finish_switch_fiber (void* fake_stack_save, const void** bottom_old,
+                                  std::size_t* size_old);
+ #endif
+
+ void
+ asan_start_switch (void** save, const void* bottom, std::size_t size)
+ {
+     #if defined(UOS_HAVE_ASAN)
+         __sanitizer_start_switch_fiber (save, bottom, size);
+     #else
+         (void)save;
+         (void)bottom;
+     #endif
+ }

```

```

+   (void)size;
+ #endif
+ }
+
+ void
+ asan_finish_switch (void* save)
+ {
+ #if defined(UOS_HAVE_ASAN)
+   __sanitizer_finish_switch_fiber (save, nullptr, nullptr);
+ #else
+   (void)save;
+ #endif
+ }
+
+ void
+ publish_pending (void)
+ {
+   // The CPU index must be re-read: this runs after a swapcontext() that
+   // may well have been performed by a different CPU.
+   const unsigned cpu = port_cpu_id ();
+   if (cpu >= OS_NCPU)
+   {
+     return;
+   }
+
+   element_t** addr = g_publish[cpu].addr;
+   if (addr == nullptr)
+   {
+     return;
+   }
+
+   g_publish[cpu].addr = nullptr;
+   __atomic_store_n (addr, g_publish[cpu].val, __ATOMIC_RELEASE);
+ }
+
+ } /* namespace host_cpu */
+
+ extern "C" void port_smp_ipi (unsigned cpu);
+
+ extern "C" void
+ port_smp_ipi (unsigned cpu)
+ {
+   host_cpu::send_ipi (cpu);
+ }
+

```

```

+#endif /* defined(__APPLE__) || defined(__linux__) */
diff --git a/src/rtos/os-core.cpp b/src/rtos/os-core.cpp
index 91689ca..c073489 100644
--- a/src/rtos/os-core.cpp
+++ b/src/rtos/os-core.cpp
@@ -1,34 +1,67 @@
/*
+ * os-core.cpp - the port's half of the µOS++ III SMP scheduler, POSIX host.
+ *
+ * This file is part of the µOS++ project (https://micro-os-plus.github.io/).
+ * Copyright (c) 2016-2025 Liviu Ionescu. All rights reserved.
+ * Copyright (c) 2026 Dan. All rights reserved.
+ *
+ * Permission to use, copy, modify, and/or distribute this software
+ * for any purpose is hereby granted, under the terms of the MIT license.
+ *
- * If a copy of the license was not distributed with this file, it can
- * be obtained from https://opensource.org/licenses/mit/.
+ * -----
+ * The model: a host thread IS a CPU.
+ *
+ * OS_NCPU host threads are created at startup and never destroyed. Each one
+ * runs the scheduler and is, for every purpose the kernel can observe, a
+ * core: it has its own interrupt mask (its signal mask), its own tick
+ * (its own timer_create timer), its own handler-mode flag and its own entry
+ * in lock_state[] and _port_ctx_pending[].
+ *
+ * µOS++ threads remain ucontext contexts switched WITHIN a CPU, which is
+ * upstream's machinery, preserved deliberately. What is new is that a context
+ * saved by one CPU may be resumed by another -- and that is the only genuinely
+ * hard part of this port, because it is a data race unless the handover is
+ * ordered. See switch_stacks() below.
+ *
+ * Read this file beside micro-os-plus-iii-aarch64/src/rtos/os-core.cpp. The
+ * two have the same functions doing the same things in the same order.
+ * -----
+ */

#if defined(__APPLE__) || defined(__linux__)

-// -----
-
#include <cassert>
#include <cstdlib>
#include <cstring>

```



```

#include <cmsis-plus/rtos/os.h>
#include <cmsis-plus/rtos/os-hooks.h>
#include <cmsis-plus/rtos/port/os-inlines.h>

#include <sys/time.h>
#include <host_cpu.hpp>

// -----

#include <sys/utsname.h>
#include <sys/time.h>
#include <unistd.h>

#ifdef __clang__
#pragma clang diagnostic ignored "-Wc++98-compat"
#endif

// -----

extern os::rtos::thread* os_idle_thread;

uint32_t signal_nesting;
#ifdef OS_USE_SMP_SCHEDULER
namespace os
+{
+ namespace rtos
+ {
+ namespace scheduler
+ {
+ extern thread* os_idle_thread_core[OS_NCPU];
+ }
+ }
+}
#endif /* defined(OS_USE_SMP_SCHEDULER) */

namespace os
{
@@ -38,138 +71,26 @@ namespace os
{
// -----

#pragma GCC diagnostic push
#pragma GCC diagnostic ignored "-Wdeprecated-declarations"
+ thread_local unsigned _this_cpu = 0;

- void

```

```

-     context::create (void* context, void* func, void* args)
-     {
-         /* class */ rtos::thread::context* th_ctx
-         = static_cast< /* class */ rtos::thread::context*> (context);
-         memset (&th_ctx->port_, 0, sizeof (th_ctx->port_));
-
-     -#pragma GCC diagnostic push
-     -#if defined(__clang__)
-     -#elif defined(__GNUC__)
-     -#pragma GCC diagnostic ignored "-Wuseless-cast"
-     -#endif
-
-         os_impl_ucontext_t* ctx = reinterpret_cast<os_impl_ucontext_t*> (
-             &(th_ctx->port_.ucontext));
-     -#pragma GCC diagnostic pop
-
-     -#if defined(OS_TRACE_RTOS_THREAD_CONTEXT)
-         trace::printf ("port::context::%s() getcontext %p\n", __func__, ctx);
-     -#endif
-
-         if (os_impl_getcontext (ctx) != 0)
-         {
-             trace::printf ("port::context::%s() getcontext failed with %s\n",
-                             __func__, strerror (errno));
-             abort ();
-         }
-
-         // The context in itself is not needed, but makecontext()
-         // requires a context obtained by getcontext().
-
-         // Remove the parent link.
-         // TODO: maybe use this to link to exit code.
-         ctx->uc_link = nullptr;
-
-         // Configure the new stack to default values.
-         ctx->uc_stack.ss_sp = th_ctx->stack ().bottom ();
-         ctx->uc_stack.ss_size = th_ctx->stack ().size ();
-         ctx->uc_stack.ss_flags = 0;
-
-     -#if defined(OS_TRACE_RTOS_THREAD_CONTEXT)
-         trace::printf ("port::context::%s() makecontext %p\n", __func__, ctx);
-     -#endif
-
-     -#pragma GCC diagnostic push
-     -#if defined(__clang__)
-     -#pragma clang diagnostic ignored "-Wc++98-compat-pedantic"

```

```

-#endif
-    os_impl_makecontext (ctx, reinterpret_cast<func_t> (func), 1, args);
-#pragma GCC diagnostic pop
-
-    // context->port_.saved = false;
-    }
-
-#pragma GCC diagnostic pop
-
-    // -----
-
-    namespace interrupts
-    {
-        // -----
-
-        sigset_t clock_set;
+        sigset_t irq_set;
-
-        // -----
-
-        // Enter an IRQ critical section
-        rtos::interrupts::state_t
-        critical_section::enter (void)
-        {
-#if defined(OS_TRACE_RTOS_SCHEDULER)
-            trace::printf ("c ");
-#endif
-            sigset_t old;
-            sigprocmask (SIG_BLOCK, &clock_set, &old);
-
-#pragma GCC diagnostic push
-#pragma GCC diagnostic ignored "-Wsign-conversion"
-            return sigismember (&old, clock::signal_number);
-#pragma GCC diagnostic pop
-        }
-
-        // Exit an IRQ critical section
-        void
-        critical_section::exit (rtos::interrupts::state_t state)
-        {
-#if defined(OS_TRACE_RTOS_SCHEDULER)
-            trace::printf (" c");
-#endif
-            sigprocmask (state ? SIG_BLOCK : SIG_UNBLOCK, &clock_set, nullptr);
-        }

```

```

-
- // =====
-
- // Enter an IRQ uncritical section
- rtos::interrupts::state_t
- uncritical_section::enter (void)
- {
-#if defined(OS_TRACE_RTOS_SCHEDULER)
-     trace::printf ("{u ");
-#endif
-     sigset_t old;
-     sigprocmask (SIG_UNBLOCK, &clock_set, &old);
-
-#pragma GCC diagnostic push
-#pragma GCC diagnostic ignored "-Wsign-conversion"
-     return sigismember (&old, clock::signal_number);
-#pragma GCC diagnostic pop
- }
-
- // Exit an IRQ critical section
- void
- uncritical_section::exit (rtos::interrupts::state_t state)
- {
-#if defined(OS_TRACE_RTOS_SCHEDULER)
-     trace::printf (" u}");
-#endif
-     sigprocmask (state ? SIG_BLOCK : SIG_UNBLOCK, &clock_set, nullptr);
- }
+ extern "C" volatile bool _in_isr[OS_NCPU];
+ volatile bool _in_isr[OS_NCPU] = {};
+
+ extern "C" volatile uint32_t signal_nesting;
+ volatile uint32_t signal_nesting = 0;
+ } /* namespace interrupts */
-
- // -----
-
- namespace scheduler
- {
+     volatile state_t lock_state[OS_NCPU] = {};
-
- // -----
-
-     state_t lock_state;
+     smp_klock_t _smp_klock = { 0, SMP_NO_OWNER, 0 };

```

```

+     volatile unsigned _port_ctx_pending[OS_NCPU] = {};

+
+     // -----

@@ -179,96 +100,301 @@ namespace os
+     /* struct */ utsname name;
+     if (::uname (&name) != -1)
+     {
-         trace::printf ("POSIX synthetic, running on %s %s %s",
+         trace::printf ("POSIX synthetic SMP, running on %s %s %s",
+                         name.machine, name.sysname, name.release);
+     }
+     else
+     {
-         trace::printf ("POSIX synthetic");
+         trace::printf ("POSIX synthetic SMP");
+     }

-     trace::puts ("; non-preemptive");
+     trace::printf ("; %d CPU%s, %d Hz tick, preemptive\n", OS_NCPU,
+                     (OS_NCPU == 1) ? "" : "s",
+                     OS_INTEGER_SYSTICK_FREQUENCY_HZ);
+ }

+ result_t
+ initialize (void)
+ {
-     signal_nesting = 0;
+     // Must be done before the first critical section: every mask
+     // operation in this port names this set.
+     ::sigemptyset (&interrupts::irq_set);
+     ::sigaddset (&interrupts::irq_set, clock::signal_number ());
+     ::sigaddset (&interrupts::irq_set, clock::ipi_signal_number ());

-     // Must be done before the first critical section.
-     sigemptyset (&interrupts::clock_set);
+     for (unsigned c = 0; c < OS_NCPU; ++c)
+     {
+         lock_state[c] = state::init;
+         _port_ctx_pending[c] = 0;
+         interrupts::_in_isr[c] = false;
+     }

-#pragma GCC diagnostic push
-#if defined(__clang__)

```

```

-#elif defined(__GNUC__)
-#pragma GCC diagnostic ignored "-Wsign-conversion"
-#endif

-    sigaddset (&interrupts::clock_set, clock::signal_number);
-#pragma GCC diagnostic pop
+    host_cpu::install_handlers ();
+
+    /*
+     * The application's hardware hooks.
+     *
+     * On a bare-metal target the kernel's own src/startup/startup.cpp
+     * calls these two before main(), and every carried test relies on
+     * that: os_startup_initialize_hardware() is where a test brings up
+     * its console, prints its banner, installs the free store and calls
+     * exception::init().
+     *
+     * Here there is no such startup. Upstream's NOTES.md states the
+     * rule -- "For portability reasons, execution starts in the main()
+     * function" -- and startup.cpp is __ARM_EABI__-guarded from end to
+     * end, so nothing calls them at all. The first thing every test's
+     * main() calls is scheduler::initialize(), so this is where that
+     * startup belongs, and the tests need no edit.
+     *
+     * Found by running: without this the banner never printed, the
+     * application free store was never installed (the kernel's
+     * malloc resource stayed in place), and exception::init() never
+     * ran -- so the first SMP fault reported nothing at all.
+     *
+     * After install_handlers(), deliberately: exception::init() wants
+     * the alternate signal stack that call sets up.
+     */
+    os_startup_initialize_hardware_early ();
+    os_startup_initialize_hardware ();
+
+    // Interrupts masked until start(), as every port does.
+    ::pthread_sigmask (SIG_BLOCK, &interrupts::irq_set, nullptr);

    return result::ok;
}

// -----

-#pragma GCC diagnostic push
-#pragma GCC diagnostic ignored "-Wdeprecated-declarations"
-

```

```

-     void
-     start (void)
+     state_t
+     locked (state_t state)
    {
-         {
-             rtos::interrupts::critical_section ics;
-
-             // Determine the next thread.
-             rtos::scheduler::current_thread_
-                 = rtos::scheduler::ready_threads_list_.unlink_head ();
-         }
+         os_assert_throw (!interrupts::in_handler_mode (), EPERM);

-#pragma GCC diagnostic push
-#if defined(__clang__)
-#elif defined(__GNUC__)
-#pragma GCC diagnostic ignored "-Wuseless-cast"
-#endif
-         os_impl_ucontext_t* new_context
-             = reinterpret_cast<os_impl_ucontext_t*> (&(
-                 rtos::scheduler::current_thread_->context_.port_.ucontext));
-#pragma GCC diagnostic pop
-
-#if defined(OS_TRACE_RTOS_THREAD_CONTEXT)
-         trace::printf ("port::scheduler::%s() ctx %p %s\n", __func__,
-                         new_context,
-                         rtos::scheduler::current_thread_->name ());
-#endif
+         if (state == state::locked)
+         {
+             // Block the tick BEFORE reading the CPU id. The tick handler
+             // swapcontext()s, and the thread can resume on another host
+             // thread: read before the block, the id could name the CPU it
+             // left, and lock_state[] and the kernel lock would be taken for
+             // that CPU, never to be released by the matching unlock.
+             ::pthread_sigmask (SIG_BLOCK, &interrupts::irq_set, nullptr);
+             const unsigned cpu = port_cpu_id ();
+             state_t tmp = lock_state[cpu];
+             if (tmp != state::locked)
+             {
+                 if (_smp_klock.owner != cpu)
+                 {
+                     _smp_klock_raw_acquire ();
+                     _smp_klock.owner = cpu;

```

```

+         }
+         _smp_klock.depth = _smp_klock.depth + 1;
+         lock_state[cpu] = state::locked;
+     }
+     return tmp;
+ }
+ else
+ {
+     // Holding the lock, the tick is blocked and the thread cannot
+     // move; not holding it, both CPUs read "unlocked" anyway.
+     const unsigned cpu = port_cpu_id ();
+     state_t tmp = lock_state[cpu];
+     if (tmp != state::unlocked)
+     {
+         lock_state[cpu] = state::unlocked;
+         if (_smp_klock.owner == cpu && _smp_klock.depth > 0)
+         {
+             const uint32_t d = _smp_klock.depth - 1;
+             _smp_klock.depth = d;
+             if (d == 0)
+             {
+                 _smp_klock.owner = SMP_NO_OWNER;
+                 _smp_klock_raw_release ();
+             }
+         }
+         ::pthread_sigmask (SIG_UNBLOCK, &interrupts::irq_set,
+                             nullptr);
+     }
+     return tmp;
+ }
+ }

-     lock_state = state::init;
+ // -----

-#if defined NDEBUG
-     os_impl_setcontext (new_context);
-#else
-     int res = os_impl_setcontext (new_context);
-     assert (res == 0);
-#endif
-     abort ();
+     void
+     wait_for_interrupt (void)
+     {

```



```

+ // The idle thread's "WFI". sigsuspend() unblocks this CPU's tick
+ // and IPI and sleeps until one of them is delivered AND handled, so
+ // an idle CPU costs nothing while still being preemptible.
+ sigset_t mask;
+ ::pthread_sigmask (SIG_SETMASK, nullptr, &mask);
+ ::sigdelset (&mask, clock::signal_number ());
+ ::sigdelset (&mask, clock::ipi_signal_number ());
+ ::sigsuspend (&mask);
+ }

// -----

- state_t
- locked (state_t state)
+ /*
+  * The context switch.
+  *
+  * Named switch_stacks() because that is what the kernel calls this
+  * seam on every port, and because it is a friend of rtos::thread --
+  * which is how it may touch context_ at all. Unlike the ARM ports it
+  * performs the switch itself rather than returning a stack pointer to
+  * an assembly restore path; there is no assembly here to return to.
+  *
+  * THE DEFERRED PUBLISH.
+  *
+  * The SMP picker in the kernel skips any thread whose stack_ptr is
+  * null, meaning "not safe to claim". The window that rule exists for
+  * is real here too: between the moment this CPU decides to leave
+  * old_thread and the moment swapcontext() has finished writing
+  * old_thread's registers into its ucontext, another CPU must not
+  * resume it -- it would run a half-saved context.
+  *
+  * So the publish cannot be done by the CPU that is leaving: once
+  * swapcontext() returns, this CPU is already executing the INCOMING
+  * thread. It is done by whoever arrives next on this CPU instead. The
+  * outgoing thread's address and value are left in this CPU's slot,
+  * and the first thing any resumed context does -- here, after
+  * swapcontext(), and in host_cpu's trampoline for a context that has
+  * never run -- is publish it.
+  *
+  * This is the same mechanism as AArch64's _smp_pub_addr/_smp_pub_val
+  * pair, which its assembly restore path applies only after SP has
+  * left the outgoing stack. Same hazard, same answer, different
+  * machine.
+  */

```

```

+     stack::element_t*
+     switch_stacks (stack::element_t* sp)
+     {
-         os_assert_throw (!interrupts::in_handler_mode (), EPERM);
+         (void)sp;
+
+         /*
+          * MASK THIS CPU FIRST. The switch must be atomic with respect to
+          * this CPU's own interrupts, and on the ARM ports it is atomic for
+          * free: the whole of switch_stacks() runs inside the IRQ path, with
+          * interrupts already masked by the exception entry.
+          *
+          * Here it is not free. reschedule() reaches this function directly
+          * from thread mode, where this CPU's tick is unmasked -- so the
+          * timer could land between clearing old_thread's publish flag and
+          * swapcontext() finishing the save, and the handler would re-enter
+          * this same function on a half-performed switch. That is a
+          * genuinely reentrant context switch, and it was the cause of the
+          * intermittent SIGSEGV this port showed on smp_test2 (roughly three
+          * runs in five) the first time it ran multi-core.
+          *
+          * The mask is saved by swapcontext() into the outgoing context and
+          * restored from the incoming one, so it follows the thread across
+          * CPUs, which is exactly what interrupt state should do.
+          */
+         sigset_t saved_mask;
+         ::pthread_sigmask (SIG_BLOCK, &interrupts::irq_set, &saved_mask);
+
+         const unsigned cpu = port_cpu_id ();
+
+         // Take the kernel lock outright. reschedule() has already returned
+         // early if this CPU still owns it, so it is not held here.
+         _smp_klock_raw_acquire ();
+         _smp_klock.owner = cpu;
+         _smp_klock.depth = 1;
+
+         rtos::thread* old_thread = host_cpu::current_thread (cpu);
+
+         stack::element_t** pub_addr
+             = &old_thread->context_.port_.stack_ptr;
+         // Any stable non-null value: to the kernel this field is a flag,
+         // and the only thing it ever asks is whether it is null.
+         stack::element_t* pub_val = reinterpret_cast<stack::element_t*> (
+             &old_thread->context_.port_.ucontext);
+
+

```

```

+      // Park the outgoing thread before the picker can see it. It may
+      // still re-pick it -- the kernel's test is
+      // `th == old_thread || stack_ptr != nullptr` -- which is correct,
+      // because this CPU never left it.
+      __atomic_store_n (pub_addr, static_cast<stack::element_t*> (nullptr),
+                        __ATOMIC_RELEASE);
+
+      rtos::scheduler::internal_switch_threads ();
+
+      rtos::thread* new_thread = host_cpu::current_thread (cpu);
+
+      if (new_thread == nullptr)
+      {
+          trace::printf (
+              "\n!!! no ready thread and no idle thread on CPU %u !!!\n",
+              cpu);
+          ::abort ();
+      }
+
-      state_t tmp;
+      if (new_thread == old_thread)
+      {
+          // Nothing to do; republish immediately and let go.
+          __atomic_store_n (pub_addr, pub_val, __ATOMIC_RELEASE);
+          _smp_klock.depth = 0;
+          _smp_klock.owner = SMP_NO_OWNER;
+          _smp_klock_raw_release ();
+          ::pthread_sigmask (SIG_SETMASK, &saved_mask, nullptr);
+          return nullptr;
+      }
+
-      {
-          rtos::interrupts::critical_section ics;
+      os_impl_ucontext_t* new_uc = &new_thread->context_.port_.ucontext;
+
+      // Claim the incoming context: from here no other CPU may take it.
+      __atomic_store_n (&new_thread->context_.port_.stack_ptr,
+                        static_cast<stack::element_t*> (nullptr),
+                        __ATOMIC_RELEASE);
+
+      host_cpu::defer_publish (cpu, pub_addr, pub_val);
+
+      /* Release the kernel lock. THE ORDER MATTERS, and it is the same
+       * order and the same reason as the AArch64 port documents at
+       * length: owner and depth are cleared BEFORE the lock word.

```

```

+      *
+      * Storing the lock word first opens a window in which another CPU
+      * wins the lock and installs its own owner/depth, which the two
+      * stores below then wipe. Its critical_section::exit() is guarded
+      * by (owner == cpu && depth > 0), so it never clears the lock word
+      * again, and every CPU spins for ever -- a silent, total freeze.
+      * On the ARM ports that was the root cause of the smp_test4
+      * hardware deadlock. */
+      _smp_klock.depth = 0;
+      _smp_klock.owner = SMP_NO_OWNER;
+      _smp_klock_raw_release ();
+
+      os_impl_ucontext_t* old_uc = &old_thread->context_.port_.ucontext;
+
+      /* Tell AddressSanitizer the stack is about to move, and where to.
+      * `asan_save` lives on the OUTGOING thread's stack, so it is still
+      * there -- and still this thread's -- whenever and on whichever CPU
+      * that thread is resumed. Compiles to nothing without -fsanitize=
+      * address; see host_cpu.hpp. */
+      void* asan_save = nullptr;
+      host_cpu::asan_start_switch (&asan_save,
+                                   new_thread->stack ().bottom (),
+                                   new_thread->stack ().size ());
+
+      if (os_impl_swapcontext (old_uc, new_uc) != 0)
+      {
+          trace::printf ("port::scheduler::%s() swapcontext failed: %s\n",
+                          __func__, strerror (errno));
+          ::abort ();
+      }
+
+      tmp = lock_state;
+      lock_state = state;
+
+      }
+
+      // Resumed -- and NOT necessarily on the CPU that left. Everything
+      // below must re-read the CPU index; nothing captured above is
+      // valid any more.
+
+      host_cpu::asan_finish_switch (asan_save);
+      host_cpu::publish_pending ();
+
+      return tmp;
+
+      // Unmask last, and only now: everything above this line is the
+      // switch, and the switch is not interruptible.
+
+      ::pthread_sigmask (SIG_SETMASK, &saved_mask, nullptr);
+
+

```

```

+         return nullptr;
+     }

    // -----

@@ -276,145 +402,141 @@ namespace os
    void
    reschedule (void)
    {
+         const unsigned cpu = port_cpu_id ();
+
        if (rtos::scheduler::locked ()
-         || rtos::interrupts::in_handler_mode ())
+         || (rtos::interrupts::in_handler_mode ()
+         && !rtos::scheduler::preemptive ()))
        {
-#if defined(OS_TRACE_RTOS_THREAD_CONTEXT)
-         // trace::printf ("port::scheduler::%s() deny\n", __func__);
-#endif
            return;
        }

-#if defined(OS_TRACE_RTOS_THREAD_CONTEXT)
-         trace::printf ("port::scheduler::%s()\n", __func__);
-#endif
+         // If this CPU still owns the kernel lock -- resume_one() called
+         // from inside an interrupts::critical_section, as message_queue's
+         // send/receive do -- switching here would strand owner and depth on
+         // the outgoing thread's context and deadlock the next acquirer.
+         // Defer to the next tick, which runs once that section has exited.
+         if (_smp_klock.owner == cpu && _smp_klock.depth > 0)
+         {
+             _port_ctx_pending[cpu] = 1;
+             return;
+         }

-         // For some complicated reasons, the context save/restore
-         // functions must be called in the same the function.
-         // The idea to inline functions does not work, since
-         // the compiler does not inline functions with context calls.
+         if (rtos::interrupts::in_handler_mode ())
+         {
+             // Taken on the way out of the handler, where the mask is right.
+             _port_ctx_pending[cpu] = 1;
+             return;
+         }

```

```

-         bool save = false;
-         rtos::thread* old_thread;
-         os_impl_ucontext_t* old_ctx;
-         os_impl_ucontext_t* new_ctx;
+         switch_stacks (nullptr);
+     }

-     {
-         rtos::interrupts::critical_section ics;
-
-         old_thread = rtos::scheduler::current_thread_;
-         if ((old_thread->state_ == rtos::thread::state::running)
-             || (old_thread->state_ == rtos::thread::state::suspended)
-             || (old_thread->state_ == rtos::thread::state::ready))
-         {
-             save = true;
-         }
-#if defined(OS_TRACE_RTOS_THREAD_CONTEXT)
-         trace::printf ("port::scheduler::%s() from %s %d %d\n", __func__,
-             old_thread->name (), old_thread->state_, save);
-#endif
+         // -----

-#pragma GCC diagnostic push
-#if defined(__clang__)
-#elif defined(__GNUC__)
-#pragma GCC diagnostic ignored "-Wuseless-cast"
-#endif
-         old_ctx = reinterpret_cast<os_impl_ucontext_t*> (
-             &old_thread->context_.port_.ucontext);
-#pragma GCC diagnostic pop
+         [[noreturn]] void
+         start (void)
+         {
+             ::pthread_sigmask (SIG_BLOCK, &interrupts::irq_set, nullptr);
+
+             // A context to switch AWAY from. The very first switch has to save
+             // something, and this CPU's host-thread context is not a µOS++
+             // thread. The ARM ports use a zeroed fake_thread for the same
+             // reason; this one is never scheduled again, so it needs no stack.
+             static os_thread_t fake_thread[OS_NCPU];
+             const unsigned cpu = port_cpu_id ();
+             memset (&fake_thread[cpu], 0, sizeof (os_thread_t));
+             fake_thread[cpu].name = "fake_thread";

```

```

+         host_cpu::current_thread (cpu)
+             = reinterpret_cast<rtos::thread*> (&fake_thread[cpu]);
+
+         for (unsigned c = 0; c < OS_NCPU; ++c)
+             {
+                 lock_state[c] = state::init;
+             }

-         rtos::scheduler::internal_switch_threads ();
+#if defined(OS_USE_SMP_SCHEDULER)
+         // The SMP picker asks each CPU for its own idle thread; CPU 0's is
+         // the kernel's. The secondaries' are installed by the board, from
+         // test-smp-boot.cpp, exactly as on the ARM boards.
+         rtos::scheduler::os_idle_thread_core[0] = ::os_idle_thread;
+#endif /* defined(OS_USE_SMP_SCHEDULER) */

-#pragma GCC diagnostic push
-#if defined(__clang__)
-#elif defined(__GNUC__)
-#pragma GCC diagnostic ignored "-Wuseless-cast"
-#endif

-         new_ctx = reinterpret_cast<os_impl_ucontext_t*> (
-             &rtos::scheduler::current_thread->context_.port_.ucontext);
-#pragma GCC diagnostic pop
-         }
+         host_cpu::start_this_cpu_tick ();

-         if (old_ctx != new_ctx)
-             {
-                 if (save)
-                     {
-#if defined(OS_TRACE_RTOS_THREAD_CONTEXT)
-                     trace::printf (
-                         "port::scheduler::%s() swapcontext %s -> %s \n",
-                         __func__, old_thread->name (),
-                         rtos::scheduler::current_thread->name ());
-#endif
-                     if (os_impl_swapcontext (old_ctx, new_ctx) != 0)
-                         {
-                             trace::printf (
-                                 "port::scheduler::%s() swapcontext failed with %s\n",
-                                 __func__, strerror (errno));
-                             abort ();
-                         }
-                     }
-             }

```

```

-         else
-         {
-#if defined(OS_TRACE_RTOS_THREAD_CONTEXT)
-             trace::printf ("port::scheduler::%s() setcontext %s\n",
-                             __func__,
-                             rtos::scheduler::current_thread->name ());
-#endif
-
-             // context->port_.saved = false;
-             if (os_impl_setcontext (new_ctx) != 0)
-             {
-                 trace::printf (
-                     "port::scheduler::%s() setcontext failed with %s\n",
-                     __func__, strerror (errno));
-                 abort ();
-             }
-         }
-     }
-     else
+     reschedule ();
+
+     // Reached only if reschedule() found nothing to run at all.
+     ::pthread_sigmask (SIG_UNBLOCK, &interrupts::irq_set, nullptr);
+     for (;;)
+     {
-#if defined(OS_TRACE_RTOS_THREAD_CONTEXT)
-         trace::printf ("port::scheduler::%s() same %s\n", __func__,
-                         old_thread->name ());
-#endif
+         ::pause ();
+     }
+ }

-#pragma GCC diagnostic pop
-
-     // -----
-
-     } /* namespace scheduler */
-
-     // -----

-     static void
-     systick_clock_signal_handler (int signum)
+     void
+     context::create (void* context, void* func, void* args)
+     {

```



```

-#if defined(OS_TRACE_RTOS_SYSCLOCK_TICK)
-    trace::printf ("i ");
-#endif
-
-    if (signum != clock::signal_number)
-    {
-        char ce = '?';
-
-#pragma GCC diagnostic push
-#pragma GCC diagnostic ignored "-Wunused-result"
+        /* class */ rtos::thread::context* th_ctx
+        = static_cast< /* class */ rtos::thread::context*> (context);
-
-        write (1, &ce, 1);
+        memset (&th_ctx->port_, 0, sizeof (th_ctx->port_));
-
-#pragma GCC diagnostic pop
+        os_impl_ucontext_t* ctx = &th_ctx->port_.ucontext;
-
-        return;
+        if (os_impl_getcontext (ctx) != 0)
+        {
+            trace::printf ("port::context::%s() getcontext failed: %s\n",
+                __func__, strerror (errno));
+            ::abort ();
+        }
-
-        signal_nesting++;
-        // Call the ticks timer ISR.
-        os_systick_handler ();
-        signal_nesting--;
-#if defined(OS_TRACE_RTOS_SYSCLOCK_TICK)
-    trace::printf (" i}");
+    // The context itself is not wanted; makecontext() merely requires one
+    // obtained from getcontext().
+    ctx->uc_link = nullptr;
+    ctx->uc_stack.ss_sp = th_ctx->stack ().bottom ();
+    ctx->uc_stack.ss_size = th_ctx->stack ().size ();
+    ctx->uc_stack.ss_flags = 0;
+
+    /*
+     * The starting signal mask: THIS CPU'S INTERRUPTS MASKED.
+     *
+     * Two wrong answers were tried before this one, which is why the
+     * reasoning is written down.

```

```

+      *
+      * getcontext() snapshots the CALLER's mask, and a thread is very
+      * often created from inside a critical section -- so inheriting it
+      * would start the thread with interrupts masked FOR EVER, because
+      * nothing would ever unmask them. Clearing the mask entirely is
+      * wrong in the opposite direction: a context resumed by
+      * switch_stacks() comes back with interrupts masked and unmask only
+      * after it has discharged this CPU's deferred publish, and a context
+      * that has never run arrives on a CPU owing exactly the same publish.
+      * Starting it unmasked lets a tick land inside the trampoline before
+      * that publish happens -- and that tick's own switch overwrites the
+      * pending slot, so the thread it was owed to is never republished and
+      * is lost from the ready list for good.
+      *
+      * So a new context begins the way a resumed one does: masked. The
+      * trampoline publishes, then unmask, and from that point the thread
+      * is preemptible like any other.
+      */
+
+#if !defined(OS_INCLUDE_LIBUCONTEXT)
+    ::sigemptyset (&ctx->uc_sigmask);
+    ::sigaddset (&ctx->uc_sigmask, clock::signal_number ());
+    ::sigaddset (&ctx->uc_sigmask, clock::ipi_signal_number ());
+
+    #endif
+
+    host_cpu::make_entry (ctx, func, args);
+
+    // Published: this context has never run, so it is complete by
+    // definition and any CPU may claim it.
+    th_ctx->port_.stack_ptr
+        = reinterpret_cast<stack::element_t*> (&th_ctx->port_.ucontext);
+    }
+
+    // =====
+
+@@ -422,75 +544,11 @@ namespace os
+    void
+    clock_systick::start (void)
+    {
+    -    // set handler
+    -    struct sigaction sa;
+    -#if defined(__APPLE__)
+    -    sa.__sigaction_u.__sa_handler = systick_clock_signal_handler;
+    -#elif defined(__linux__)
+    -#pragma GCC diagnostic push
+    -#if defined(__clang__)
+    -#pragma clang diagnostic ignored "-Wdisabled-macro-expansion"

```

```

-#endif
-    sa.sa_handler = systick_clock_signal_handler;
-#pragma GCC diagnostic pop
-#else
-#error Platform unsupported
-#endif
-    sigemptyset (&sa.sa_mask);
-    sa.sa_flags = SA_RESTART;
-
-    if (sigaction (clock::signal_number, &sa, nullptr) != 0)
-    {
-        trace::printf ("port::clock_systick::%s() sigaction() failed\n",
-            __func__);
-        abort ();
-    }
-
-    // set timer
-    /* struct */ itimerval tv;
-    // first clear all fields
-#if defined(__APPLE__)
-    memset (&tv, 0, sizeof (tv));
-#else
-    timerclear (&tv.it_value);
-#endif
-    // then set the required ones
-
-#if 1
-    tv.it_value.tv_sec = 0;
-    tv.it_value.tv_usec = 1000000 / rtos::clock_systick::frequency_hz;
-    tv.it_interval.tv_sec = 0;
-    tv.it_interval.tv_usec = 1000000 / rtos::clock_systick::frequency_hz;
-#else
-    tv.it_value.tv_sec = 1;
-    tv.it_value.tv_usec = 0; // 1000000 /
-        // rtos::clock_systick::frequency_hz;
-    tv.it_interval.tv_sec = 1;
-    tv.it_interval.tv_usec
-        = 0; // 1000000 / rtos::clock_systick::frequency_hz;
-#endif
-
-    if (setitimer (ITIMER_REAL, &tv, nullptr) != 0)
-    {
-        trace::printf ("port::clock_systick::%s() setitimer() failed\n",
-            __func__);
-        abort ();
-    }

```

```

-     }
-
-#if 0
-    // Used for initial debugging, to see the signals arriving
-    pause ();
-    for (int i = 50; i > 0; --i)
-    {
-        for (int j = 100; j > 0; --j)
-        {
-            char c = '.';
-            write (1, &c, 1);
-        }
-        char cn = '\n';
-        write (1, &cn, 1);
-    }
-#endif
+    // Each CPU arms its own timer when it starts; this is CPU 0's, and
+    // the secondaries' are armed by host_cpu::start_secondary_cpus().
+    // Only CPU 0 advances the kernel clock -- exactly as on the BCM2837,
+    // where all four cores take a 1 ms PPI but only core 0 calls
+    // os_systick_handler(). The others use theirs to preempt themselves.
+    }

    // =====
@@ -498,15 +556,13 @@ namespace os
    static uint64_t previous_timestamp;

    static uint64_t
-    get_current_micros (void);
-
-    uint64_t
    get_current_micros (void)
    {
-        /* struct */ timeval tp;
-        gettimeofday (&tp, nullptr);
+        /* struct */ timespec tp;
+        ::clock_gettime (CLOCK_MONOTONIC, &tp);

-        return static_cast<uint64_t> (tp.tv_sec * 1000000 + tp.tv_usec);
+        return static_cast<uint64_t> (tp.tv_sec) * 1000000ULL
+            + static_cast<uint64_t> (tp.tv_nsec) / 1000ULL;
    }

    void
@@ -518,8 +574,8 @@ namespace os

```

```

uint32_t
clock_highres::input_clock_frequency_hz (void)
{
-    // The posix system clock resolution is 1 us, so it makes no
-    // sense to assume a frequency higher than 1 MHz.
+    // CLOCK_MONOTONIC is read here at microsecond resolution, so a
+    // higher frequency would be a claim the source cannot support.
    return 1000000;
}

@@ -538,9 +594,8 @@ namespace os
    clock_highres::cycles_since_tick (void)
    {
        uint64_t ts = get_current_micros ();
-        uint32_t delta = static_cast<uint32_t> (ts - previous_timestamp);

-        return delta;
+        return static_cast<uint32_t> (ts - previous_timestamp);
    }

    } /* namespace port */
@@ -549,4 +604,12 @@ namespace os

// -----

+// Never inlined, and never merged or hoisted: see _this_cpu in os-inlines.h.
+extern "C" __attribute__((noinline)) unsigned
+port_cpu_id (void)
+{
+    __asm__ volatile("" ::: "memory");
+    return os::rtos::port::_this_cpu;
+}
+
+
+    #endif /* defined(__APPLE__) || defined(__linux__) */
diff --git a/include/cmsis-plus/rtos/port/os-decls.h b/include/cmsis-plus/rtos/port/os-decls.h
index 9cc63ca..825f285 100644
--- a/include/cmsis-plus/rtos/port/os-decls.h
+++ b/include/cmsis-plus/rtos/port/os-decls.h
@@ -83,6 +83,8 @@
 
 #if defined(__clang__)
 #pragma clang diagnostic ignored "-Wc++98-compat"
+#pragma clang diagnostic ignored "-Wc++98-compat-pedantic"
+#pragma clang diagnostic ignored "-Wreserved-identifier"
 #endif

```

```

namespace os
diff --git a/include/cmsis-plus/rtos/port/os-inlines.h b/include/cmsis-plus/rtos/port/os-inlines.h
index 943da20..e9e0e69 100644
--- a/include/cmsis-plus/rtos/port/os-inlines.h
+++ b/include/cmsis-plus/rtos/port/os-inlines.h
@@ -44,6 +44,8 @@

    #if defined(__clang__)
    #pragma clang diagnostic ignored "-Wc++98-compat"
    #pragma clang diagnostic ignored "-Wc++98-compat-pedantic"
    #pragma clang diagnostic ignored "-Wunsafe-buffer-usage"
    #endif

    // Out of line on purpose; see _this_cpu below.
diff --git a/src/exception_handler.cpp b/src/exception_handler.cpp
index 2a09db9..e607d03 100644
--- a/src/exception_handler.cpp
+++ b/src/exception_handler.cpp
@@ -10,6 +10,21 @@

    #if defined(__APPLE__) || defined(__linux__)

    +// Clang -Weverything hardening for the SMP host port (the destination harness is
    +// stricter than the smp branch's own build): nullptr/alias/range-for, per-CPU
    +// array indexing, signal macros, fatal hooks and the ucontext makecontext cast.
    +// Placed before the includes so the port headers are covered too.
    #if defined(__clang__)
    #pragma clang diagnostic ignored "-Wc++98-compat"
    #pragma clang diagnostic ignored "-Wc++98-compat-pedantic"
    #pragma clang diagnostic ignored "-Wunsafe-buffer-usage"
    #pragma clang diagnostic ignored "-Wdisabled-macro-expansion"
    #pragma clang diagnostic ignored "-Wmissing-noreturn"
    #pragma clang diagnostic ignored "-Wunreachable-code"
    #pragma clang diagnostic ignored "-Wunreachable-code-return"
    #pragma clang diagnostic ignored "-Wcast-function-type-strict"
    #endif
    +
    #include <csignal>
    #include <cstring>
    #include <initializer_list>
@@ -24,6 +39,7 @@
    #include <exception_handler.hpp>
    #include <host_cpu.hpp>

```

```

+
namespace
{
    // Everything below runs in a signal handler after the process is already
diff --git a/src/free-store.cpp b/src/free-store.cpp
index acc7e95..eb743b7 100644
--- a/src/free-store.cpp
+++ b/src/free-store.cpp
@@ -42,6 +42,21 @@

#if defined(__APPLE__) || defined(__linux__)

+// Clang -Weverything hardening for the SMP host port (the destination harness is
+// stricter than the smp branch's own build): nullptr/alias/range-for, per-CPU
+// array indexing, signal macros, fatal hooks and the ucontext makecontext cast.
+// Placed before the includes so the port headers are covered too.
+#if defined(__clang__)
+#pragma clang diagnostic ignored "-Wc++98-compat"
+#pragma clang diagnostic ignored "-Wc++98-compat-pedantic"
+#pragma clang diagnostic ignored "-Wunsafe-buffer-usage"
+#pragma clang diagnostic ignored "-Wdisabled-macro-expansion"
+#pragma clang diagnostic ignored "-Wmissing-noreturn"
+#pragma clang diagnostic ignored "-Wunreachable-code"
+#pragma clang diagnostic ignored "-Wunreachable-code-return"
+#pragma clang diagnostic ignored "-Wcast-function-type-strict"
+#endif
+
+
#include <cstddef>
#include <new>

@@ -50,6 +65,7 @@
#include <cmsis-plus/memory/first-fit-top.h>
#include <cmsis-plus/estd/memory_resource>

+
using namespace os;

#if defined(OS_TYPE_APPLICATION_MEMORY_RESOURCE)
diff --git a/src/host_cpu.cpp b/src/host_cpu.cpp
index 87bd73f..9eeb32b 100644
--- a/src/host_cpu.cpp
+++ b/src/host_cpu.cpp
@@ -10,6 +10,21 @@

#if defined(__APPLE__) || defined(__linux__)

```

```

+// Clang -Weverything hardening for the SMP host port (the destination harness is
+// stricter than the smp branch's own build): nullptr/alias/range-for, per-CPU
+// array indexing, signal macros, fatal hooks and the ucontext makecontext cast.
+// Placed before the includes so the port headers are covered too.
+#if defined(__clang__)
+#pragma clang diagnostic ignored "-Wc++98-compat"
+#pragma clang diagnostic ignored "-Wc++98-compat-pedantic"
+#pragma clang diagnostic ignored "-Wunsafe-buffer-usage"
+#pragma clang diagnostic ignored "-Wdisabled-macro-expansion"
+#pragma clang diagnostic ignored "-Wmissing-noreturn"
+#pragma clang diagnostic ignored "-Wunreachable-code"
+#pragma clang diagnostic ignored "-Wunreachable-code-return"
+#pragma clang diagnostic ignored "-Wcast-function-type-strict"
+#endif
+
+
+  #include <cerrno>
+  #include <cstdio>
+  #include <cstdlib>
+@@ -26,6 +41,7 @@
+
+  #include <host_cpu.hpp>
+
+
+  /* Defined by the board (test/boards/<board>/src/smp.cpp), with the same
+   * meaning as on the ARM boards: 0 = not started, 3 = scheduler entered. The
+   * tests wait on it before they start timing anything. A board that does not
+diff --git a/src/rtos/os-core.cpp b/src/rtos/os-core.cpp
index c073489..f59b35a 100644
--- a/src/rtos/os-core.cpp
+++ b/src/rtos/os-core.cpp
@@ -30,6 +30,21 @@
+
+  #if defined(__APPLE__) || defined(__linux__)
+
+  +// Clang -Weverything hardening for the SMP host port (the destination harness is
+  +// stricter than the smp branch's own build): nullptr/alias/range-for, per-CPU
+  +// array indexing, signal macros, fatal hooks and the ucontext makecontext cast.
+  +// Placed before the includes so the port headers are covered too.
+  #if defined(__clang__)
+  #pragma clang diagnostic ignored "-Wc++98-compat"
+  #pragma clang diagnostic ignored "-Wc++98-compat-pedantic"
+  #pragma clang diagnostic ignored "-Wunsafe-buffer-usage"
+  #pragma clang diagnostic ignored "-Wdisabled-macro-expansion"
+  #pragma clang diagnostic ignored "-Wmissing-noreturn"

```



```
+#pragma clang diagnostic ignored "-Wunreachable-code"
+#pragma clang diagnostic ignored "-Wunreachable-code-return"
+#pragma clang diagnostic ignored "-Wcast-function-type-strict"
+#endif
+
+    #include <cassert>
+    #include <cstdlib>
+    #include <cstring>
```

Part B — Step 13 + 14 — Cortex-M

```

diff --git a/include/cmsis-plus/rtos/port/os-inlines.h b/include/cmsis-plus/rtos/port/os-inlines.h
index bcf1c44..33a1a69 100644
--- a/include/cmsis-plus/rtos/port/os-inlines.h
+++ b/include/cmsis-plus/rtos/port/os-inlines.h
@@ -95,12 +95,14 @@ namespace os
    return locked (state::unlocked);
}

+
+   inline bool __attribute__ ((always_inline))
+   locked (void)
+   {
+       return lock_state != state::unlocked;
+   }
+
+
+   inline void __attribute__ ((always_inline))
+   wait_for_interrupt (void)
+   {
@@ -230,6 +232,7 @@ namespace os

#endif

+
+   // Apparently not required by architecture, but used by
+   // FreeRTOS, with an unconvincing motivation ("... ensure
+   // the code is completely within the specified behaviour
@@ -244,6 +247,7 @@ namespace os
    inline void __attribute__ ((always_inline))
    critical_section::exit (rtos::interrupts::state_t state)
    {
+
+   #if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)

+   #if defined(OS_INTEGER_RTOS_CRITICAL_SECTION_INTERRUPT_PRIORITY)
@@ -362,6 +366,18 @@ namespace os
        ;
    }

+
+   inline constexpr bool __attribute__ ((always_inline))
+   clock_highres::has_hardware_counter (void) noexcept
+   {
+       return false;
+   }
+
+

```

```

+
+     inline uint64_t __attribute__ ((always_inline))
+     clock_highres::hardware_counter (void) noexcept
+     {
+         return 0;
+     }
+
+
+     inline uint32_t __attribute__ ((always_inline))
+     clock_highres::input_clock_frequency_hz (void)
+     {
@@ -391,7 +407,7 @@ namespace os
        // yet processed, so the total cycles count in steady_count_
        // does not yet reflect the correct value and needs to be
        // adjusted by one full cycle length.
-        if (SysTick->CTRL & SCB_ICSR_PENDSTSET_Msk)
+        if ((SCB->ICSR & SCB_ICSR_PENDSTSET_Msk) != 0)
        {
            // Sample the decrementing counter again to validate the
            // initial sample.
diff --git a/include/cmsis-plus/rtos/port/os-c-decls.h b/include/cmsis-plus/rtos/port/os-c-decls.h
index 2eacaf3..348df94 100644
--- a/include/cmsis-plus/rtos/port/os-c-decls.h
+++ b/include/cmsis-plus/rtos/port/os-c-decls.h
@@ -48,6 +48,13 @@ typedef struct
     os_port_thread_stack_element_t* stack_ptr;
 } os_port_thread_context_t;

+#if defined(OS_USE_SMP_SCHEDULER)
+
+/* The kernel lock's owner when nobody holds it (os-decls.h, smp_klock_t). */
+#define SMP_NO_OWNER 0xFFFFFFFFu
+
+#endif /* defined(OS_USE_SMP_SCHEDULER) */
+
+#define OS_INTEGER_RTOS_STACK_FILL_MAGIC (0xEFBEADDE)

#define OS_HAS_INTERRUPTS_STACK
diff --git a/include/cmsis-plus/rtos/port/os-decls.h b/include/cmsis-plus/rtos/port/os-decls.h
index 8036342..1747acc 100644
--- a/include/cmsis-plus/rtos/port/os-decls.h
+++ b/include/cmsis-plus/rtos/port/os-decls.h
@@ -116,8 +116,34 @@ namespace os
    constexpr state_t init = unlocked;
} /* namespace state */

```

```

+#if defined(OS_USE_SMP_SCHEDULER)
+
+    // The kernel's SMP scheduler. Both cores declare it here: upstream's
+    // at OS_NCPU=1 (os-core.cpp), and the RP2350's at OS_NCPU=1 or 2
+    // (os-core-rp2350.cpp, whose include-rp2350/ has no os-decls.h of its
+    // own). One lock state per CPU, as the SMP kernel indexes it.
+    extern state_t lock_state[OS_NCPU];
+    extern uint32_t lock_primask[OS_NCPU];
+
+    // The kernel lock, recursive. owner/depth record who holds it and how
+    // deeply, the bookkeeping the SMP ports keep. What excludes the other
+    // core is the core's business: with one CPU, masking interrupts IS the
+    // mutual exclusion, so there is no lock word and no LDREX/STREX; on
+    // the RP2350 it is SIO hardware spinlock 0 (include-rp2350/.../os-inlines.h).
+    struct smp_klock_t
+    {
+        volatile uint32_t owner;
+        volatile uint32_t depth;
+    };
+
+    extern smp_klock_t _smp_klock;
+
+#else
+
+    extern state_t lock_state;
+
+#endif /* defined(OS_USE_SMP_SCHEDULER) */
+
+    } /* namespace scheduler */
+
+    using thread_context_t = struct context_s
diff --git a/include/cmsis-plus/rtos/port/os-inlines.h b/include/cmsis-plus/rtos/port/os-inlines.h
index 33a1a69..813d114 100644
--- a/include/cmsis-plus/rtos/port/os-inlines.h
+++ b/include/cmsis-plus/rtos/port/os-inlines.h
@@ -64,6 +64,9 @@ namespace os
    trace::printf ("0/0+");
+
+    #endif
+
+    trace::printf ("", preemptive");
+#if defined(OS_USE_SMP_SCHEDULER)
+    trace::printf ("", SMP %u CPU", static_cast<unsigned> (OS_NCPU));
+#endif
+
+    #if (defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)) \
+        && defined(OS_INTEGER_RTOS_CRITICAL_SECTION_INTERRUPT_PRIORITY)
+        trace::printf ("", BASEPRI(%u)",

```

```

@@ -95,6 +98,47 @@ namespace os
    return locked (state::unlocked);
}

+#if defined(OS_USE_SMP_SCHEDULER)
+
+    // One CPU: it is always CPU 0 (os-core.cpp asserts OS_NCPU == 1).
+    inline unsigned __attribute__ ((always_inline))
+    port_cpu_id (void)
+    {
+        return 0;
+    }
+
+    inline bool __attribute__ ((always_inline))
+    locked (void)
+    {
+        return lock_state[port_cpu_id ()] != state::unlocked;
+    }
+
+    // Enter/leave the kernel lock. The caller has already masked
+    // interrupts (critical_section, switch_stacks); with one CPU that is
+    // the exclusion, and this only records the owner and the nesting
+    // depth. Nothing spins: there is no other CPU to wait for.
+    inline void __attribute__ ((always_inline))
+    _smp_klock_enter (void)
+    {
+        _smp_klock.owner = port_cpu_id ();
+        _smp_klock.depth = _smp_klock.depth + 1;
+    }
+
+    inline void __attribute__ ((always_inline))
+    _smp_klock_exit (void)
+    {
+        if (_smp_klock.depth > 0)
+        {
+            uint32_t d = _smp_klock.depth - 1;
+            _smp_klock.depth = d;
+            if (d == 0)
+            {
+                _smp_klock.owner = SMP_NO_OWNER;
+            }
+        }
+    }
+
+#else

```

```

        inline bool __attribute__ ((always_inline))
        locked (void)
@@ -102,6 +146,7 @@ namespace os
        return lock_state != state::unlocked;
    }

+endif /* defined(OS_USE_SMP_SCHEDULER) */

        inline void __attribute__ ((always_inline))
        wait_for_interrupt (void)
@@ -232,6 +277,10 @@ namespace os

    #endif

+if defined(OS_USE_SMP_SCHEDULER)
+    // Interrupts are masked: take the kernel lock.
+    scheduler::_smp_klock_enter ();
+endif

    // Apparently not required by architecture, but used by
    // FreeRTOS, with an unconvincing motivation ("... ensure
@@ -247,6 +296,10 @@ namespace os
        inline void __attribute__ ((always_inline))
        critical_section::exit (rtos::interrupts::state_t state)
        {
+if defined(OS_USE_SMP_SCHEDULER)
+    // Release the kernel lock while interrupts are still masked.
+    scheduler::_smp_klock_exit ();
+endif

    #if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)

diff --git a/src/rtos/os-core.cpp b/src/rtos/os-core.cpp
index d88d5ec..ad9b65a 100644
--- a/src/rtos/os-core.cpp
+++ b/src/rtos/os-core.cpp
@@ -62,12 +62,40 @@ static_assert (OS_INTEGER_RTOS_CRITICAL_SECTION_INTERRUPT_PRIORITY

    #endif /* defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__) */

+// The kernel's SMP scheduler on ONE CPU. This core has no second core to
+// start, no cross-core lock word and no IPI: the SMP branch below exists so
+// the kernel's SMP code paths (per-CPU current thread and lock state, the
+// affinity-aware picker, the stack_ptr liveness rule) run on a single Cortex-M.

```

```

+// More CPUs need a board that supplies a lock and an IPI (see CMakeLists.txt).
+#if defined(OS_USE_SMP_SCHEDULER)
+static_assert (OS_NCPU == 1,
+
+               "the upstream Cortex-M core runs the SMP scheduler on 1 CPU "
+
+               "only");
+#endif /* defined(OS_USE_SMP_SCHEDULER) */
+
+// -----

#if defined(__ARM_ARCH_6M__)
extern uint32_t __vectors_start;
#endif

+#if defined(OS_USE_SMP_SCHEDULER)
+// The kernel's idle thread, registered as CPU 0's idle in start().
+extern os::rtos::thread* os_idle_thread;
+
+namespace os
+{
+
+  namespace rtos
+  {
+
+    {
+
+      // The kernel's per-CPU idle threads (kernel os-core.cpp).
+      extern thread* os_idle_thread_core[OS_NCPU];
+    } /* namespace scheduler */
+  } /* namespace rtos */
+} /* namespace os */
+#endif /* defined(OS_USE_SMP_SCHEDULER) */
+
+// -----

extern "C"
@@ -358,7 +386,12 @@ namespace os

    namespace scheduler
    {
+#if defined(OS_USE_SMP_SCHEDULER)
+    state_t lock_state[OS_NCPU];
+    smp_klock_t _smp_klock = { SMP_NO_OWNER, 0 };
+#else
+    state_t lock_state;
+#endif /* defined(OS_USE_SMP_SCHEDULER) */

    result_t

```



```

        initialize (void)
@@ -494,7 +527,15 @@ namespace os
        rtos::thread* pth = reinterpret_cast<rtos::thread*> (&fake_thread);

        // Make the fake thread look like the current thread.
+#if defined(OS_USE_SMP_SCHEDULER)
+        rtos::scheduler::current_thread_[0] = pth;
+
+        // The picker falls back to the CPU's idle thread; CPU 0's is the
+        // kernel's.
+        rtos::scheduler::os_idle_thread_core[0] = ::os_idle_thread;
+#else
        rtos::scheduler::current_thread_ = pth;
+#endif /* defined(OS_USE_SMP_SCHEDULER) */

        // Trigger the PendSV; the exception will happen a bit later,
        // after re-enabling the interrupts.
@@ -507,7 +548,11 @@ namespace os

        #endif

+#if defined(OS_USE_SMP_SCHEDULER)
+        lock_state[0] = state::init;
+#else
        lock_state = state::init;
+#endif /* defined(OS_USE_SMP_SCHEDULER) */

        // Enable all interrupts; allow PendSV to occur.
        __enable_irq ();
@@ -531,8 +576,17 @@ namespace os
        {
            rtos::interrupts::critical_section ics;

+#if defined(OS_USE_SMP_SCHEDULER)
+            // With one CPU the scheduler lock stays what it is upstream: a
+            // flag the picker reads. It holds no interrupt mask; no other
+            // core can run the picker meanwhile.
+            unsigned cpu = port_cpu_id ();
+            tmp = lock_state[cpu];
+            lock_state[cpu] = state;
+#else
            tmp = lock_state;
            lock_state = state;
+#endif /* defined(OS_USE_SMP_SCHEDULER) */
        }

```

```

        return tmp;
@@ -830,6 +884,44 @@ namespace os
        SCB->ICSR = SCB_ICSR_PENDSVCLR_Msk;
#pragma GCC diagnostic pop

+#if defined(OS_USE_SMP_SCHEDULER)
+
+        // Interrupts are masked: take the kernel lock for the picker.
+        _smp_klock_enter ();
+
+        unsigned cpu = port_cpu_id ();
+        rtos::thread* old_thread = rtos::scheduler::current_thread[cpu];
+
+        // Save the current SP in the initial context. This also publishes
+        // the outgoing thread as switchable again: the SMP picker skips a
+        // thread whose stack_ptr is null (its context is live on a CPU).
+        old_thread->context_.port_.stack_ptr = sp;
+
+        rtos::scheduler::internal_switch_threads ();
+
+        rtos::thread* new_thread = rtos::scheduler::current_thread[cpu];
+
+        if (new_thread == nullptr)
+        {
+            // No ready thread and no idle thread -- fatal. Release the
+            // kernel lock first so the picker cannot be entered by anything
+            // else, then halt with interrupts masked (do not restore the
+            // saved mask: the scheduler is already torn).
+            _smp_klock_exit ();
+            __asm__ volatile ("cpsid if" ::: "memory");
+            for (;;)
+                __asm__ volatile ("wfi");
+        }
+
+        // Prepare a local copy of the new thread SP, and mark its context
+        // live on this CPU (the same thread, when it was re-picked).
+        stack::element_t* out_sp = new_thread->context_.port_.stack_ptr;
+        new_thread->context_.port_.stack_ptr = nullptr;
+
+        _smp_klock_exit ();
+
+    #else
+
+        rtos::thread* old_thread = rtos::scheduler::current_thread;

```

```

        // Save the current SP in the initial context.
@@ -851,6 +943,8 @@ namespace os
        stack::element_t* out_sp
            = rtos::scheduler::current_thread->context_.port_.stack_ptr;

+endif /* defined(OS_USE_SMP_SCHEDULER) */
+
+if defined(__ARM_ARCH_7M__) || defined(__ARM_ARCH_7EM__)

+if defined(OS_INTEGER_RTOS_CRITICAL_SECTION_INTERRUPT_PRIORITY)
@@ -886,6 +980,17 @@ namespace os

// -----

+if defined(OS_USE_SMP_SCHEDULER)
+// The kernel's CPU index (declared extern "C" by the kernel).
+extern "C" unsigned
+port_cpu_id (void)
+{
+    return os::rtos::port::scheduler::port_cpu_id ();
+}
+endif /* defined(OS_USE_SMP_SCHEDULER) */
+
+// -----
+
+using namespace os::rtos;

/**

```